

# Digital Design & Computer Arch.

## Lecture 2: Transistors, Gates, Combinational Logic

Prof. Onur Mutlu

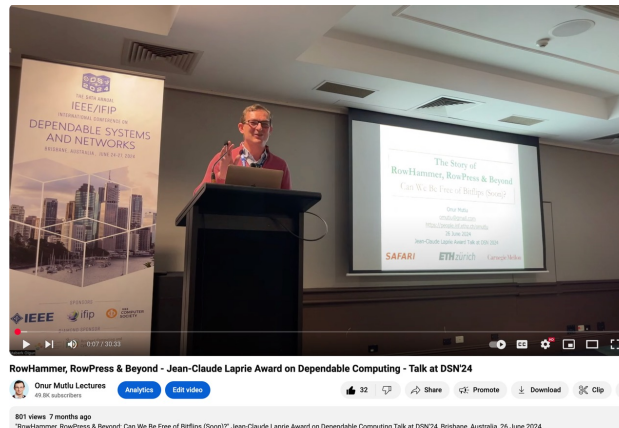
ETH Zürich

Spring 2026

20 February 2026

# Extra Credit Assignment: Talk Analysis

- The Story of RowHammer, RowPress & Beyond
- Watch and analyze this short lecture (30 minutes)
  - <https://www.youtube.com/watch?v=U1EcqXlclKU> (June 2024)

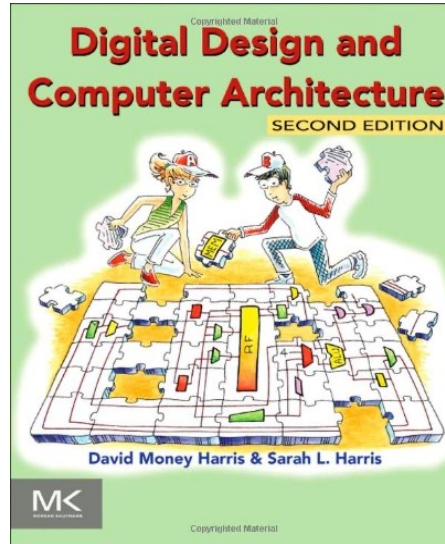


- Assignment – for 1% extra credit
  - Write a good 1-page individualized summary (no AI use)
    - What are your key takeaways? What did you learn?
    - What surprised you about the content presented? What excited you?
    - What do you think solutions should be like?
    - Submit your summary to Moodle – deadline March 21

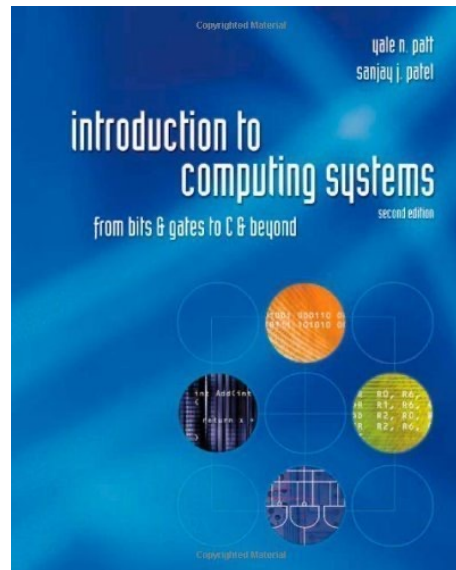
# Reading Assignments for This/Next Week

---

- Chapters 1-2 in Harris & Harris



- Chapters 1,2,3 in Patt and Patel



- Supplementary Lecture Slides on Binary Numbers

# Reading Assignments for This/Next Week

---

## ■ This week

### □ Introduction

- P&P Chapters 1 & 2 + H&H Chapter 1

### □ Combinational Logic

- P&P Chapter 3 until 3.3 + H&H Chapter 2

## ■ Next week

### □ Hardware Description Languages and Verilog

- H&H Chapter 4 until 4.3 and 4.5

### □ Sequential Logic

- P&P Chapter 3.4 until end + H&H Chapter 3 in full

## ■ Within 2-3 weeks, we will be done with

- **P&P Chapters 1-3 + H&H Chapters 1-4**

# What Will We Learn Today?

---

- Basic building blocks of modern computers
  - Transistors
  - Logic gates
- Boolean algebra
- Combinational logic circuits
- How to use Boolean algebra to represent combinational circuits
- Minimizing logic circuits

All Computers are Built Upon  
the Same Building Blocks

# Building Blocks of Modern Computers

# Transistors

# Transistors

## ■ Computers are built from very large numbers of very small (and relatively simple) structures: **transistors**

- ❑ Intel 4004, in 1971, had **2300 MOS transistors**
- ❑ Intel's Pentium IV microprocessor, 2000, was made up of more than **42 Million MOS transistors**
- ❑ Apple's M2 Max, offered for sale in 2022, is made up of more than **67 Billion MOS transistors**
- ❑ Micron's V-NAND flash memory (2022), is made up of more than **5 Trillion MOS transistors**

## ■ This lecture

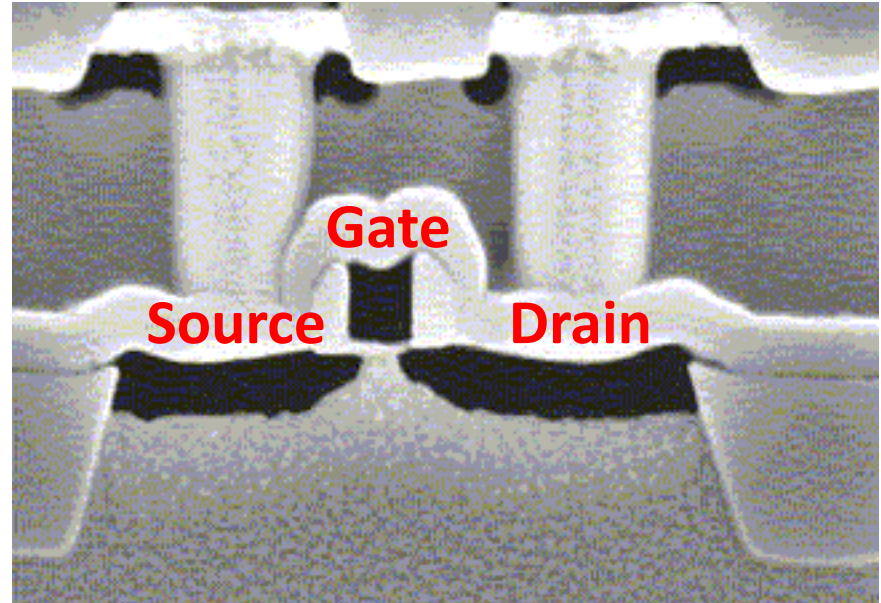
- ❑ How the MOS transistor works (as logic element)
- ❑ How transistors are connected to form logic gates
- ❑ How logic gates are interconnected to form larger units that are needed to construct a computer

|                                |
|--------------------------------|
| Problem                        |
| Algorithm                      |
| Program/Language               |
| Runtime System<br>(VM, OS, MM) |
| ISA (Architecture)             |
| Microarchitecture              |
| Logic                          |
| Devices                        |
| Electrons                      |

# MOS Transistor

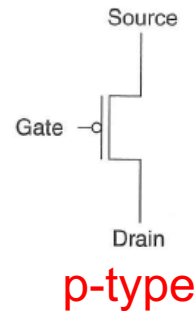
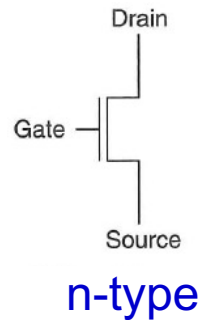
---

- By combining
  - Conductors (**M**etal)
  - Insulators (**O**xide)
  - **S**emiconductors
- We get a Transistor (MOS)
- Why is this useful?
  - We can combine many of these to realize simple logic gates
- The **electrical properties** of metal-oxide semiconductors are well **beyond** the scope of what we want to understand in this course
  - They are below our lowest level of abstraction



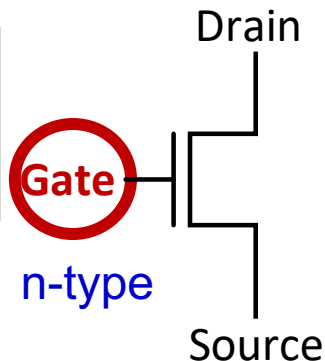
# Different Types of MOS Transistors

- There are two types of MOS transistors: **n-type** and **p-type**

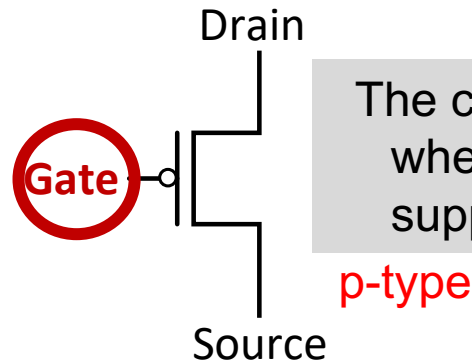


- They both operate "**logically**," very similar to the way wall switches work

The circuit is closed when the gate is supplied with 3V

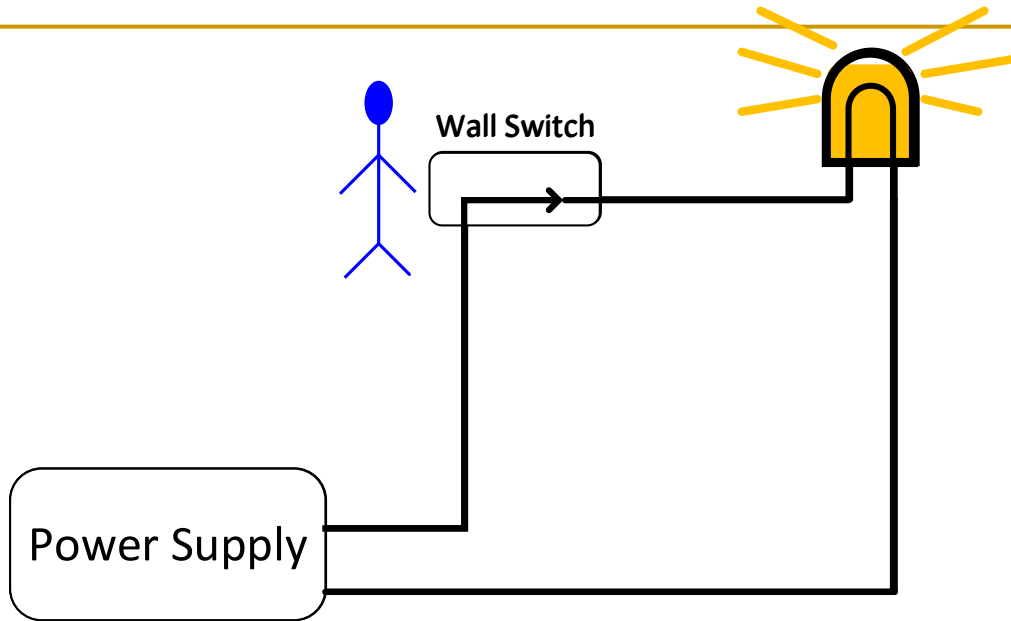


The circuit is closed when the gate is supplied with 0V



# How Does a Wall Switch Work?

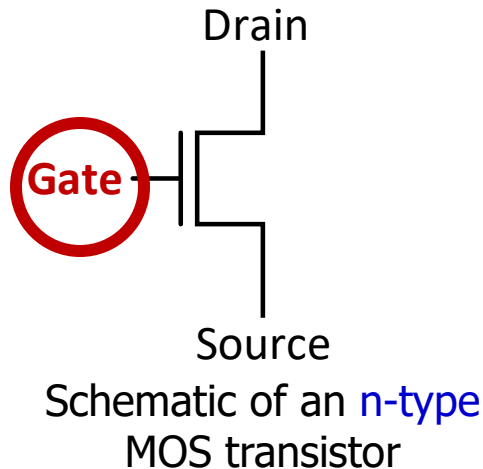
---



- ❑ In order for the lamp to glow, **electrons must flow**
- ❑ In order for electrons to flow, there must be a **closed circuit** from the power supply to the lamp and back to the power supply
- ❑ The lamp can be **turned on and off** by simply manipulating the wall switch to make or break the closed circuit

# Transistor Works As a Switch

- Instead of the wall switch, we could use an **n-type** or a **p-type** MOS transistor to make or break the closed circuit



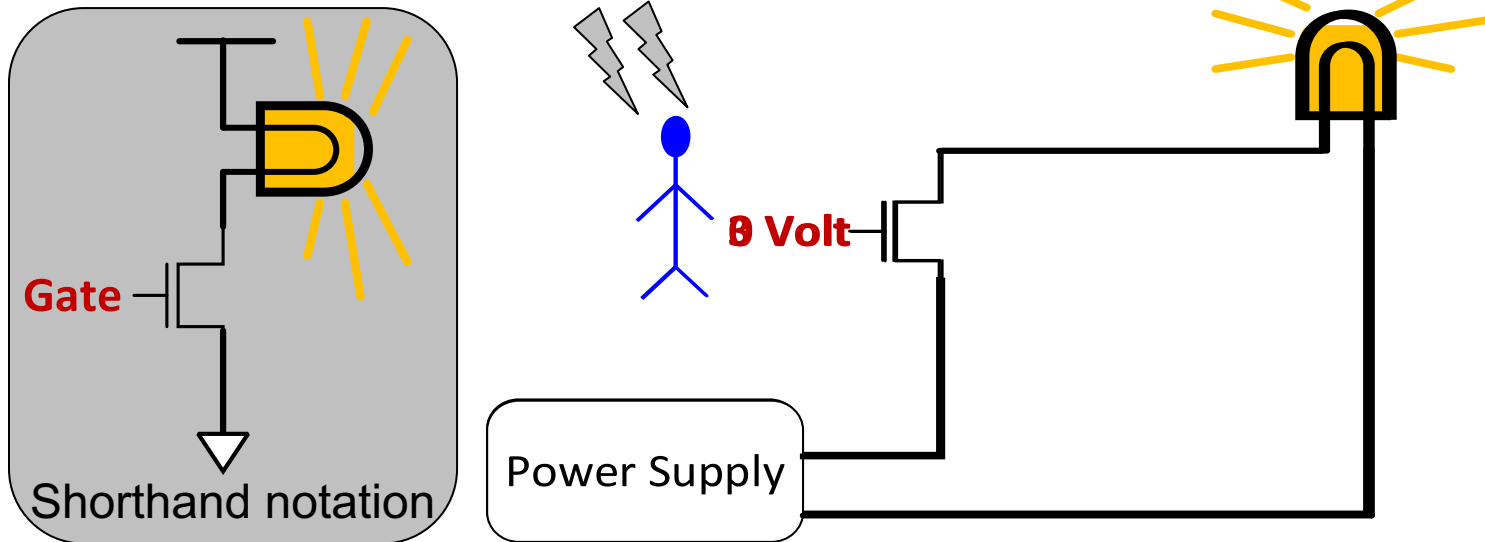
If the gate of an **n-type** transistor is supplied with a **high** voltage, the connection from source to drain acts like a piece of wire (i.e., the circuit is closed)

Depending on the technology, high voltage can range from 0.3V to 3V

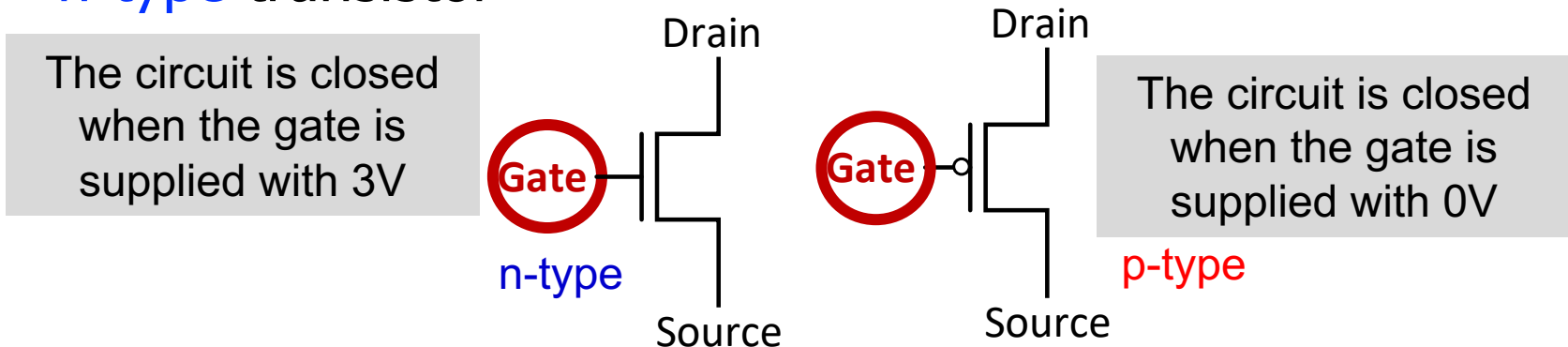
If the gate of the **n-type** transistor is supplied with **zero** voltage, the connection between the source and drain is **broken** (i.e., the circuit is open)

# Abstraction: Transistor As a Switch

- **n-type** transistor in a circuit with a battery and a bulb



- **p-type** transistor works in exactly the opposite fashion from **n-type** transistor

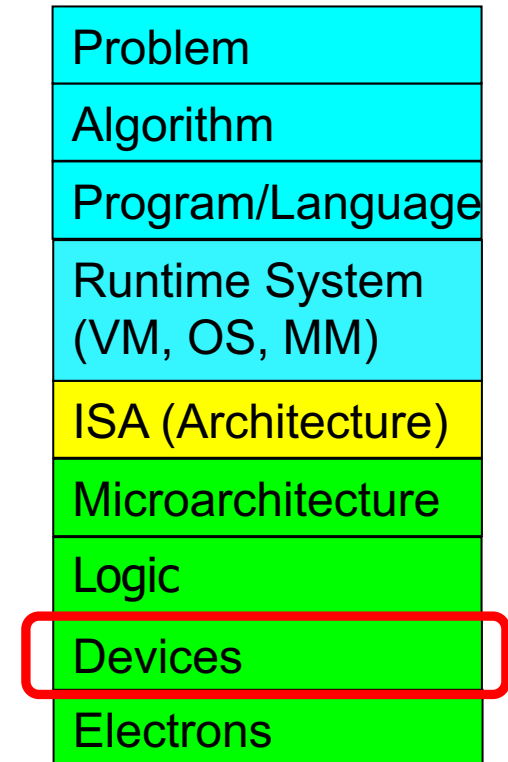


# Logic Gates

# One Level Higher in the Abstraction

---

- **Now, we know how a MOS transistor works**
- How do we build logic structures out of MOS transistors?
- We construct basic logical units out of individual MOS transistors
- These **logical units** are called **logic gates**
  - They implement simple **Boolean** functions

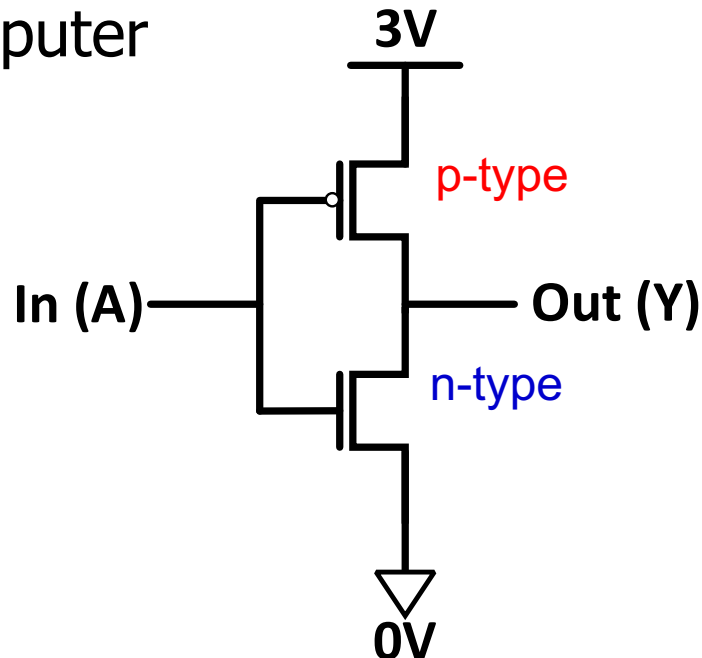


# Making Logic Blocks Using CMOS Technology

- Modern computers use both **n-type** and **p-type** transistors, i.e. Complementary MOS (**CMOS**) technology

$$\text{nMOS} + \text{pMOS} = \text{CMOS}$$

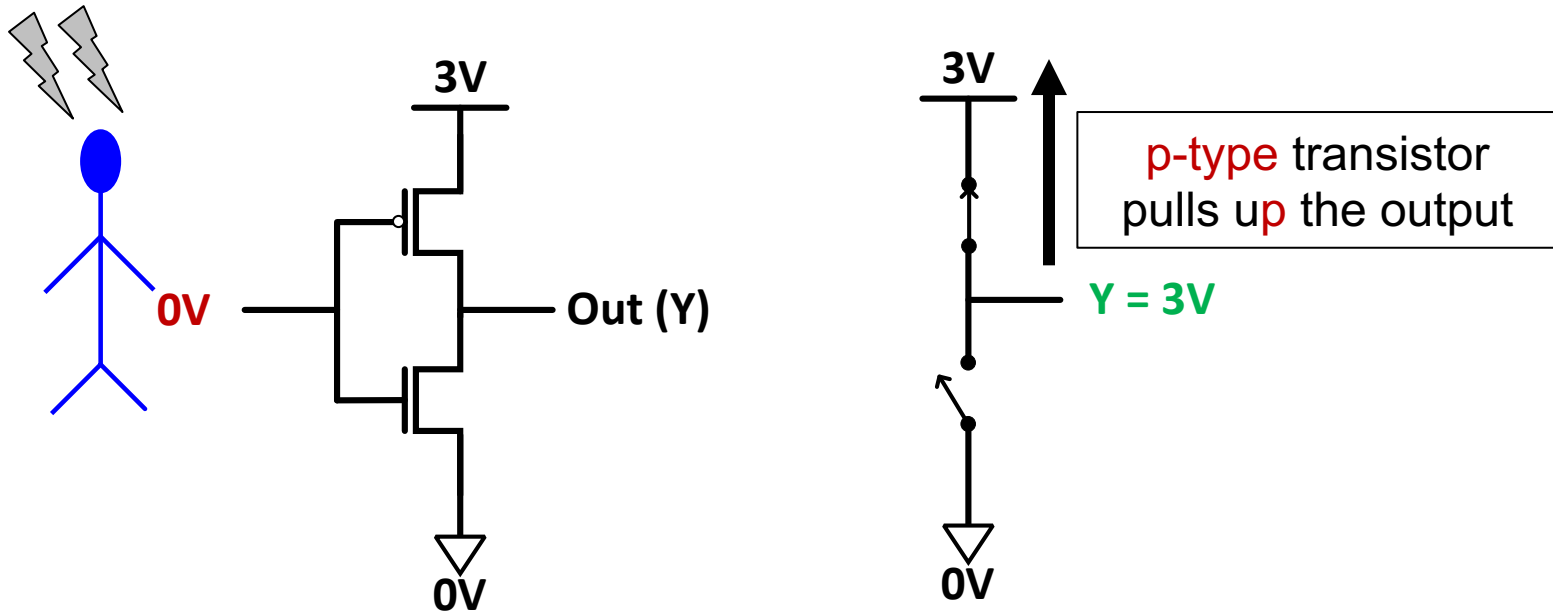
- The simplest logic structure that exists in a modern computer



What does this circuit do?

# Functionality of Our CMOS Circuit

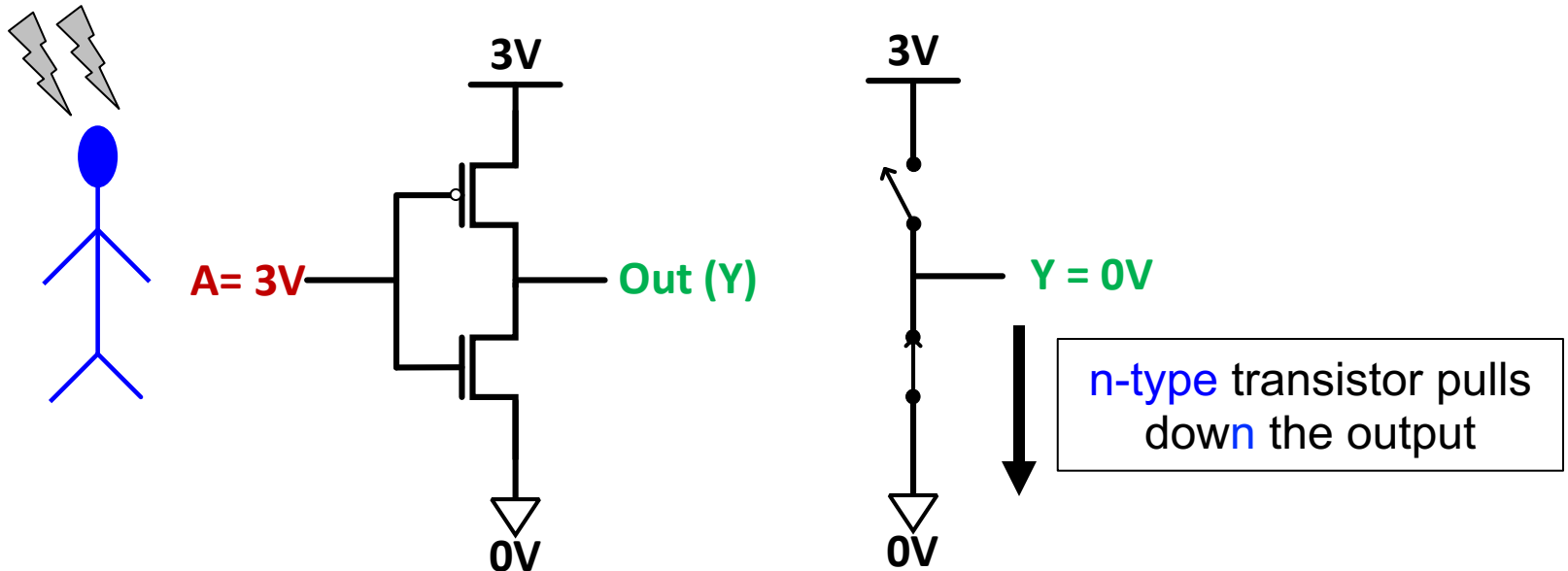
What happens when the input is connected to 0V?



p-type transistors are good at pulling up the voltage

# Functionality of Our CMOS Circuit

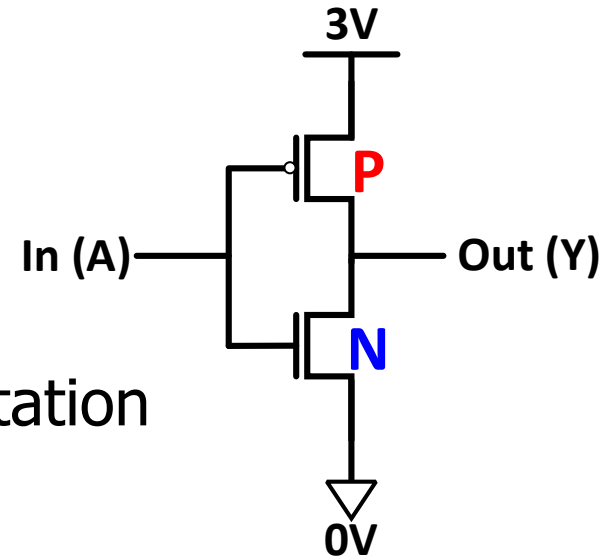
What happens when the input is connected to 3V?



n-type transistors are good at pulling down the voltage

# CMOS NOT Gate (Inverter)

- This is the **CMOS NOT Gate**
- Why do we call it NOT?
  - If  $A = 0V$  then  $Y = 3V$
  - If  $A = 3V$  then  $Y = 0V$
- **Digital circuit:** one possible interpretation
  - Interpret **0V** as logical (binary) **0** value
  - Interpret **3V** as logical (binary) **1** value

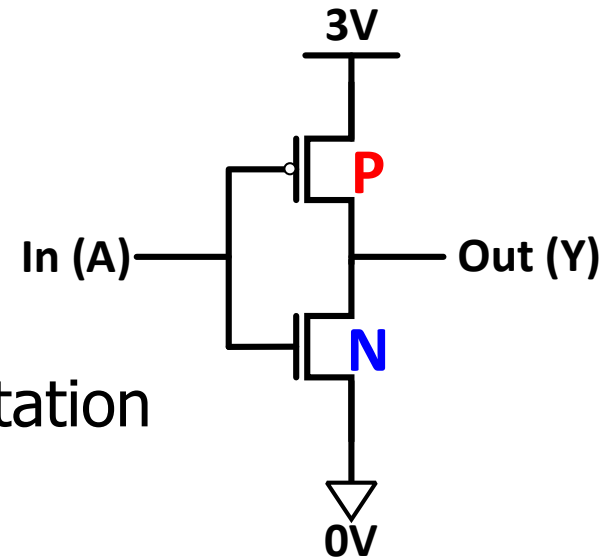


| A | P   | N   | Y |
|---|-----|-----|---|
| 0 | ON  | OFF | 1 |
| 1 | OFF | ON  | 0 |

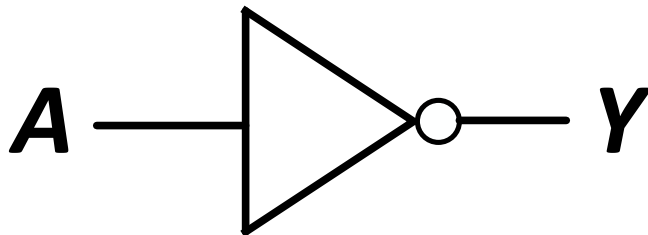
$$Y = \overline{A}$$

# CMOS NOT Gate (Inverter)

- This is actually the CMOS NOT Gate
- Why do we call it NOT?
  - If  $A = 0V$  then  $Y = 3V$
  - If  $A = 3V$  then  $Y = 0V$
- **Digital circuit:** one possible interpretation
  - Interpret  $0V$  as logical (binary)  $0$  value
  - Interpret  $3V$  as logical (binary)  $1$  value



$$Y = \overline{A}$$



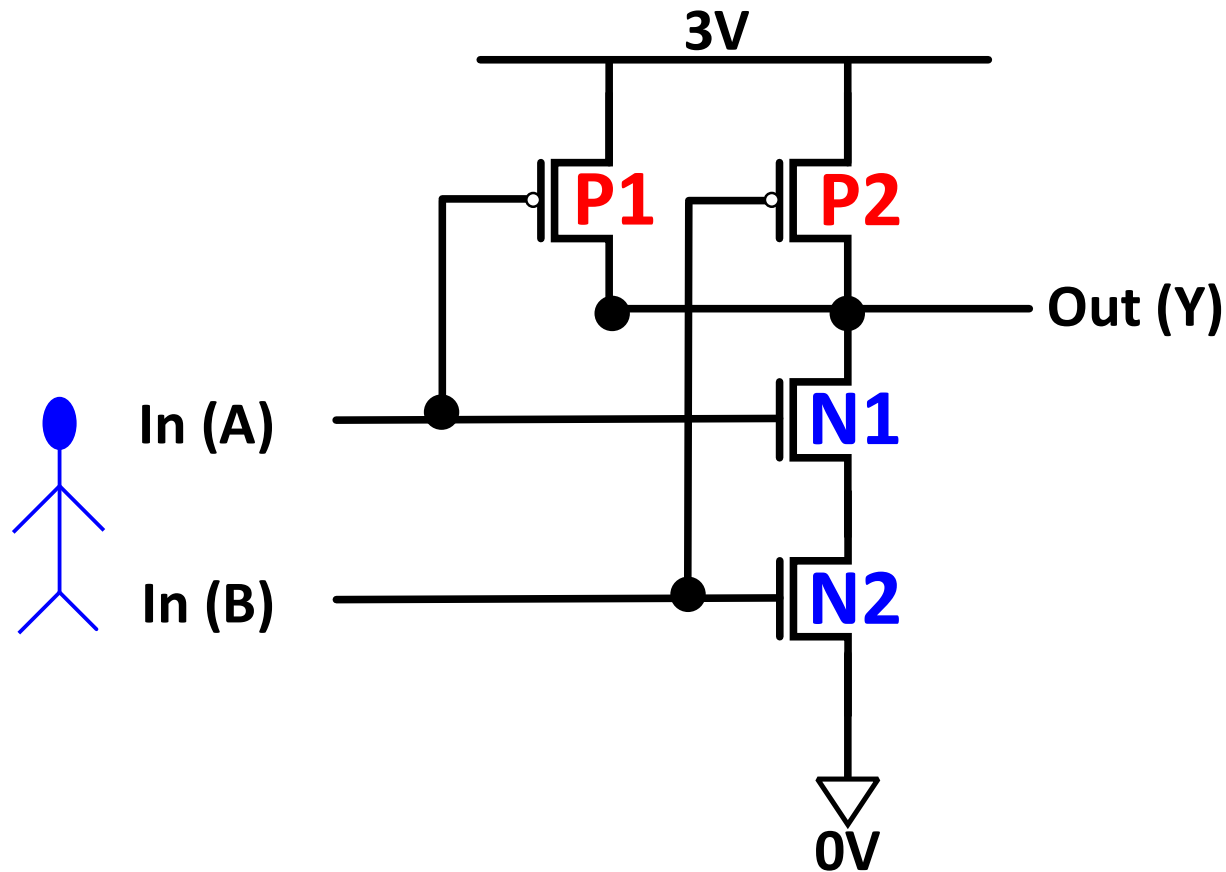
We call this a **NOT** gate  
or an **inverter**  
(bubble indicates inversion)

**Truth table:** shows the logical output of the circuit for each possible input

| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

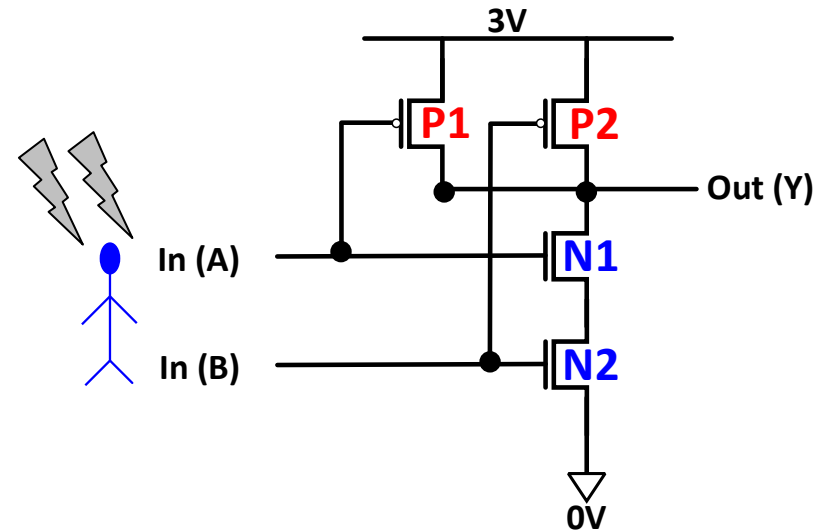
# Another CMOS Gate: What Is This?

- Let's build more complex gates!



# CMOS NAND Gate

- Let's build more complex gates!



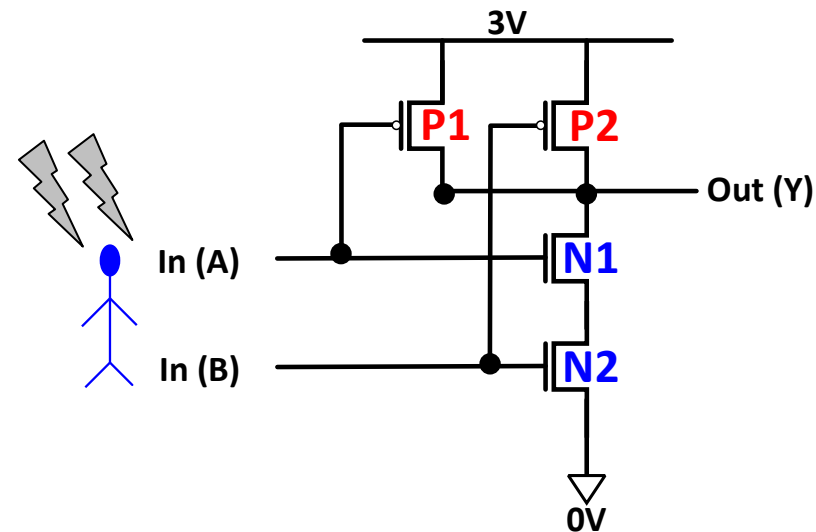
$$Y = \overline{A \cdot B} = \overline{AB}$$

| A | B | P1  | P2  | N1  | N2  | Y |
|---|---|-----|-----|-----|-----|---|
| 0 | 0 | ON  | ON  | OFF | OFF | 1 |
| 0 | 1 | ON  | OFF | OFF | ON  | 1 |
| 1 | 0 | OFF | ON  | ON  | OFF | 1 |
| 1 | 1 | OFF | OFF | ON  | ON  | 0 |

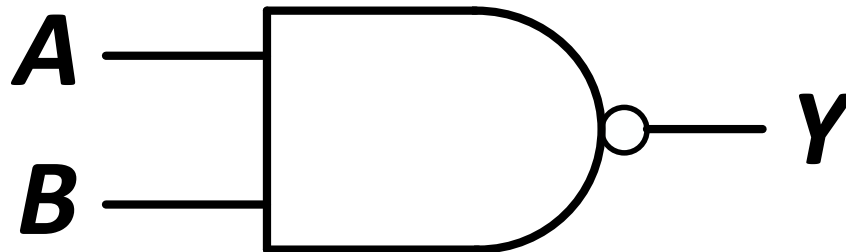
- P1 and P2 are in **parallel**; **only one must be ON to pull up the output to 3V**
- N1 and N2 are connected in **series**; **both must be ON to pull down the output to 0V**

# CMOS NAND Gate

- Let's build more complex gates!



$$Y = \overline{A \cdot B} = \overline{AB}$$



| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

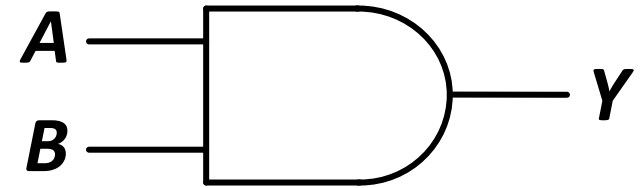
We call this a **NAND** gate  
(bubble indicates inversion)

# CMOS AND Gate

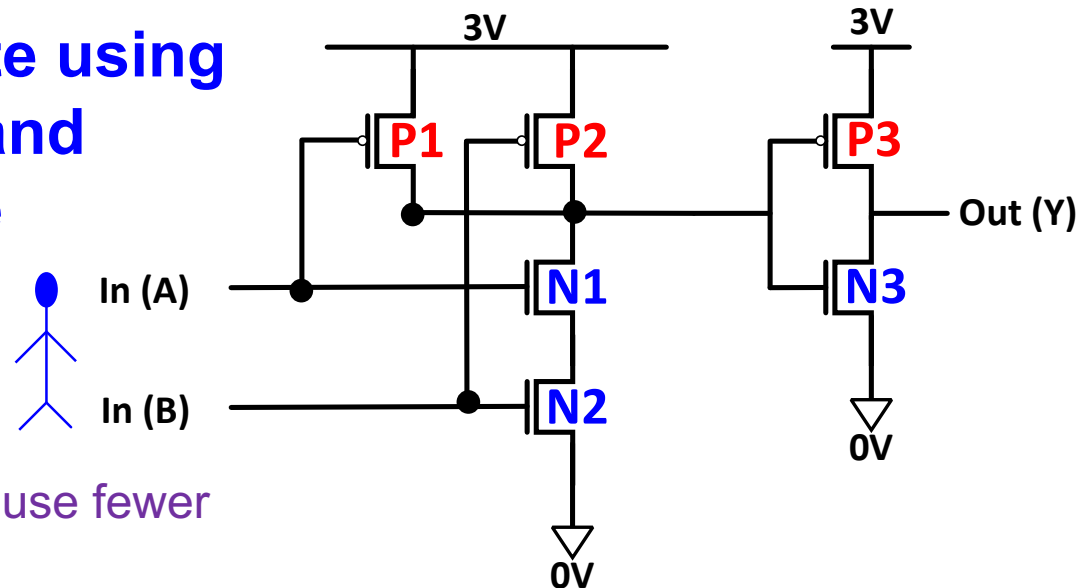
- How can we make an AND gate?

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

$$Y = A \cdot B = AB$$

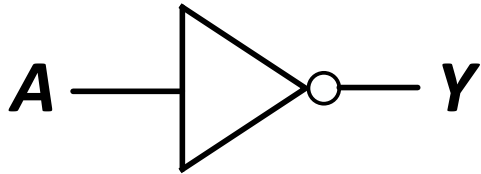


We make an **AND** gate using  
one **NAND** gate and  
one **NOT** gate

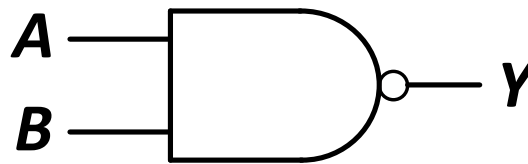


Food for thought: Can we not use fewer transistors for the AND gate?

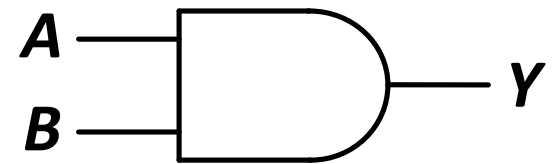
# CMOS NOT, NAND, AND Gates



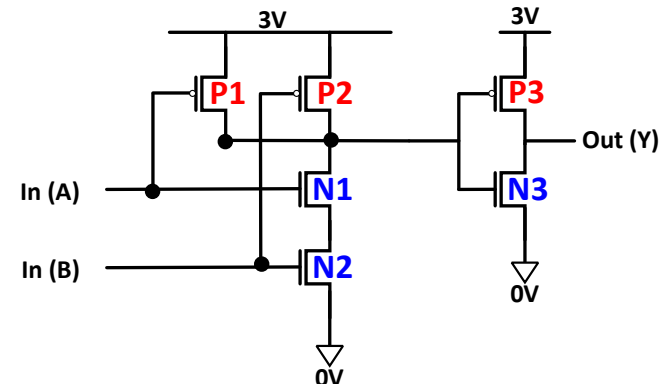
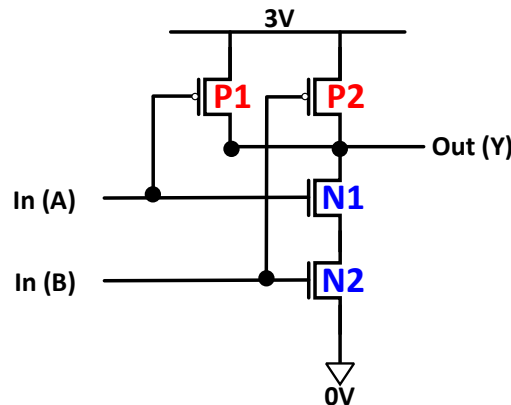
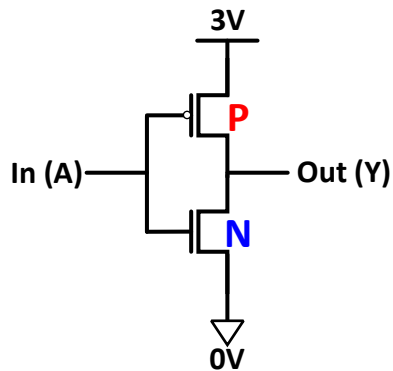
| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |



| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

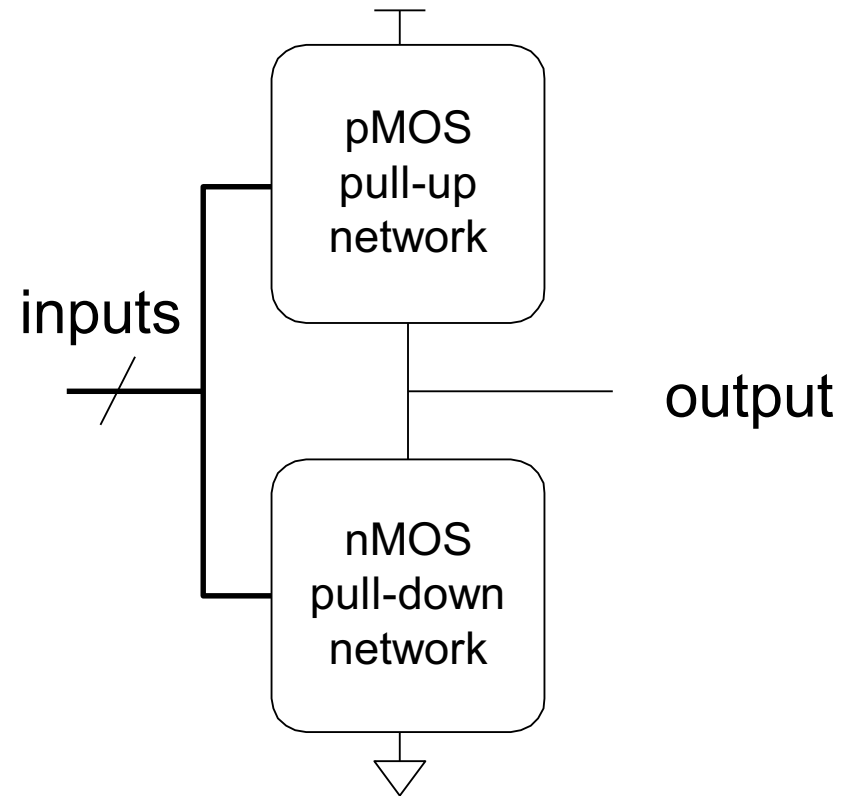


| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |



# General CMOS Gate Structure

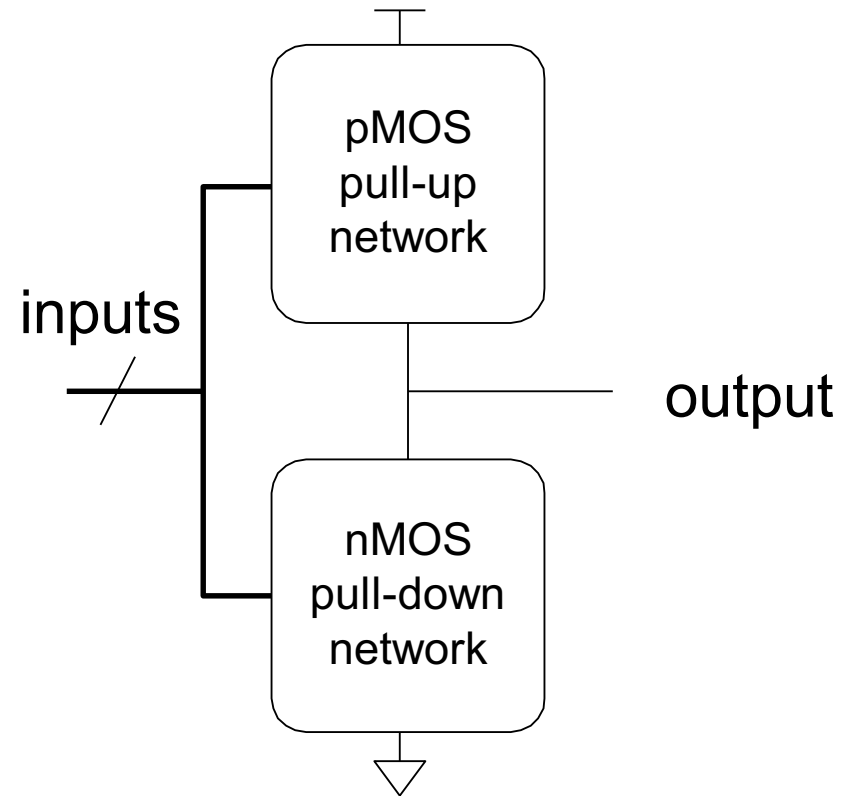
- The general form used to construct any inverting logic gate, such as: NOT, NAND, or NOR
  - The networks may consist of transistors in series or in parallel
  - When transistors are in **parallel**, the network is **ON** if **one** of the transistors is **ON**
  - When transistors are in **series**, the network is **ON** only if **all** transistors are **ON**



pMOS transistors are used for pull-up  
nMOS transistors are used for pull-down

# General CMOS Gate Structure (II)

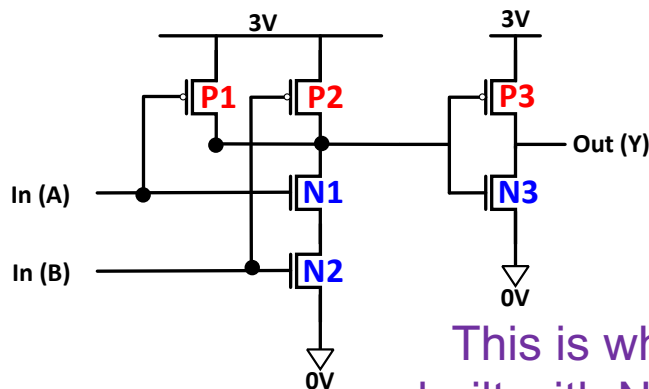
- Exactly one network should be ON, and the other network should be OFF at any given time
  - If both networks are ON at the same time, there is a **short circuit** → likely incorrect operation
  - If both networks are OFF at the same time, the output is **floating** → undefined



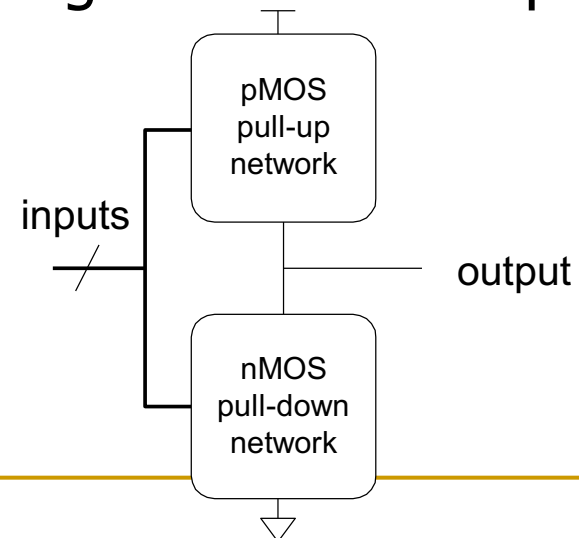
**pMOS** transistors are used for pull-up  
**nMOS** transistors are used for pull-down

# Digging Deeper: Why This Structure?

- MOS transistors are **imperfect** switches
- pMOS transistors pass 1's well but 0's poorly (holes carry charge)
- nMOS transistors pass 0's well but 1's poorly (electrons carry charge)
- **p**MOS transistors are good at "pulling up**p**" the output
- **n**MOS transistors are good at "pulling down**n**" the output



This is why AND is built with NAND + NOT



# Digging Deeper: Latency

- Which one is slower?
  - ❑ Transistors in series
  - ❑ Transistors in parallel
- Series connections are slower than parallel connections
  - ❑ More resistance on the wire
- How do you alleviate this latency?
  - ❑ See H&H Section 1.7.8 for an example:  
**pseudo-nMOS Logic**

Used in the past when pMOS transistors  
could not be fabricated well

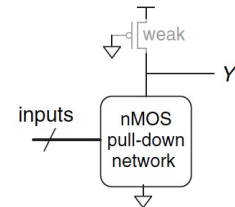


Figure 1.39 Generic pseudo-nMOS gate

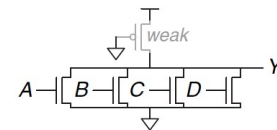


Figure 1.40 Pseudo-nMOS four-input NOR gate

# Digging Deeper: Power Consumption

---

## ■ **Dynamic Power Consumption**

- Power used to charge capacitance as signals change ( $0 \leftrightarrow 1$ )
- $C * V^2 * f$ 
  - $C$  = capacitance of the circuit (wires and gates)
  - $V$  = supply voltage
  - $f$  = charging frequency of the capacitor

## ■ **Static Power Consumption**

- Power used when signals do not change
- $V * I_{\text{leakage}}$ 
  - supply voltage \* leakage current

## ■ **Energy Consumption**

- Power \* Time

# Common Logic Gates

**Buffer**



| A | Z |
|---|---|
| 0 | 0 |
| 1 | 1 |

**AND**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**OR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**XOR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Inverter**



| A | Z |
|---|---|
| 0 | 1 |
| 1 | 0 |

**NAND**



| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**NOR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**XNOR**

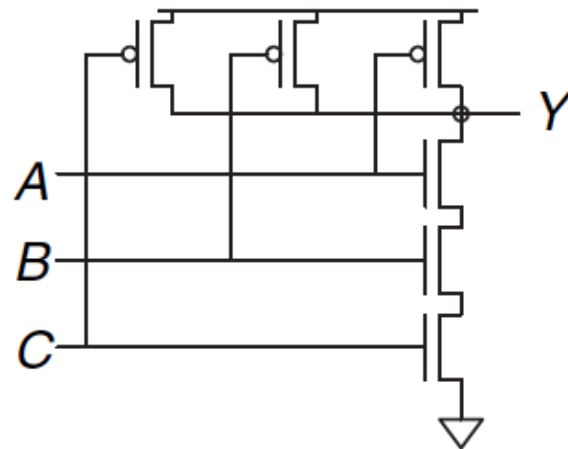


| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

# Larger Gates

---

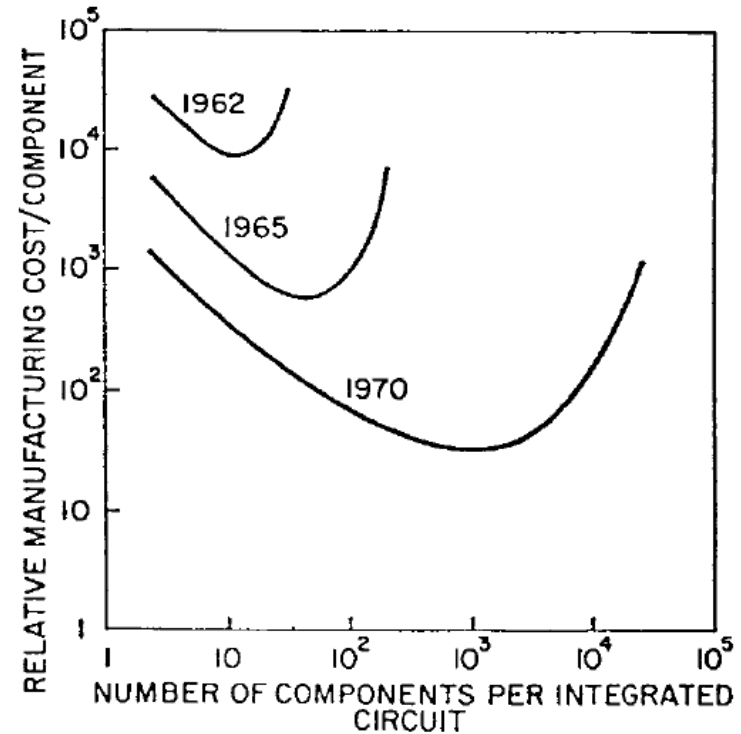
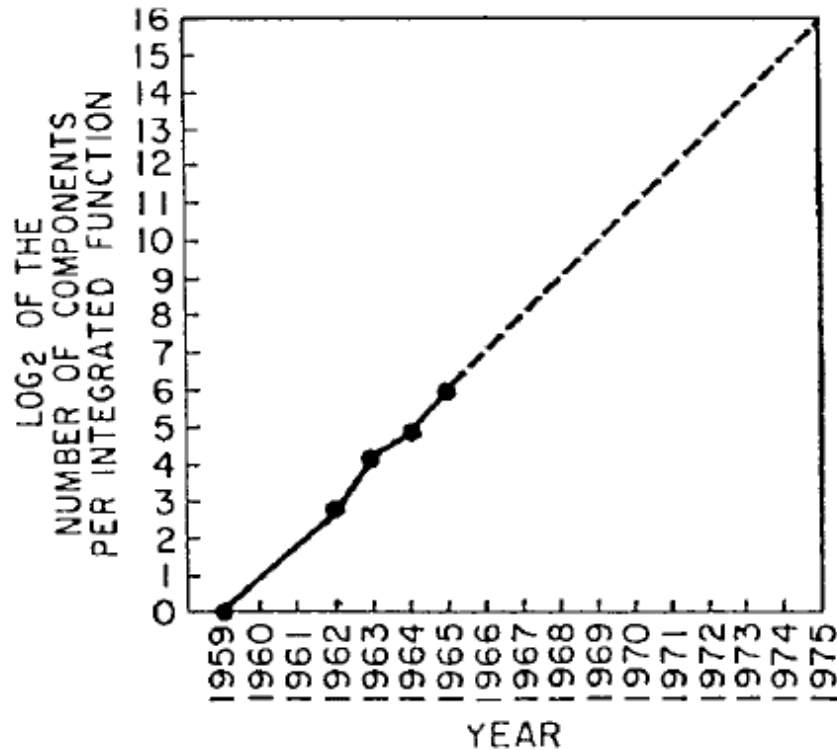
- We can extend the gates to more than 2 inputs
- Example: 3-input AND gate, 10-input NOR gate
- See your readings



**Figure 1.35** Three-input NAND gate schematic

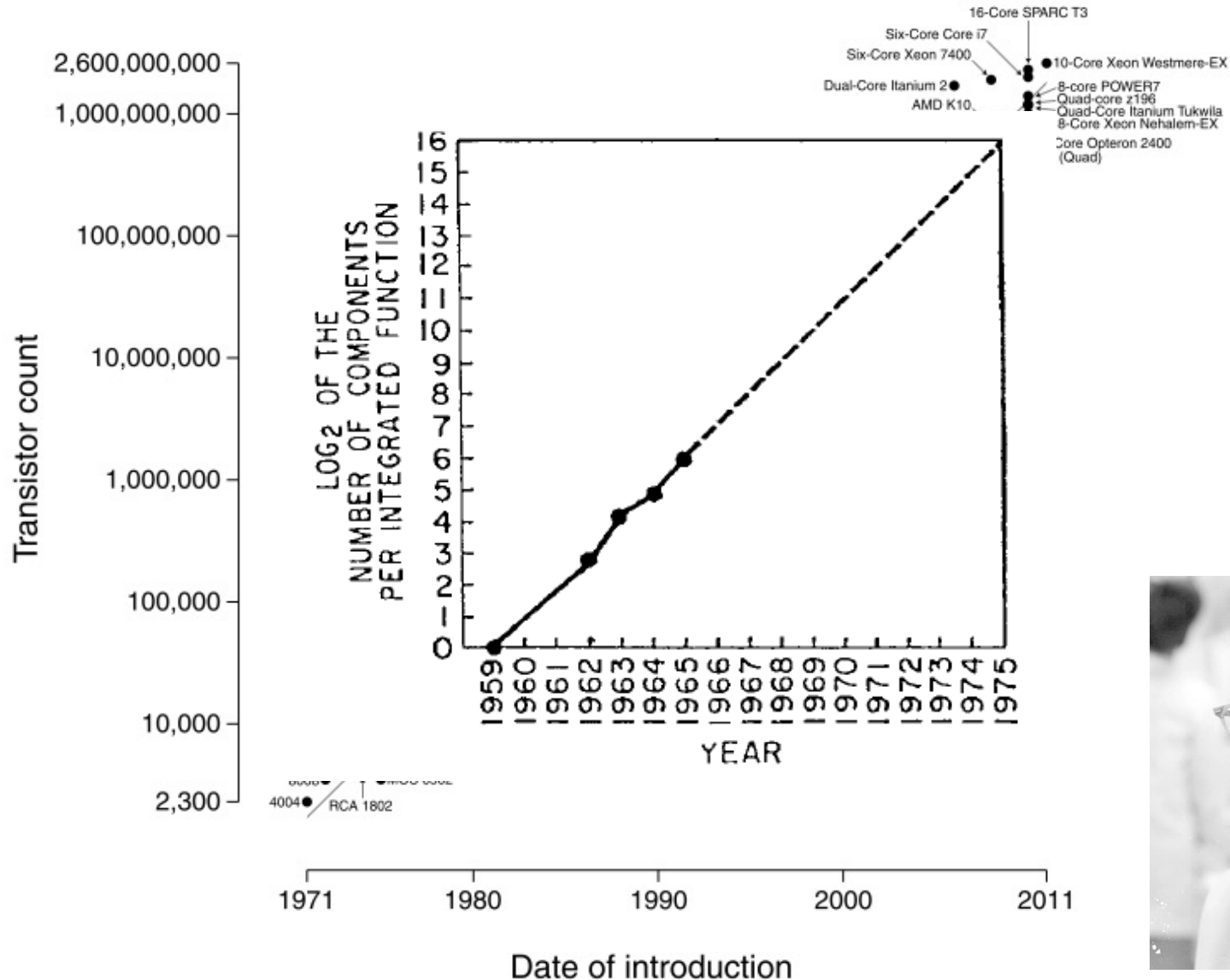
# Aside: Moore's Law: Enabler of Many Gates on a Chip

# An Enabler: Moore's Law



Moore, “Cramming more components onto integrated circuits,”  
Electronics Magazine, 1965. Component counts double every other year

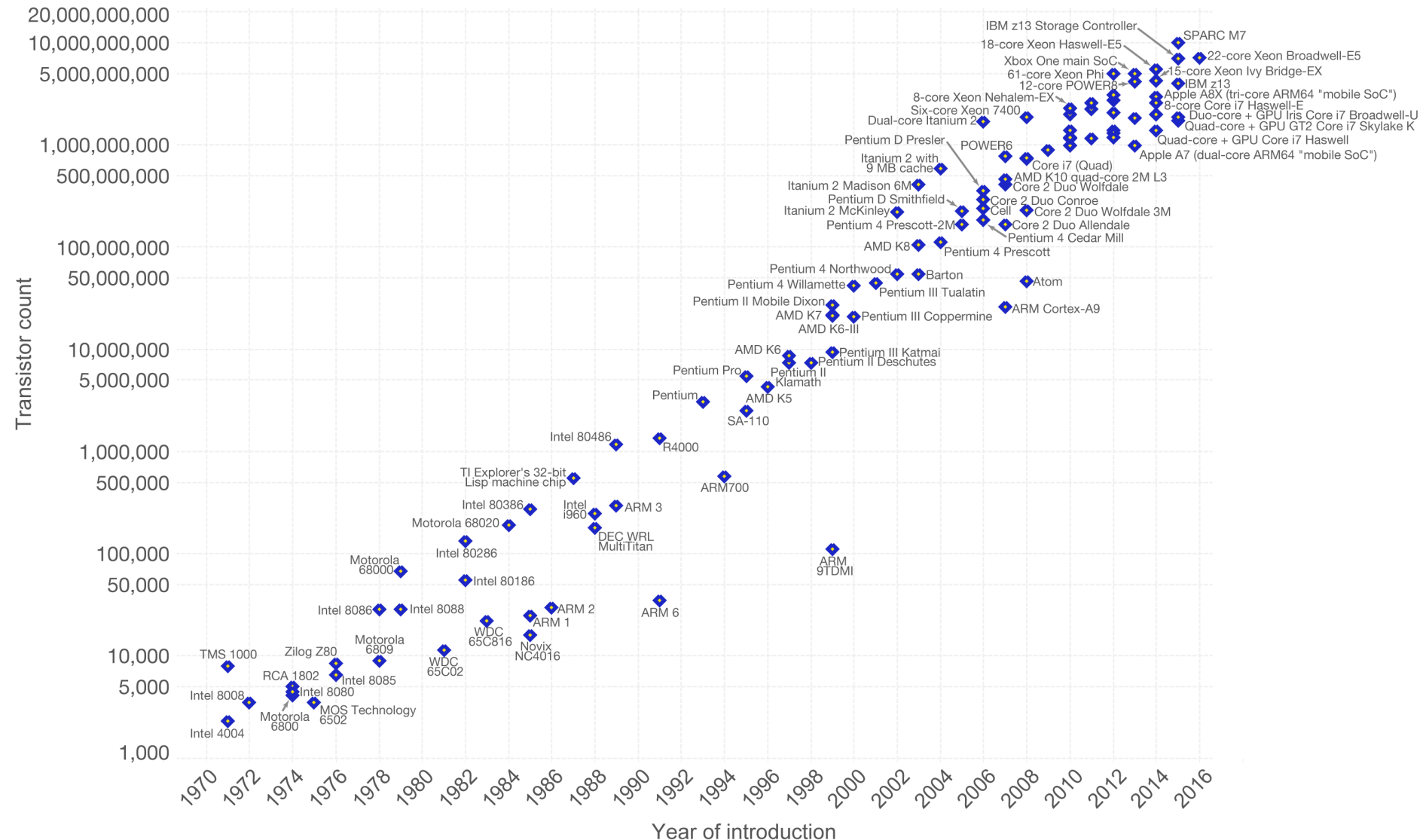
# Microprocessor Transistor Counts 1971-2011 & Moore's Law



Number of transistors on an integrated circuit doubles ~ every two years



Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are strongly linked to Moore's law.



Data source: Wikipedia ([https://en.wikipedia.org/wiki/Transistor\\_count](https://en.wikipedia.org/wiki/Transistor_count))

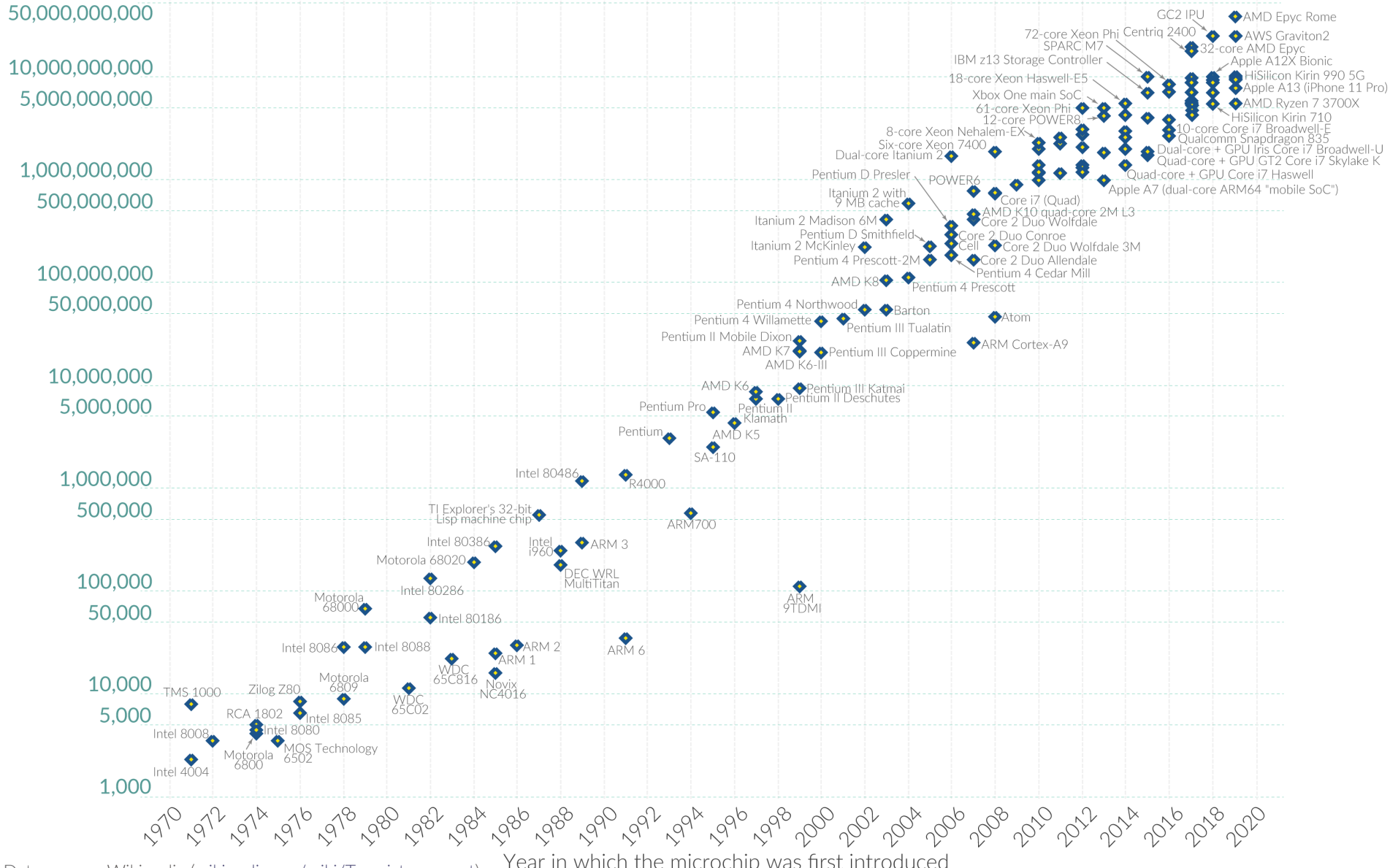
The data visualization is available at [OurWorldinData.org](https://ourworldindata.org). There you find more visualizations and research on this topic.

Licensed under [CC-BY-SA](#) by the author Max Roser.

# Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

## Transistor count



# Recommended Reading

---

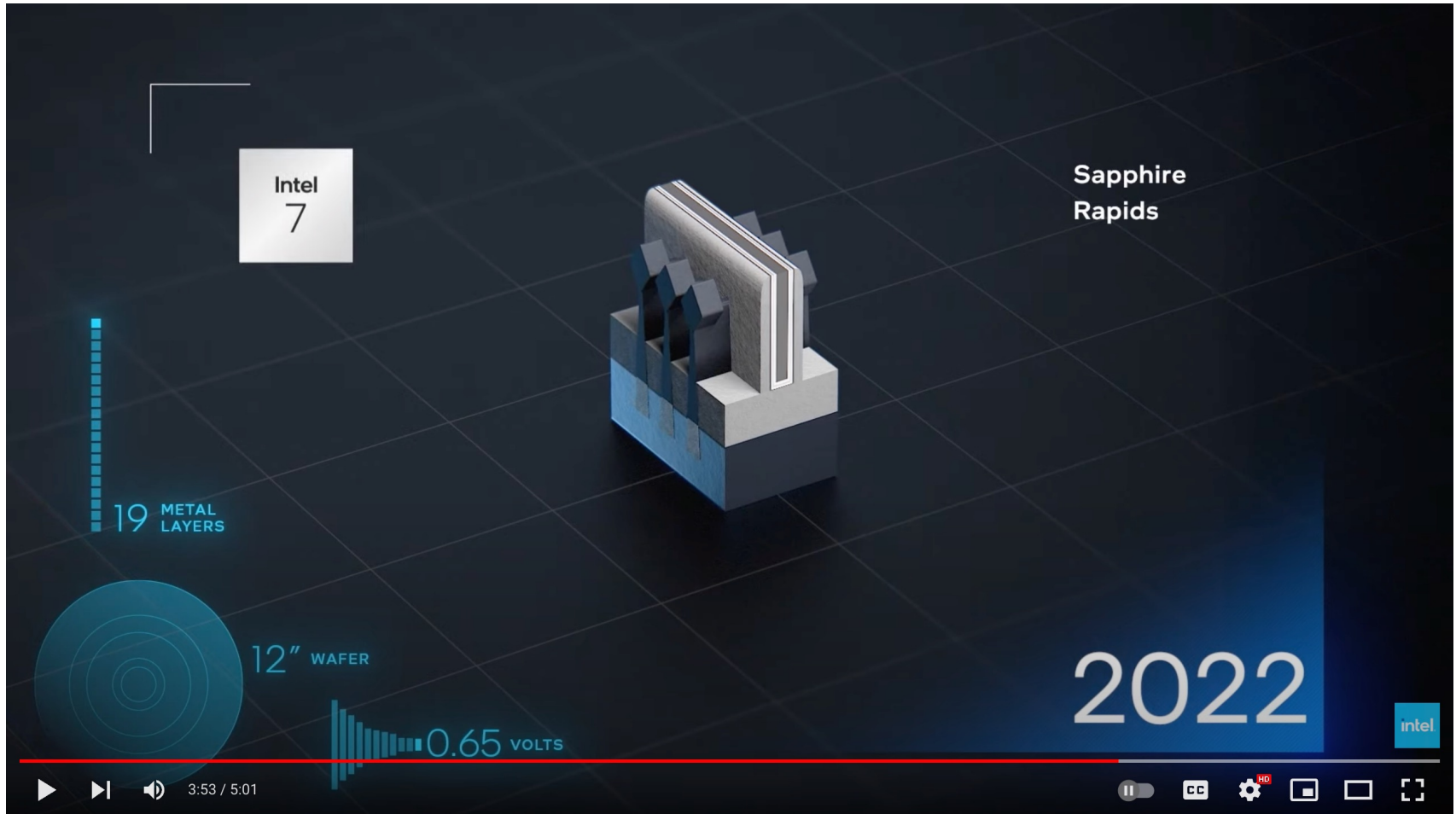
- Moore, “Cramming more components onto integrated circuits,” Electronics Magazine, 1965.
- Only 3 pages
- A quote:  
*"With unit cost falling as the number of components per circuit rises, by 1975 economics may dictate squeezing as many as 65 000 components on a single silicon chip."*
- Another quote:  
*"Will it be possible to remove the heat generated by tens of thousands of components in a single silicon chip?"*

# How Do We Keep Moore's Law: Innovation

---

- **Manufacturing smaller transistors/structures**
  - Some structures are already a few atoms in size
- **Using materials with better properties**
  - Copper instead of Aluminum (better conductor)
  - Hafnium Oxide, air for insulators
  - Making sure all materials are compatible is the challenge
- **Enabling precision manufacturing**
  - Extreme ultraviolet (EUV) light to pattern <10nm structures
- **Creating new device technologies**
  - FinFET, Gate All Around transistor, Single Electron Transistor...

# A 5-Minute Video on Transistor Innovation



Evolution of Transistor Innovation

12,441 views • Feb 22, 2022

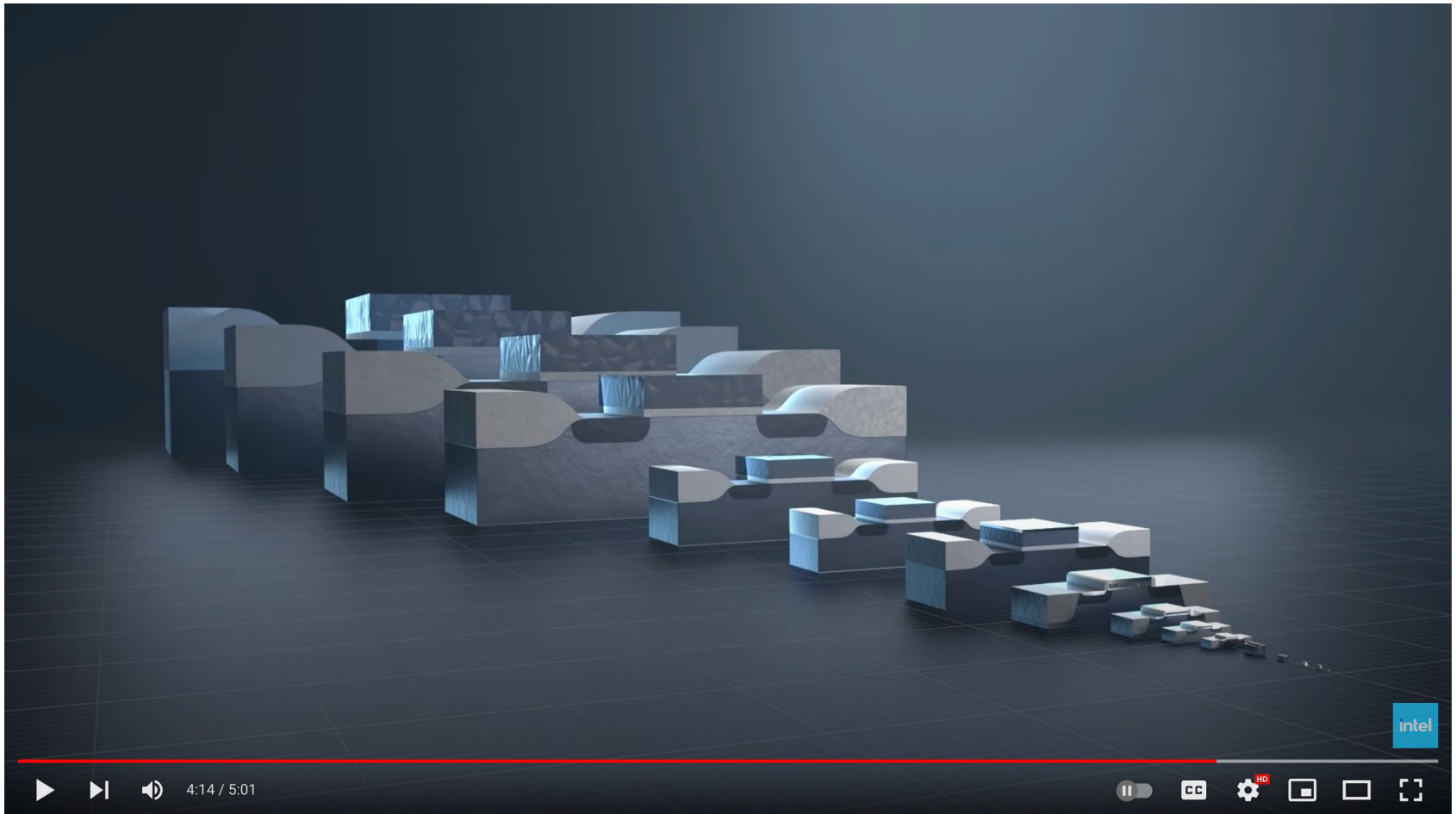
628 DISLIKE SHARE CLIP SAVE ...



SUBSCRIBE

<https://www.youtube.com/watch?v=Z7M8etXUEUU>

# A 5-Minute Video on Transistor Innovation



Evolution of Transistor Innovation

12,460 views • Feb 22, 2022

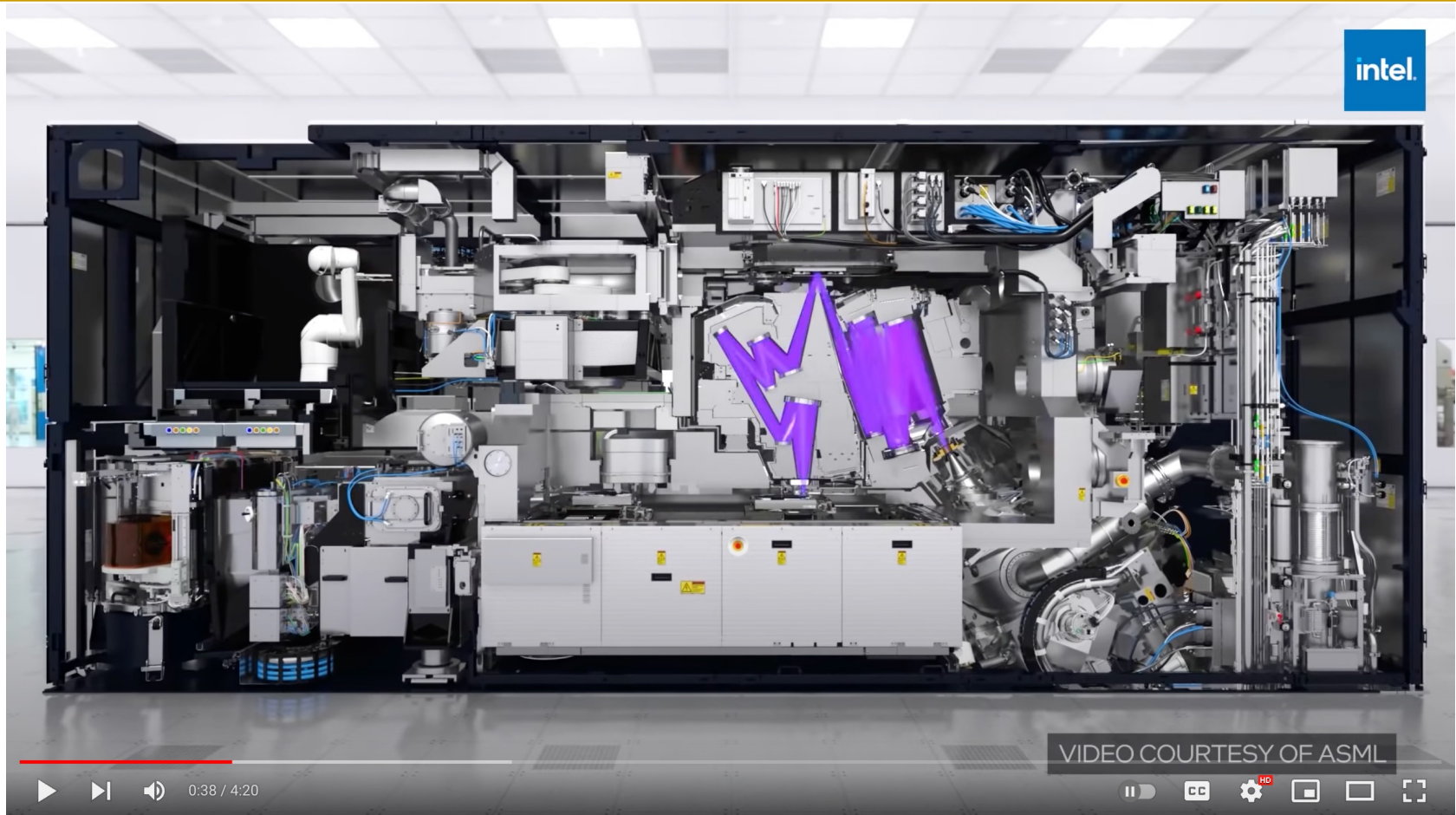
628 DISLIKE SHARE CLIP SAVE ...



SUBSCRIBE

<https://www.youtube.com/watch?v=Z7M8etXUEUU>

# Enabling Manufacturing Tech: EUV



#EUV #chip #Intel

Behind this Door: Learn about EUV, Intel's Most Precise, Complex Machine

78,354 views • Dec 21, 2021

LIKE DISLIKE SHARE CLIP SAVE ...

intel Intel Newsroom  
25.9K subscribers

SUBSCRIBE

<https://www.youtube.com/watch?v=Jv40Viz-KTc>

# Enabling Manufacturing Tech: EUV

## Extreme ultraviolet lithography

🌐 11 languages ▾

Article [Talk](#)

[Read](#) [Edit](#) [View history](#)

From Wikipedia, the free encyclopedia

**Extreme ultraviolet lithography** (also known as **EUV** or **EUVL**) is an optical [lithography](#) technology used in [semiconductor device fabrication](#) to make [integrated circuits](#) (ICs). It uses [extreme ultraviolet](#) (EUV) wavelengths, roughly spanning a 2% [FWHM](#) bandwidth near *13.5 nm* (13.36nm - 13.65nm at 50% power), using a laser-pulsed *tin plasma*, to produce a pattern by using a reflective photomask to expose a substrate covered by [photoresist](#). [EUV](#) (10-124nm) is the band between [X-Rays](#) (0.1-10nm) and overlapping slightly with [UVC](#) (100-280nm). It is currently applied only in the most advanced semiconductor device fabrication.

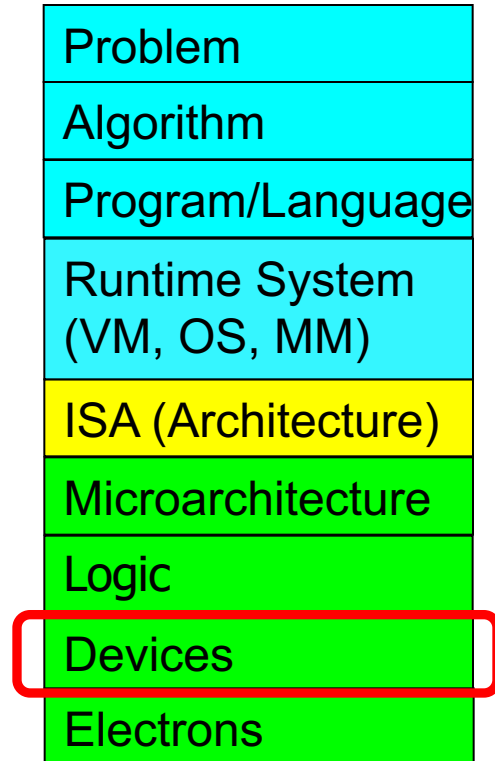
As of 2022, [ASML Holding](#) is the only company who produces and sells EUV systems for chip production, targeting 5 nm and 3 nm. At the 2019 [International Electron Devices Meeting](#) (IEDM), TSMC reported use of EUV for 5 nm in contact, via, metal line, and cut layers, where the cuts can be applied to fins, gates or metal lines.<sup>[1][2]</sup> At IEDM 2020, TSMC reported their 5 nm minimum metal pitch to be reduced 30% from that of 7 nm,<sup>[3]</sup> which was 40 nm.<sup>[4]</sup> Samsung's 5 nm is lithographically the same design rule as 7 nm, with a minimum metal pitch of 36 nm.<sup>[5]</sup>

### History [\[ edit \]](#)

In the 1960s, visible light was used for IC-production, with wavelengths as small as 435 [nm](#) (mercury "g line"). Later [UV](#) light was used, with wavelength of at first 365nm (mercury "i line"), then excimer wavelengths first of 248 nm ([krypton fluoride laser](#)) and then **193** nm ([argon fluoride laser](#)), which was called [deep UV](#). The next step, going even smaller, was dubbed Extreme UV or EUV. The EUV technology was considered impossible by many. EUV is absorbed by glass and even air, so instead of using lenses, as before, to focus the beams of light, mirrors in a vacuum would be needed and a reliable production of EUV was also problematic. The then leading producers of steppers, [Japanese](#) companies [Canon](#) and [Nikon](#) gave up trying. And some even predicted the end of [Moore's law](#).

# Innovation At the Bottom Enables Computing

---



# Historical: Opportunities at the Bottom

---

## There's Plenty of Room at the Bottom

---

From Wikipedia, the free encyclopedia

**"There's Plenty of Room at the Bottom: An Invitation to Enter a New Field of Physics"** was a lecture given by physicist [Richard Feynman](#) at the annual [American Physical Society](#) meeting at [Caltech](#) on December 29, 1959.<sup>[1]</sup> Feynman considered the possibility of direct manipulation of individual atoms as a more powerful form of synthetic chemistry than those used at the time. Although versions of the talk were reprinted in a few popular magazines, it went largely unnoticed and did not inspire the conceptual beginnings of the field. Beginning in the 1980s, nanotechnology advocates cited it to establish the scientific credibility of their work.

# Historical: Opportunities at the Bottom (II)

---

## There's Plenty of Room at the Bottom

---

From Wikipedia, the free encyclopedia

Feynman considered some ramifications of a general ability to manipulate matter on an atomic scale. He was particularly interested in the possibilities of denser computer circuitry, and microscopes that could see things much smaller than is possible with scanning electron microscopes. These ideas were later realized by the use of the scanning tunneling microscope, the atomic force microscope and other examples of scanning probe microscopy and storage systems such as Millipede, created by researchers at IBM.

Feynman also suggested that it should be possible, in principle, to make nanoscale machines that "arrange the atoms the way we want", and do chemical synthesis by mechanical manipulation.

He also presented the possibility of "swallowing the doctor", an idea that he credited in the essay to his friend and graduate student Albert Hibbs. This concept involved building a tiny, swallowable surgical robot.

# Extra Assignment: Moore's Law (I)

---

- **Paper review**
- G.E. Moore. "Cramming more components onto integrated circuits," Electronics magazine, 1965
  
- **Optional Assignment for Your Benefit**
  - **Write a 1-page individual review**
    - What are your key takeaways? What did you learn?
    - What surprised you about the content presented? What excited you?
    - How do you think Moore's law evolved since the paper was written?
    - What do you think (future) solutions should be like?

# Combinational Logic Circuits

# We Can Now Build Logic Circuits

---

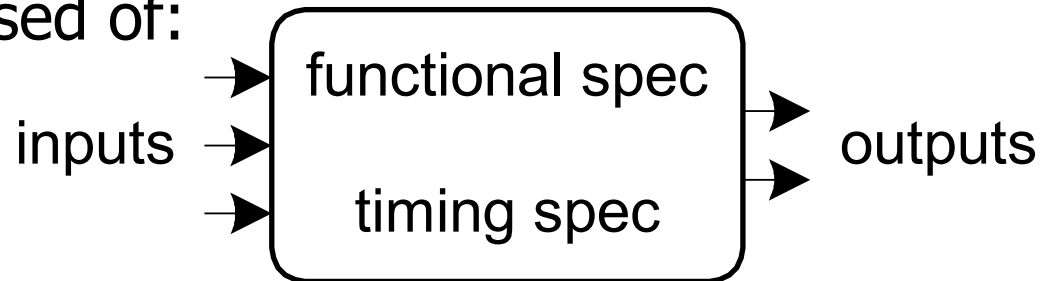
Now, we understand the workings of the basic logic gates

What is our next step?

Build some of the logic structures that are important components of the microarchitecture of a computer

- A logic circuit is composed of:

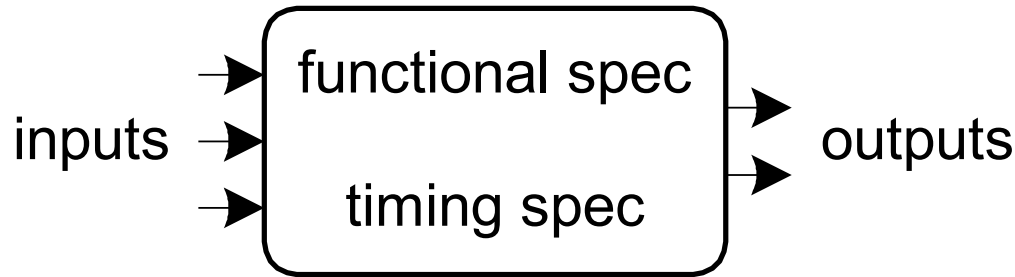
- Inputs
- Outputs



- *Functional specification* (describes relationship between inputs and outputs)
- *Timing specification* (describes the delay between inputs changing and outputs responding)

# Types of Logic Circuits

---



## ■ **Combinational Logic**

### □ **Memoryless**

- Outputs are strictly dependent on the combination of input values that are being applied to circuit *right now*

## ■ **Later we will learn: Sequential Logic**

### □ **Has memory**

- Structure stores history → Can "store" data values
- Outputs are determined by previous (historical) and current values of inputs

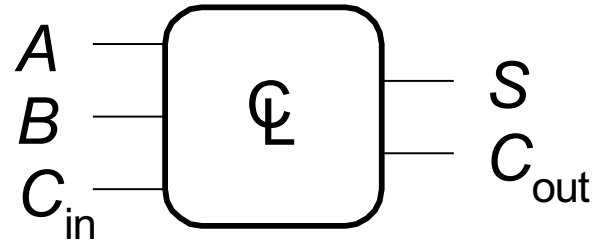
# Boolean Logic Equations

# Functional Specification

- **Functional specification** of outputs in terms of inputs
- What do we mean by “function”?
  - Unique **mapping** from input values to output values
  - The **same** input values produce the **same** output value every time
  - **No memory** (output does not depend on past input values)
- **Example (full 1-bit adder – more later):**

$$S = F(A, B, C_{in})$$

$$C_{out} = G(A, B, C_{in})$$

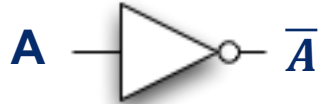


$$S = A \oplus B \oplus C_{in} \quad \text{3-input XOR}$$

$$C_{out} = AB + AC_{in} + BC_{in} \quad \text{3-input majority}$$

# Simple Equations: NOT / AND / OR

$\bar{A}$  (reads “not A”) is 1 iff A is 0



| A | $\bar{A}$ |
|---|-----------|
| 0 | 1         |
| 1 | 0         |

$A \cdot B$  (reads “A and B”) is 1 iff A and B are both 1



| A | B | $A \cdot B$ |
|---|---|-------------|
| 0 | 0 | 0           |
| 0 | 1 | 0           |
| 1 | 0 | 0           |
| 1 | 1 | 1           |

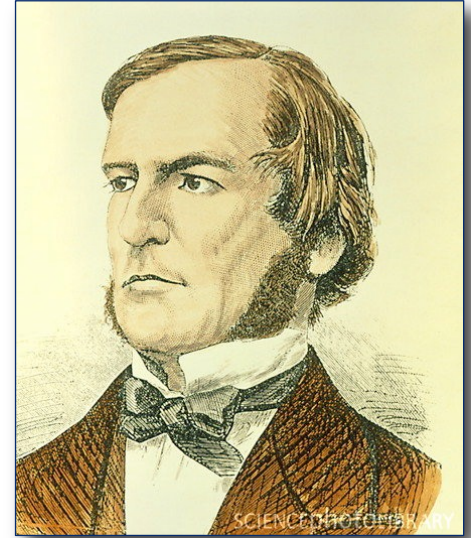
$A + B$  (reads “A or B”) is 1 iff either A or B is 1



| A | B | $A + B$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 1       |

# Boolean Algebra: Big Picture

- An algebra on 1's and 0's
  - with AND, OR, NOT operations
- What you start with
  - **Axioms:** basic things about objects and operations you just assume to be true at the start
- What you derive first
  - **Laws and theorems:** allow you to manipulate Boolean expressions
  - ...also allow us to do **simplification on Boolean expressions**
- What you derive later
  - More "sophisticated" properties useful for manipulating digital designs represented in the form of Boolean equations



# Boolean Algebra: Axioms

| <i>Formal version</i>                                                                       | <i>English version</i>                                                                                                     |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1. <i>B</i> contains at least two elements,<br><i>0</i> and <i>1</i> , such that $0 \neq 1$ | Math formality...                                                                                                          |
| 2. <i>Closure</i> $a, b \in B$ ,<br>(i) $a + b \in B$<br>(ii) $a \bullet b \in B$           | Result of AND, OR stays<br>in set you start with                                                                           |
| 3. <i>Commutative Laws</i> : $a, b \in B$ ,<br>(i)<br>(ii)                                  | For primitive AND, OR of<br>2 inputs, order doesn't matter                                                                 |
| 4. <i>Identities</i> : $0, 1 \in B$<br>(i)<br>(ii)                                          | There are identity elements<br>for AND, OR, that give you back<br>what you started with                                    |
| 5. <i>Distributive Laws</i> :<br>(i)<br>(ii)                                                | <ul style="list-style-type: none"><li>• distributes over +, just like algebra</li><li>+ distributes over •, also</li></ul> |
| 6. <i>Complement</i> :<br>(i)<br>(ii)                                                       | There is a complement element;<br>AND/ORing with it gives the identity elm.                                                |

# Boolean Algebra: Duality

---

## ■ Observation

- All the axioms come in “dual” form
- Anything true for an expression also true for its dual
- So any derivation you could make that is true, can be flipped into dual form, and it stays true

## ■ Duality — More formally

- A dual of a Boolean expression is derived by replacing
  - Every AND operation with... an OR operation
  - Every OR operation with... an AND
  - Every constant 1 with... a constant 0
  - Every constant 0 with... a constant 1
  - But don't change any of the literals or play with the complements!

**Example**

$$\begin{aligned} a \cdot (b + c) &= (a \cdot b) + (a \cdot c) \\ \rightarrow a + (b \cdot c) &= (a + b) \cdot (a + c) \end{aligned}$$

# Boolean Algebra: Useful Laws

---

## *Operations with 0 and 1:*

$$1. X + 0 = X$$

$$2. X + 1 = 1$$

Dual



$$1D. X \cdot 1 = X$$

$$2D. X \cdot 0 = 0$$

AND, OR with identities  
gives you back the original  
variable or the identity

---

## *Idempotent Law:*

$$3. X + X = X$$

$$3D. X \cdot X = X$$

AND, OR with self = self

---

## *Involution Law:*

$$4. \overline{\overline{X}} = X$$

double complement =  
no complement

---

## *Laws of Complementarity:*

$$5. X + \overline{X} = 1$$

$$5D. X \cdot \overline{X} = 0$$

AND, OR with complement  
gives you an identity

---

## *Commutative Law:*

$$6. X + Y = Y + X$$

$$6D. X \cdot Y = Y \cdot X$$

Just an axiom...

---

# Useful Laws (continued)

---

## *Associative Laws:*

$$\begin{aligned} 7. (X + Y) + Z &= X + (Y + Z) \\ &= X + Y + Z \end{aligned}$$

$$\begin{aligned} 7D. (X \cdot Y) \cdot Z &= X \cdot (Y \cdot Z) \\ &= X \cdot Y \cdot Z \end{aligned}$$

Parenthesis order  
does not matter

## *Distributive Laws:*

$$8. X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$$

$$8D. X + (Y \cdot Z) = (X + Y) \cdot (X + Z) \quad \text{Axiom}$$

## *Simplification Theorems:*

9.

9D.

10.

10D.

11.

11D.

Useful for  
simplifying  
expressions

Actually worth remembering — they show up a lot in real designs...

# Boolean Algebra: Proving Things

---

*Proving theorems via axioms of Boolean Algebra:*

**EX: Prove the theorem:  $X \cdot Y + X \cdot \bar{Y} = X$**

**Distributive (5)**

**Complement (6)**

**Identity (4)**

**EX2: Prove the theorem:  $X + X \cdot Y = X$**

**Identity (4)**

**Distributive (5)**

**Identity (2)**

**Identity (4)**

# DeMorgan's Law: Enabling Transformations

---

*DeMorgan's Law:*

$$12. \overline{(X + Y + Z + \dots)} = \bar{X} \cdot \bar{Y} \cdot \bar{Z} \cdot \dots$$

$$12D. \overline{(X \cdot Y \cdot Z \cdot \dots)} = \bar{X} + \bar{Y} + \bar{Z} + \dots$$

---

## ■ Think of this as a transformation

- Let's say we have:

$$F = A + B + C$$

- Applying DeMorgan's Law (12), gives us

$$F = \overline{\overline{(A + B + C)}} = \overline{(\bar{A} \cdot \bar{B} \cdot \bar{C})}$$

At least one of A, B, C is TRUE --> It is **not** the case that A, B, C are **all** false

# DeMorgan's Law (Continued)

These are conversions between **different types of logic functions**  
They can prove useful **if you do not have every type of gate...**  
**Or, if some types of gates are more desirable to use than others...**

$$A = \overline{(X + Y)} = \bar{X}\bar{Y}$$

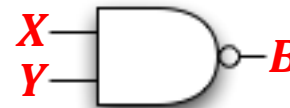
**NOR is equivalent to AND  
with inputs complemented**



| X | Y | $\overline{X + Y}$ | $\bar{X}$ | $\bar{Y}$ | $\bar{X}\bar{Y}$ |
|---|---|--------------------|-----------|-----------|------------------|
| 0 | 0 | 1                  | 1         | 1         | 1                |
| 0 | 1 | 0                  | 1         | 0         | 0                |
| 1 | 0 | 0                  | 0         | 1         | 0                |
| 1 | 1 | 0                  | 0         | 0         | 0                |

$$B = \overline{(XY)} = \bar{X} + \bar{Y}$$

**NAND is equivalent to OR  
with inputs complemented**



| X | Y | $\overline{XY}$ | $\bar{X}$ | $\bar{Y}$ | $\bar{X} + \bar{Y}$ |
|---|---|-----------------|-----------|-----------|---------------------|
| 0 | 0 | 1               | 1         | 1         | 1                   |
| 0 | 1 | 1               | 1         | 0         | 1                   |
| 1 | 0 | 1               | 0         | 1         | 1                   |
| 1 | 1 | 0               | 0         | 0         | 0                   |

# Using Boolean Equations to Represent a Logic Circuit

# Boolean Equations Enable Us To...

---

- Represent the function of a combinational logic block
  - Functional Specification
- Methodically transform the function into simpler functions
  - which lead to different hardware realizations
  - Logic Minimization or Logic Simplification
  - We can automate this process → Computer-Aided Design or Electronic Design Automation
- Different Boolean expressions lead to different logic gate implementations
  - Different hardware area, cost, latency, energy properties

# Standardized Function Representations

---

- Enable a single, universally-agreed-on way of representing a Boolean function starting from its truth table
  - Also called “canonical representations”
- Sum of Products (SOP) form
- Product of Sums (POS) form

# Sum of Products Form: Key Idea

---

- Assume **we have the truth table of Boolean Function F**
- How do we express the function in terms of the inputs in a **standard** manner?
- Idea: **Sum of Products** form
- Express the truth table as a two-level Boolean expression
  - that contains **all** input variable combinations that result in a 1 output
  - If ANY of the combinations of input variables that results in a 1 is TRUE, then the output is 1
  - **$F = \text{OR of all input variable combinations that result in a 1}$**

# Sum of Products Form: Example

## Sum of Products Form (SOP)

Also known as **disjunctive normal form** or **minterm expansion**

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$F = \overline{A}BC + A\overline{B}\overline{C} + A\overline{B}C + AB\overline{C} + ABC$

0 1 1      1 0 0      1 0 1      1 1 0      1 1 1

F = OR of all input variable combinations that result in a 1

All Boolean equations can be written in SOP form

Find all the input combinations (minterms) for which the output of the function is TRUE.

# Some Definitions (for a 3-Input Function)

---

- **Complement:** variable with a bar over it  
 $\bar{A}, \bar{B}, \bar{C}$
- **Literal:** variable or its complement  
 $A, \bar{A}, B, \bar{B}, C, \bar{C}$
- **Implicant:** product (AND) of literals  
 $(A \cdot B \cdot \bar{C}), (\bar{A} \cdot C), (B \cdot \bar{C})$
- **Minterm:** product (AND) that includes **all** input variables  
 $(A \cdot B \cdot \bar{C}), (\bar{A} \cdot \bar{B} \cdot C), (\bar{A} \cdot B \cdot \bar{C})$
- **Maxterm:** sum (OR) that includes **all** input variables  
 $(A + \bar{B} + \bar{C}), (\bar{A} + B + \bar{C}), (A + B + \bar{C})$

# Two-Level Canonical (Standard) Forms

---

- **Truth table** is the unique **signature** of a Boolean *function* ...
  - But, it is an expensive representation
- A Boolean function can have many alternative Boolean expressions
  - i.e., many alternative Boolean expressions (and gate realizations) may have the same truth table (and function)
  - **If they all specify the same thing, why do we care?**
    - Different Boolean expressions lead to different logic gate implementations → Different cost, latency, energy properties
- **Canonical** form: **standard form for a Boolean expression**
  - Provides a unique algebraic signature

# Two-Level Canonical Forms: SOP

## Sum of Products Form (SOP)

Also known as **disjunctive normal form** or **minterm expansion**

| A | B | C | F |   | 0 1 1       | 1 0 0             | 1 0 1       | 1 1 0       | 1 1 1 |
|---|---|---|---|---|-------------|-------------------|-------------|-------------|-------|
| 0 | 0 | 0 | 0 |   | $\bar{A}BC$ | $A\bar{B}\bar{C}$ | $A\bar{B}C$ | $AB\bar{C}$ | $ABC$ |
| 0 | 0 | 1 | 0 |   |             |                   |             |             |       |
| 0 | 1 | 0 | 0 |   |             |                   |             |             |       |
| 0 | 1 | 1 | 1 | ← |             |                   |             |             |       |
| 1 | 0 | 0 | 1 | ← |             |                   |             |             |       |
| 1 | 0 | 1 | 1 | ← |             |                   |             |             |       |
| 1 | 1 | 0 | 1 | ← |             |                   |             |             |       |
| 1 | 1 | 1 | 1 | ← |             |                   |             |             |       |

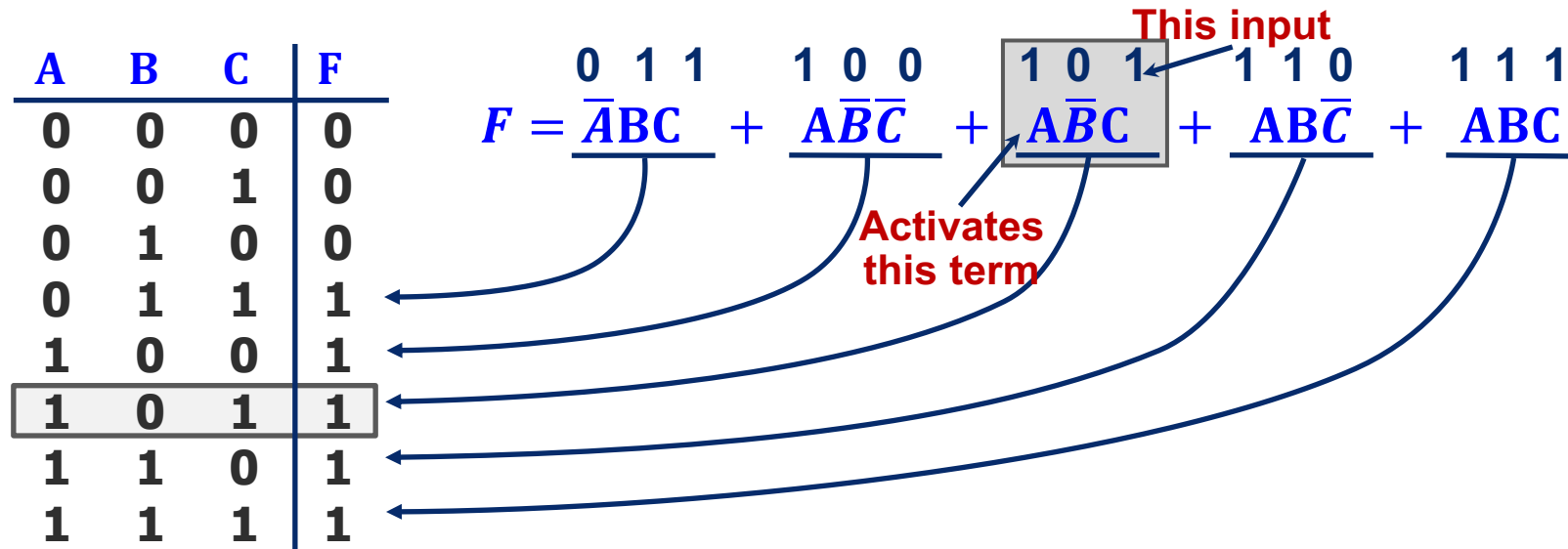
- Each row in a truth table has a minterm
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)

All Boolean equations can be written in SOP form

$F = \text{OR of all input variable combinations that result in a 1}$

Find all the input combinations (minterms) for which the output of the function is TRUE.

# SOP Form — Why Does It Work?



- Only the shaded product term —  $\bar{A}BC = 1 \cdot \bar{0} \cdot 1$  — will be 1
- No other product terms will “turn on” — they will all be 0
- So if inputs A B C correspond to a product term in expression,
  - We get  $0 + 0 + \dots + 1 + \dots + 0 + 0 = 1$  for output
- If inputs A B C do not correspond to any product term in expression
  - We get  $0 + 0 + \dots + 0 = 0$  for output

The function evaluates to TRUE (i.e., output is 1)  
 if **any** of the **Products** (minterms) causes the output to be 1

# Standard Notation for SOP Form

- Standard “shorthand” notation
  - If we agree on the **order** of the variables in the rows of truth table...
    - then we can enumerate each row with the decimal number that corresponds to the binary number created by the input pattern

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

**100 = decimal 4 so this is minterm #4, or m4**

**111 = decimal 7 so this is minterm #7, or m7**

f =

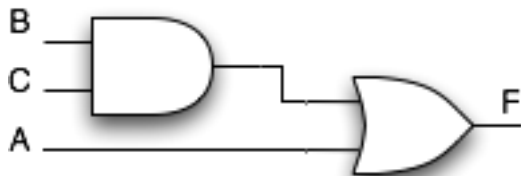
**We can write this as a sum of products**

**Or, we can use a summation notation**

# Canonical SOP Form

| A | B | C | minterms                                     |
|---|---|---|----------------------------------------------|
| 0 | 0 | 0 | $\overline{A}\overline{B}\overline{C} = m_0$ |
| 0 | 0 | 1 | $\overline{A}\overline{B}C = m_1$            |
| 0 | 1 | 0 | $\overline{A}B\overline{C} = m_2$            |
| 0 | 1 | 1 | $\overline{A}BC = m_3$                       |
| 1 | 0 | 0 | $A\overline{B}\overline{C} = m_4$            |
| 1 | 0 | 1 | $A\overline{B}C = m_5$                       |
| 1 | 1 | 0 | $AB\overline{C} = m_6$                       |
| 1 | 1 | 1 | $ABC = m_7$                                  |

Shorthand Notation for  
Minterms of 3 Variables



2-Level AND/OR  
Realization

*F in canonical form:*

$$F(A,B,C) = \sum m(3,4,5,6,7) \\ = m_3 + m_4 + m_5 + m_6 + m_7$$

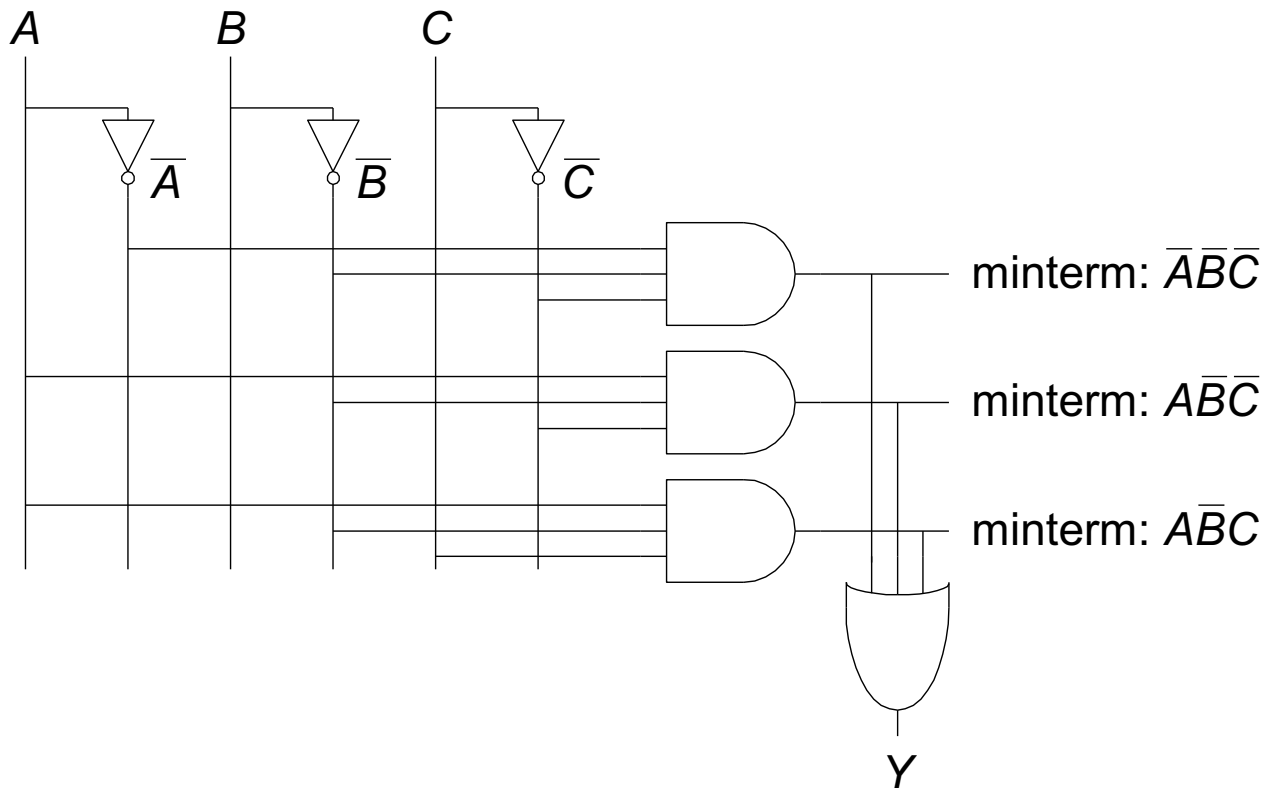
*F =*

*canonical form  $\neq$  minimal form*

*F*

# From SOP to Gates

- **SOP (sum-of-products) leads to two-level logic**
- Example:  $Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$



# Canonical Sum of Products Form: Summary

---

- Any 1-bit function can be represented as a Sum of Products
- A “Product” is the Boolean AND that includes ALL input variables of the function → minterm
- The 1-bit Output of the Function can be represented as
  - Sum (OR) of all minterms that lead to a 1 in the Output
- Logically
  - The function evaluates to TRUE (i.e., output is 1) if ANY of the Products (minterms) causes the Output to be 1
  - SOP form represents the function as the SUM (OR) of all Products (minterms) that cause the Output to be 1

# Alternative Canonical Form: POS

## DeMorgan of SOP of $\bar{F}$

Find all the input combinations (maxterms) for which the output of the function is FALSE.

$F$  = AND of all input variable combinations that result in a 0

## Product of Sums (POS)

product

$$F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$$

sums

Each sum term represents one of the “zeros” of the function

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$F = \overset{0}{(A + B + C)} \overset{0}{(A + B + \bar{C})} \overset{1}{(A + \bar{B} + C)}$

This input

Activates this term

For the given input, only the shaded sum term will equal 0

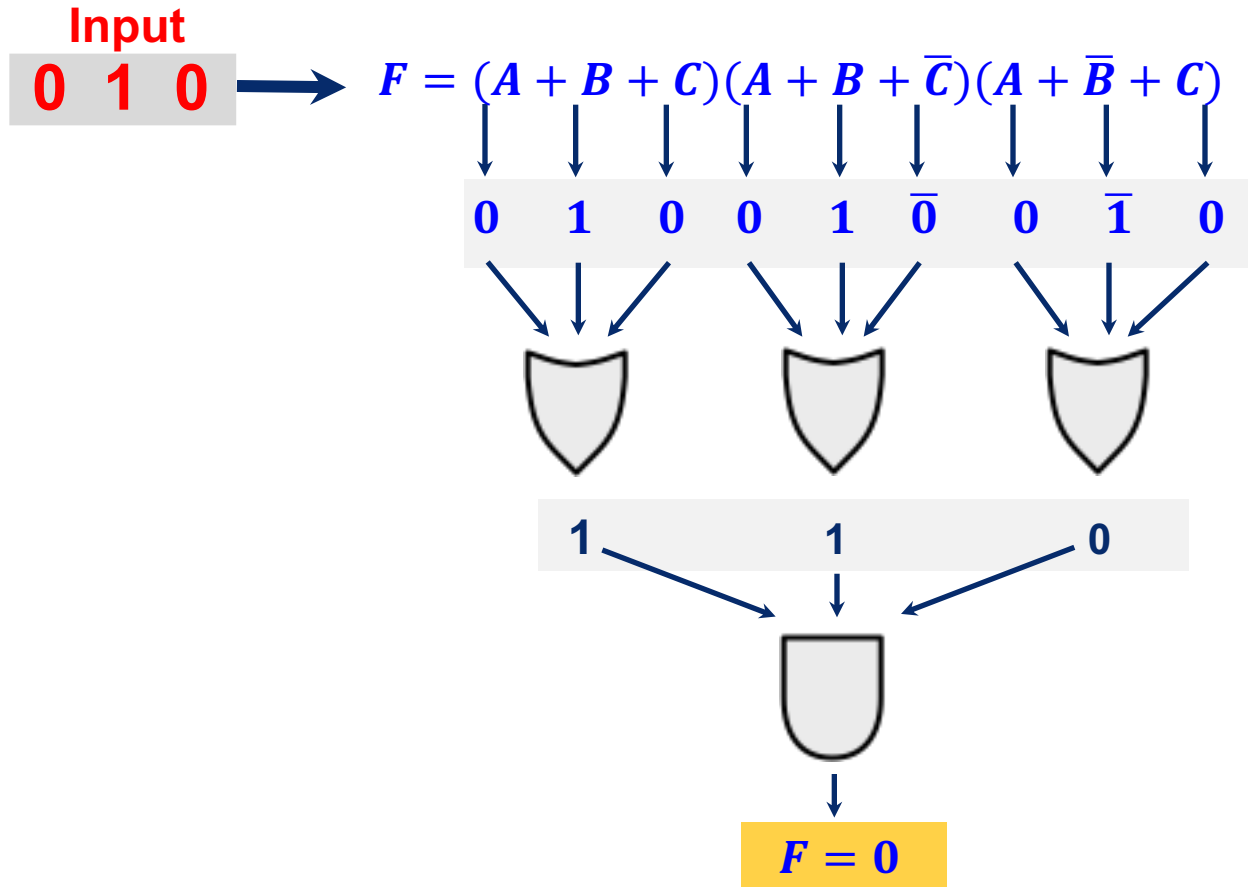
$$A + \bar{B} + C = 0 + \bar{1} + 0$$

Anything ANDed with 0 is 0; Output F will be 0

The function evaluates to FALSE (i.e., output is 0)  
if **any** of the Sums (maxterms) causes the output to be 0

# Consider $A=0, B=1, C=0$

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |



Only one of the products will be 0, anything ANDed with 0 is 0

Therefore, the output is  $F = 0$

# POS: How to Write It

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$$F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$$

$$A \quad \bar{B} \quad C$$

$$A + \bar{B} + C$$

**Maxterm form:**

1. Find truth table rows where F is 0
2. 0 in input col → true literal
3. 1 in input col → complemented literal
4. OR the literals to get a Maxterm
5. AND together all the Maxterms

**Or just remember" POS of  $F$  is the same as the DeMorgan of SOP of  $\bar{F}$**

# Notation for the Canonical POS Form

## *Product of Sums / Conjunctive Normal Form / Maxterm Expansion*

$$F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$$

$$\prod M(0, 1, 2)$$

| A | B | C | Maxterms                           |
|---|---|---|------------------------------------|
| 0 | 0 | 0 | $A + B + C = M0$                   |
| 0 | 0 | 1 | $A + B + \bar{C} = M1$             |
| 0 | 1 | 0 | $A + \bar{B} + C = M2$             |
| 0 | 1 | 1 | $A + \bar{B} + \bar{C} = M3$       |
| 1 | 0 | 0 | $\bar{A} + B + C = M4$             |
| 1 | 0 | 1 | $\bar{A} + B + \bar{C} = M5$       |
| 1 | 1 | 0 | $\bar{A} + \bar{B} + C = M6$       |
| 1 | 1 | 1 | $\bar{A} + \bar{B} + \bar{C} = M7$ |

Maxterm shorthand notation  
for a function of three variables

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Note that you  
form the  
maxterms around  
the “zeros” of the  
function

This is **not** the  
complement of  
the function!

# Useful Conversions

---

1. **Minterm to Maxterm conversion:**

rewrite minterm shorthand using maxterm shorthand  
replace minterm indices with the indices not already used

E.g.,  $F(A, B, C) = \sum m(3, 4, 5, 6, 7) = \prod M(0, 1, 2)$

2. **Maxterm to Minterm conversion:**

rewrite maxterm shorthand using minterm shorthand  
replace maxterm indices with the indices not already used

E.g.,  $F(A, B, C) = \prod M(0, 1, 2) = \sum m(3, 4, 5, 6, 7)$

3. **Expansion of F to expansion of  $\bar{F}$ :**

$$\begin{array}{ll} \text{E. g., } F(A, B, C) = \sum m(3, 4, 5, 6, 7) & \longrightarrow \bar{F}(A, B, C) = \sum m(0, 1, 2) \\ = \prod M(0, 1, 2) & \longrightarrow = \prod M(3, 4, 5, 6, 7) \end{array}$$

4. **Minterm expansion of F to Maxterm expansion of  $\bar{F}$ :**

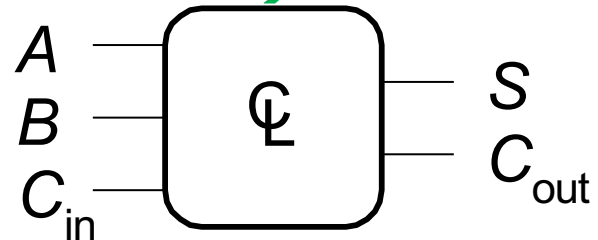
rewrite in Maxterm form, using the same indices as F

$$\begin{array}{ll} \text{E. g., } F(A, B, C) = \sum m(3, 4, 5, 6, 7) & \longrightarrow \bar{F}(A, B, C) = \prod M(3, 4, 5, 6, 7) \\ = \prod M(0, 1, 2) & \longrightarrow = \sum m(0, 1, 2) \end{array}$$

# Logic Simplification (or Minimization)

- Using Boolean Algebra, we can simplify the SOP or POS form of any function in a methodical way
- Starting with the canonical SOP or POS form enables convenience and automation
  - Truth table → SOP/POS form → Boolean Simplification Rules
- **Example (full 1-bit adder – more later):**

$$\begin{aligned} S &= F(A, B, C_{in}) \\ C_{out} &= G(A, B, C_{in}) \end{aligned}$$

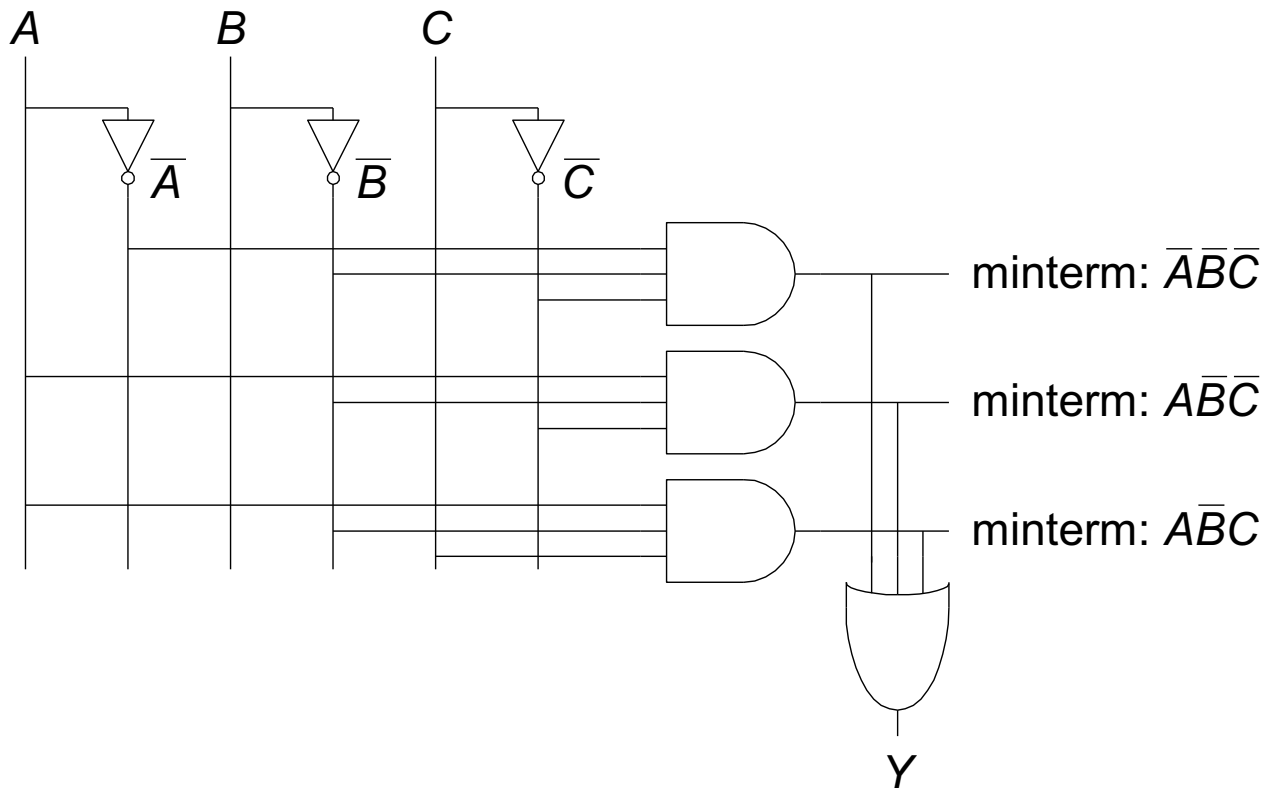


$$\begin{aligned} S &= A \oplus B \oplus C_{in} \quad \text{3-input XOR} \\ C_{out} &= AB + AC_{in} + BC_{in} \quad \text{3-input majority} \end{aligned}$$

# Logic Simplification Example: SOP Form

- **SOP (sum-of-products) form of function Y**

- Example:  $Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$



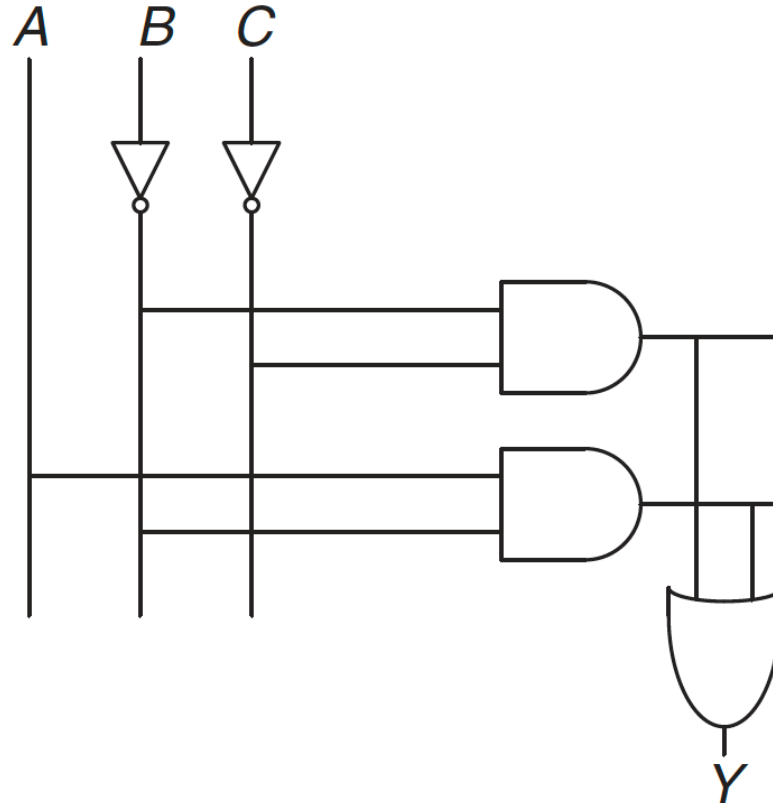
**SOP form does NOT directly lead to *minimal* logic**

# Logic Simplification Example: Simplified

---

- **Simplified form of function Y**

- Example:  $Y = (\overline{B} \cdot \overline{C}) + (A \cdot \overline{B})$



# Let's Cover Some Basic Combinational Blocks

# Combinational Building Blocks used in Modern Computers

# Recall: Common Logic Gates

**Buffer**



| A | Z |
|---|---|
| 0 | 0 |
| 1 | 1 |

**AND**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**OR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**XOR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Inverter**



| A | Z |
|---|---|
| 0 | 1 |
| 1 | 0 |

**NAND**



| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**NOR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**XNOR**



| A | B | Z |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

# Combinational Building Blocks

---

- Combinational logic is often grouped into larger building blocks to build more **complex systems**
  - Hides the **unnecessary gate-level details** to emphasize the function of the building block
  - We now examine:
    - Decoder
    - Multiplexer
    - Full adder
    - PLA (Programmable Logic Array)
-

# We Will Cover Many Building Blocks

---

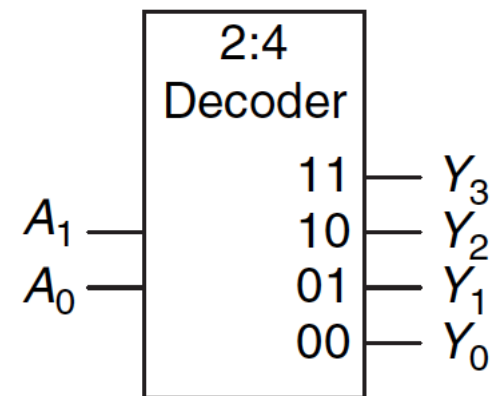
- Basic logic gates (AND, OR, NOT, NAND, NOR, XOR)
  - Decoder
  - Multiplexer
  - Full Adder
  - Programmable Logic Array (PLA)
  - Comparator
  - Arithmetic Logic Unit (ALU)
  - Tri-State Buffer
- 
- Standard form representations: SOP & POS
  - Logic simplification via Boolean Algebra
  - Logical completeness

# Decoder

# Decoder

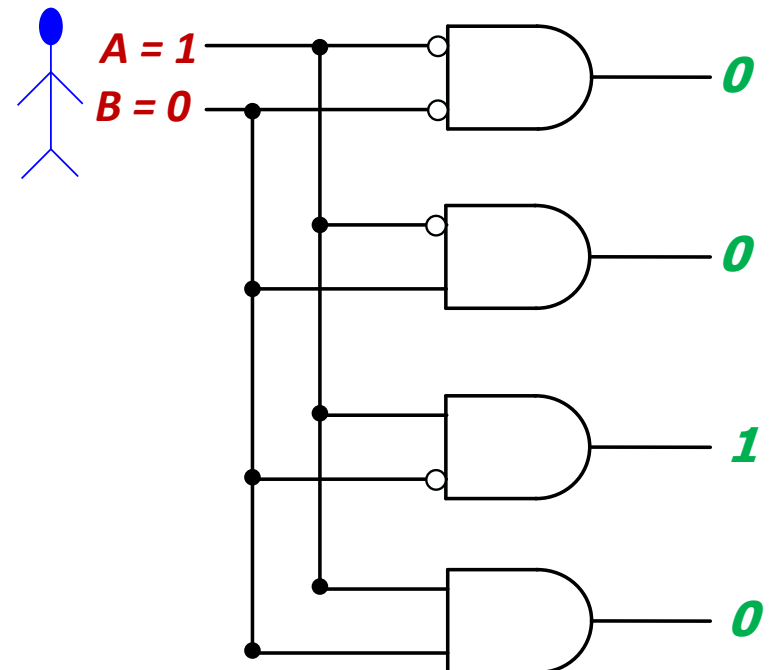
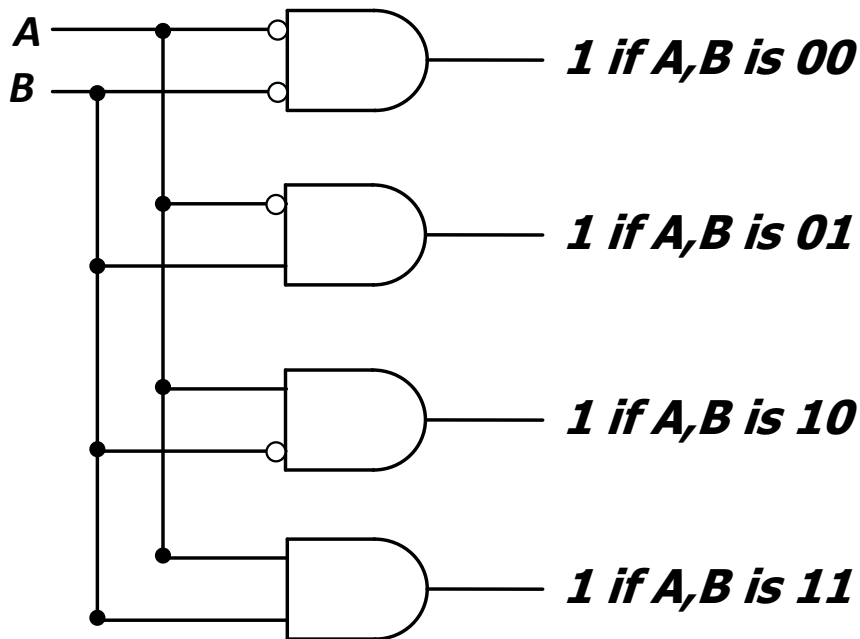
- “Input pattern detector”
- $n$  inputs and  $2^n$  outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The **output** that is logically 1 is the output corresponding to the input **pattern** that the logic circuit is expected to detect
- Example: 2-to-4 decoder

| $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 1     |
| 0     | 1     | 0     | 0     | 1     | 0     |
| 1     | 0     | 0     | 1     | 0     | 0     |
| 1     | 1     | 1     | 0     | 0     | 0     |



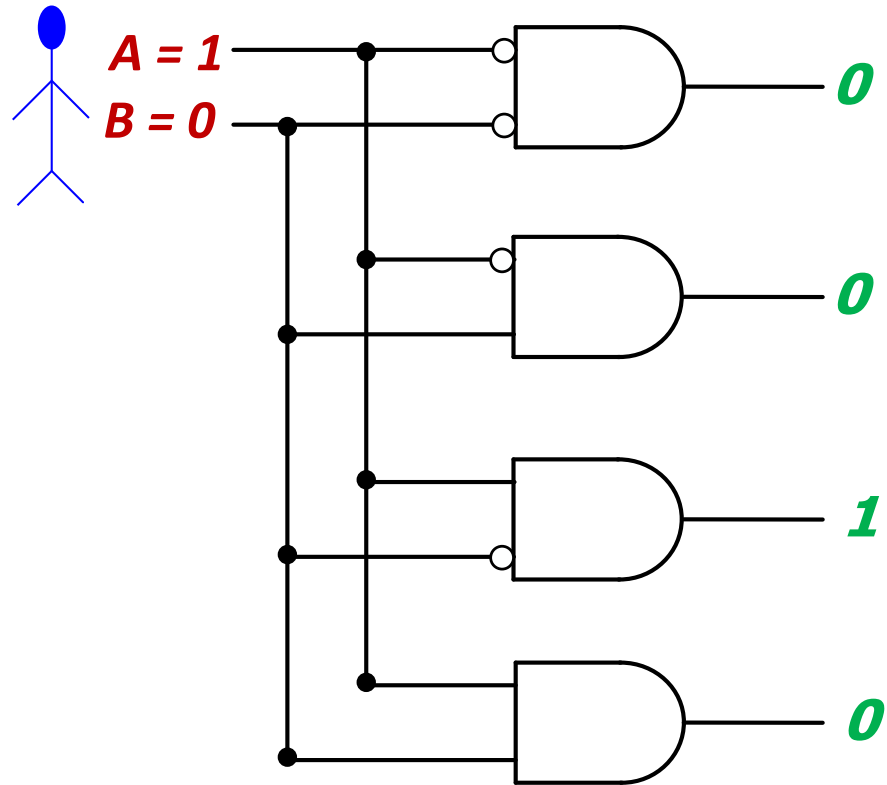
# Decoder (I)

- $n$  inputs and  $2^n$  outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The **output** that is logically 1 is the output corresponding to the input **pattern** that the logic circuit is expected to detect



# Decoder (II)

- The decoder is useful in determining how to interpret a bit pattern
  - **It could be the address of a location in memory, that the processor intends to read from**
  - **It could be an instruction in the program and the processor needs to decide what action to take (based on *instruction opcode*)**



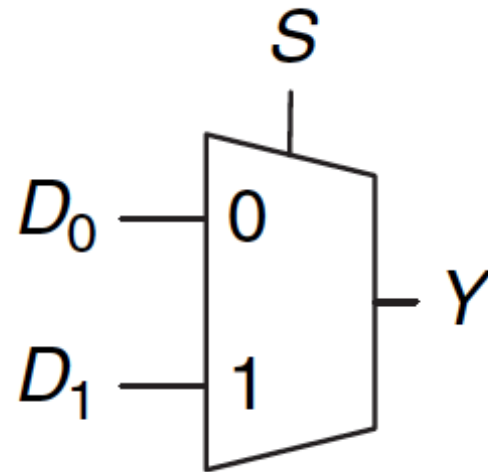
# Multiplexer (MUX)

# Multiplexer (MUX), or Selector

---

- **Selects** one of the  $N$  inputs to connect it to the output
  - ▣ based on the value of a  $\log_2 N$ -bit control input called **select**
- Example: 2-to-1 MUX

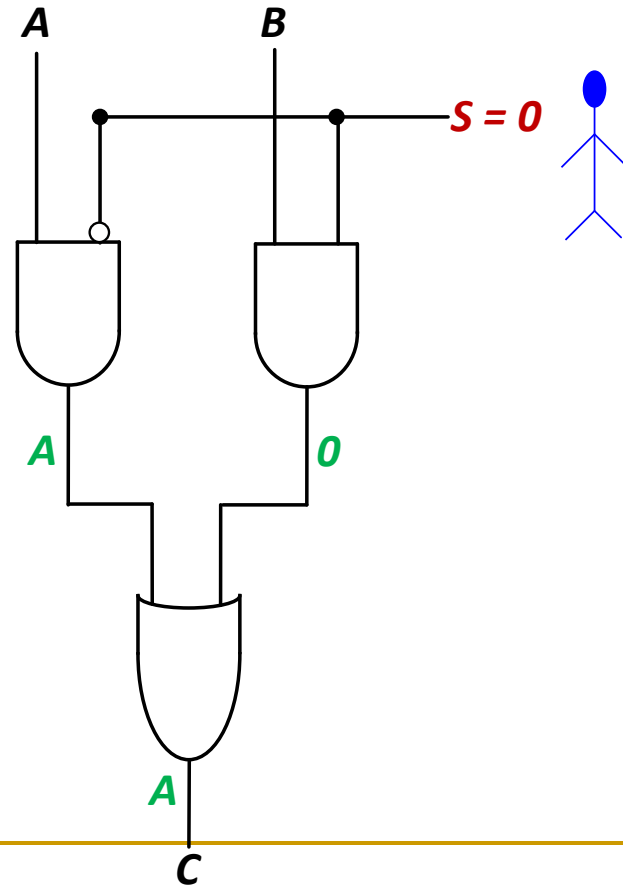
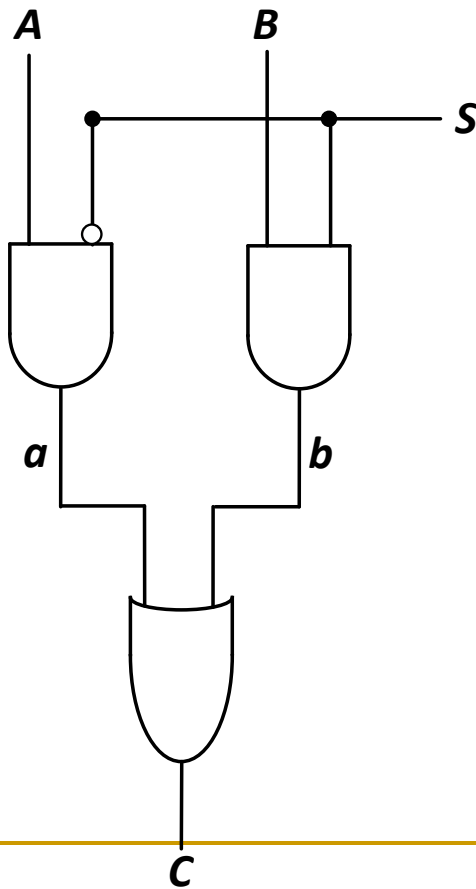
| $S$ | $D_1$ | $D_0$ | $Y$ |
|-----|-------|-------|-----|
| 0   | 0     | 0     | 0   |
| 0   | 0     | 1     | 1   |
| 0   | 1     | 0     | 0   |
| 0   | 1     | 1     | 1   |
| 1   | 0     | 0     | 0   |
| 1   | 0     | 1     | 0   |
| 1   | 1     | 0     | 1   |
| 1   | 1     | 1     | 1   |



# Multiplexer (MUX), or Selector (II)

---

- **Selects** one of the  $N$  inputs to connect it to the output
  - based on the value of a  $\log_2 N$ -bit control input called **select**
- Example: 2-to-1 MUX

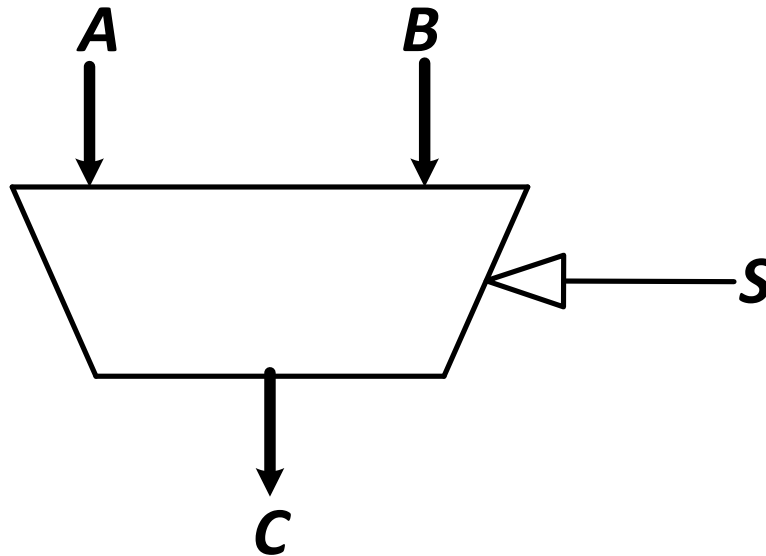


# Multiplexer (MUX), or Selector (III)

---

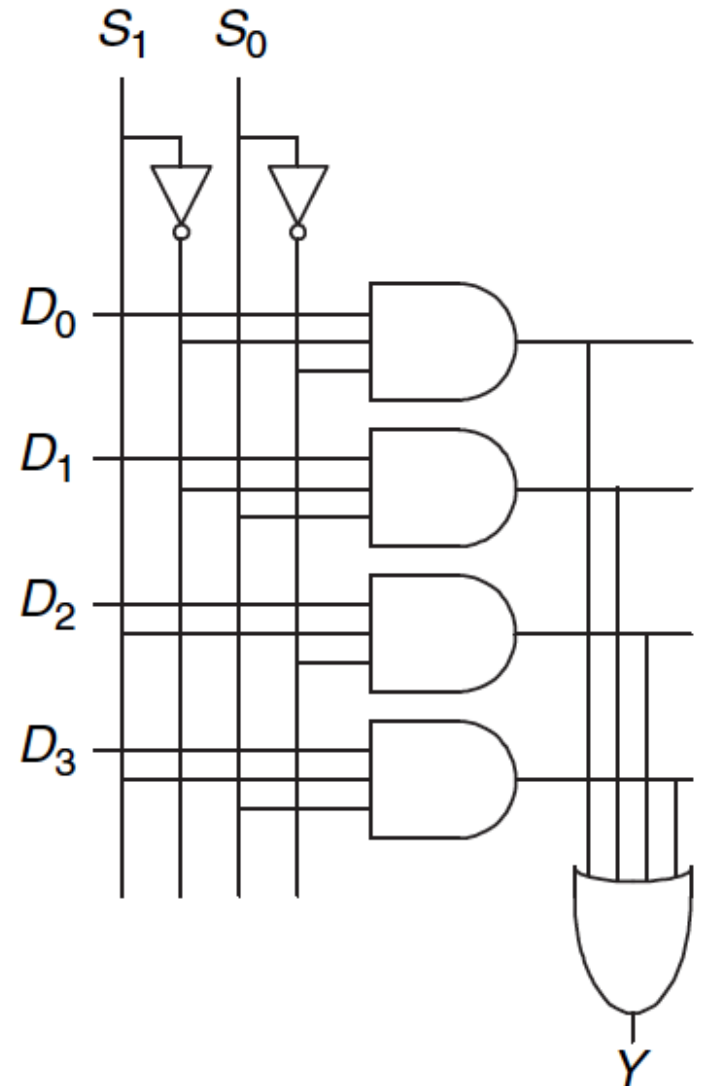
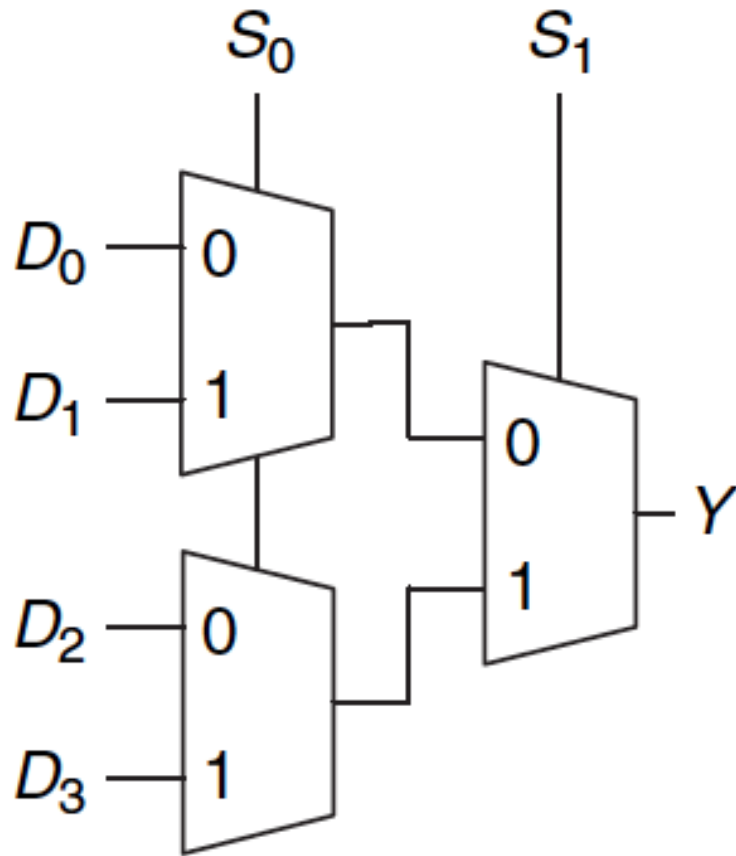
- The output C is always connected to either the input A or the input B
  - Output value depends on the value of the **select line S**

| <b>S</b> | <b>C</b> |
|----------|----------|
| 0        | A        |
| 1        | B        |



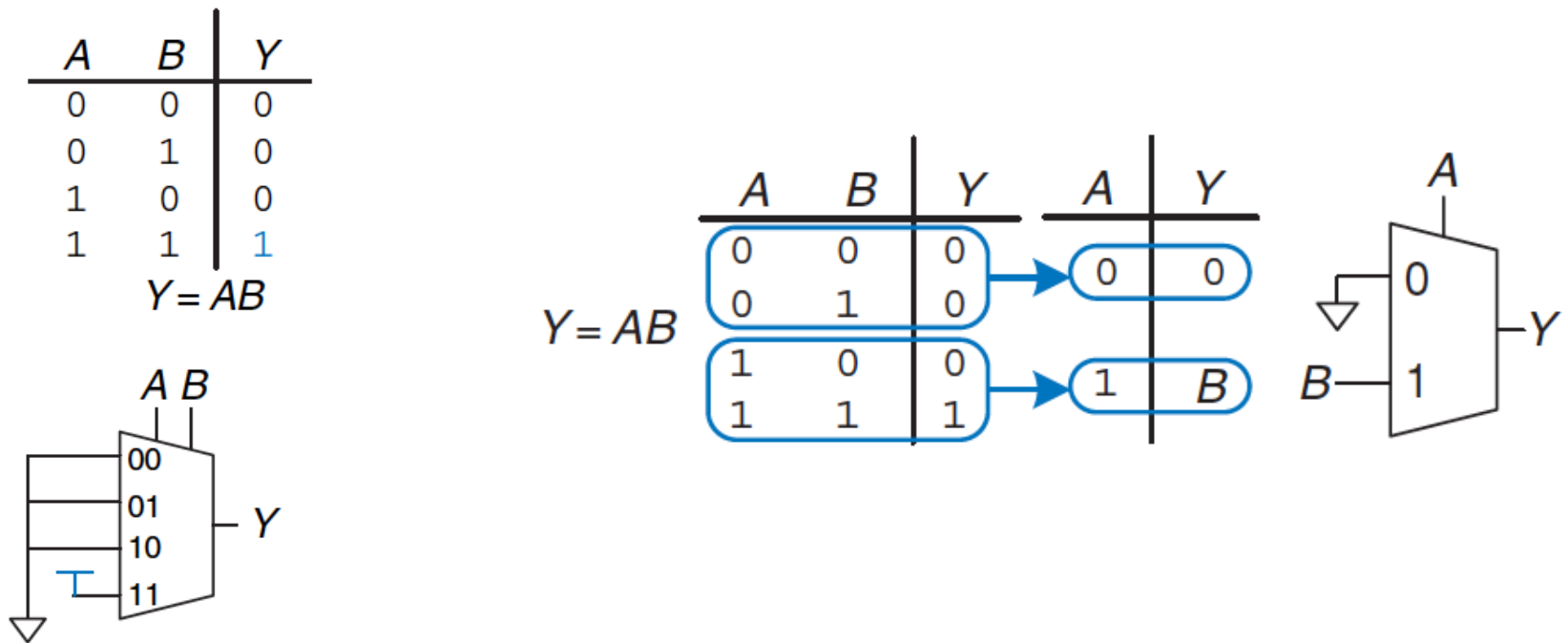
- **Your task:** Draw the schematic for an 4-input (4:1) MUX
  - Gate level: as a combination of basic AND, OR, NOT gates
  - Module level: As a combination of 2-input (2:1) MUXes

# A 4-to-1 Multiplexer



# Aside: Logic Using Multiplexers

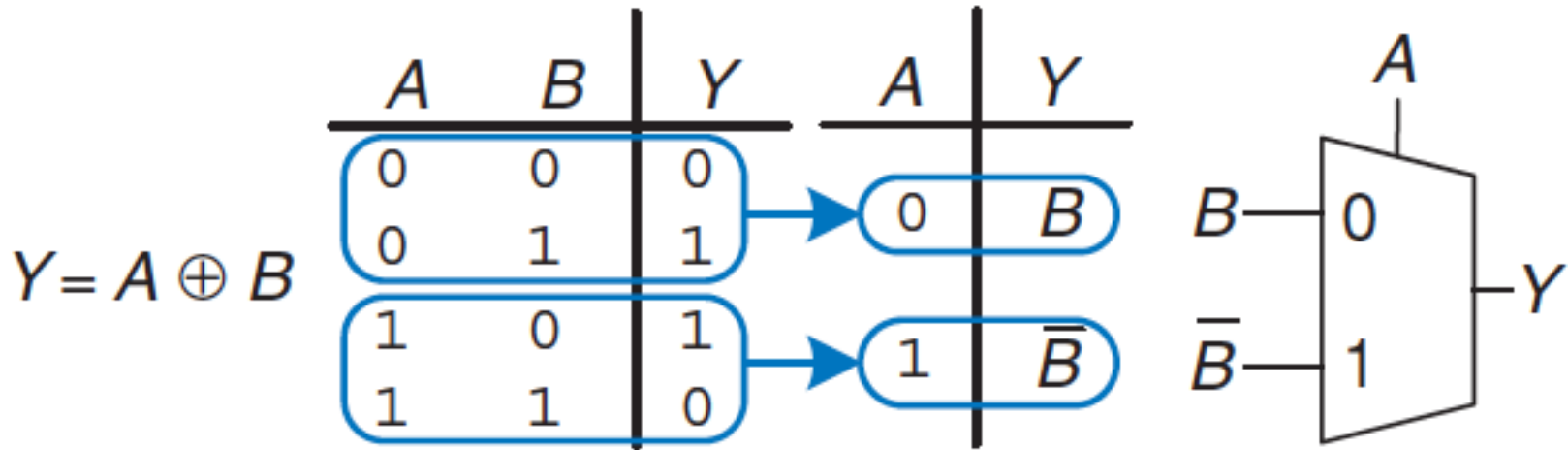
- Multiplexers can be used as “lookup tables” to perform logic functions



**Figure 2.59** 4:1 multiplexer implementation of two-input AND function

# Aside: Logic Using Multiplexers (II)

- Multiplexers can be used as “lookup tables” to perform logic functions

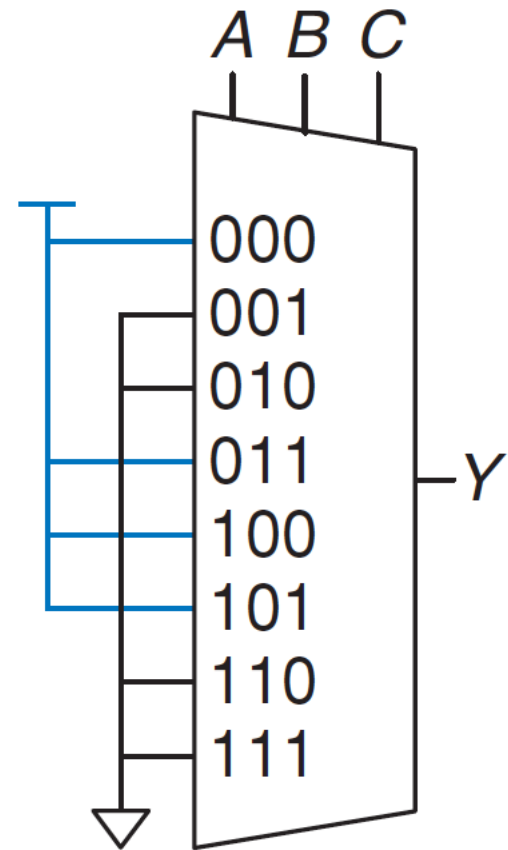


# Aside: Logic Using Multiplexers (III)

- Multiplexers can be used as lookup tables to perform logic functions

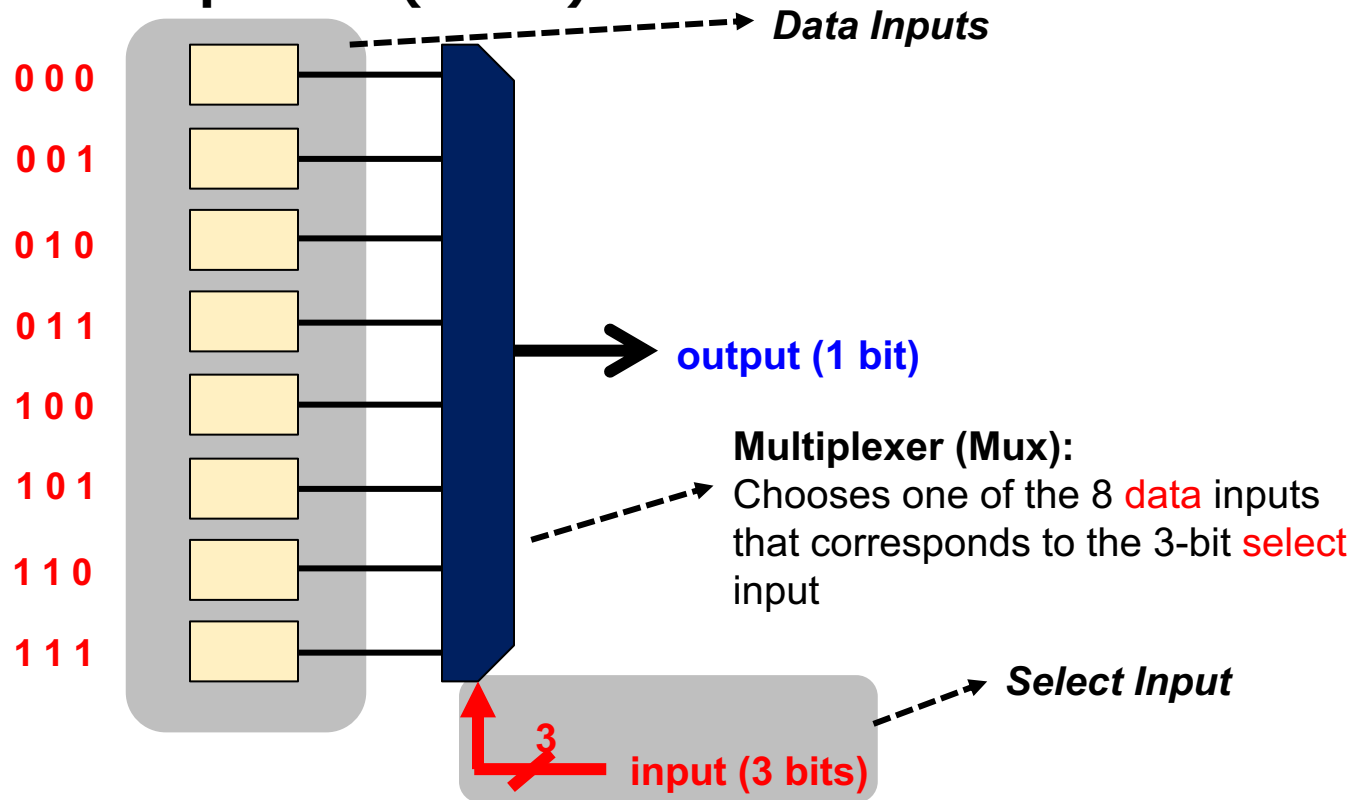
| <i>A</i> | <i>B</i> | <i>C</i> | <i>Y</i> |
|----------|----------|----------|----------|
| 0        | 0        | 0        | 1        |
| 0        | 0        | 1        | 0        |
| 0        | 1        | 0        | 0        |
| 0        | 1        | 1        | 1        |
| 1        | 0        | 0        | 1        |
| 1        | 0        | 1        | 1        |
| 1        | 1        | 0        | 0        |
| 1        | 1        | 1        | 0        |

$$Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$$



# 8-Input Lookup Table (LUT)

## ■ 3-bit input LUT (3-LUT)



3-LUT can implement  
**any** 3-bit input function

# An Example of Programming a LUT

- A function that outputs '1' when there are **at least two '1's in a 3-bit input**

In C:

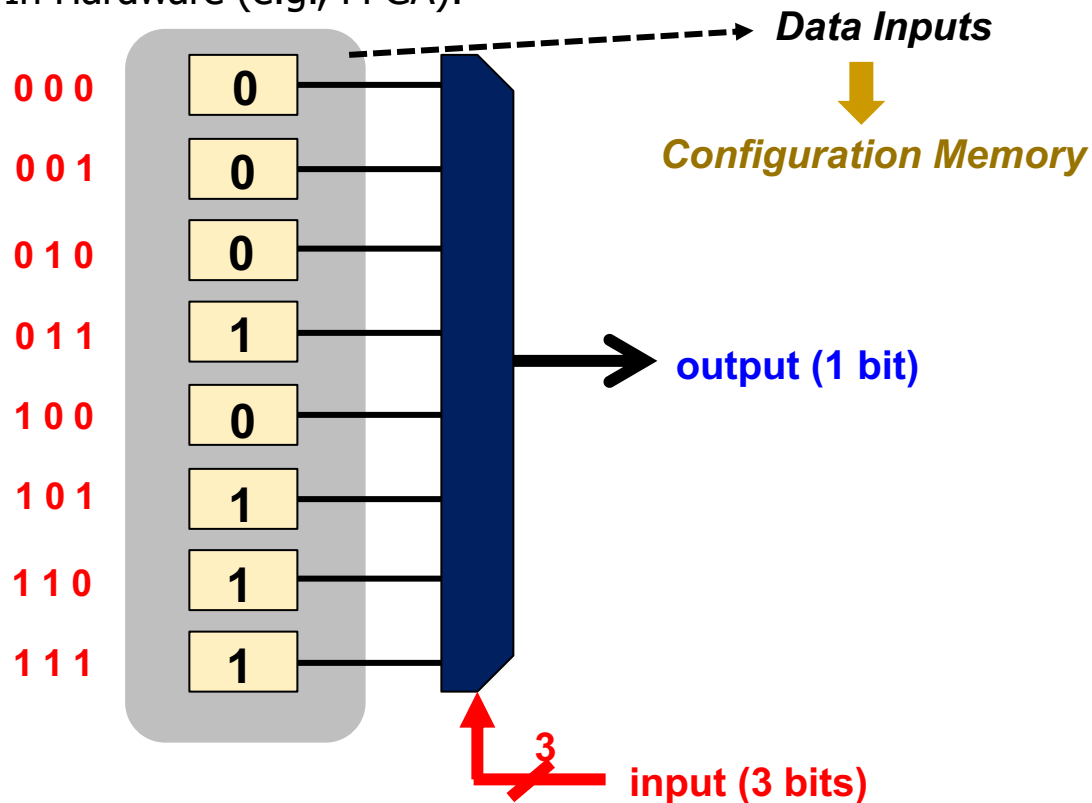
```
int count = 0;
for(int i = 0; i < 3; i++) {
    count += input & 1;
    input = input >> 1;
}

if(count > 1) return 1;

return 0;
```

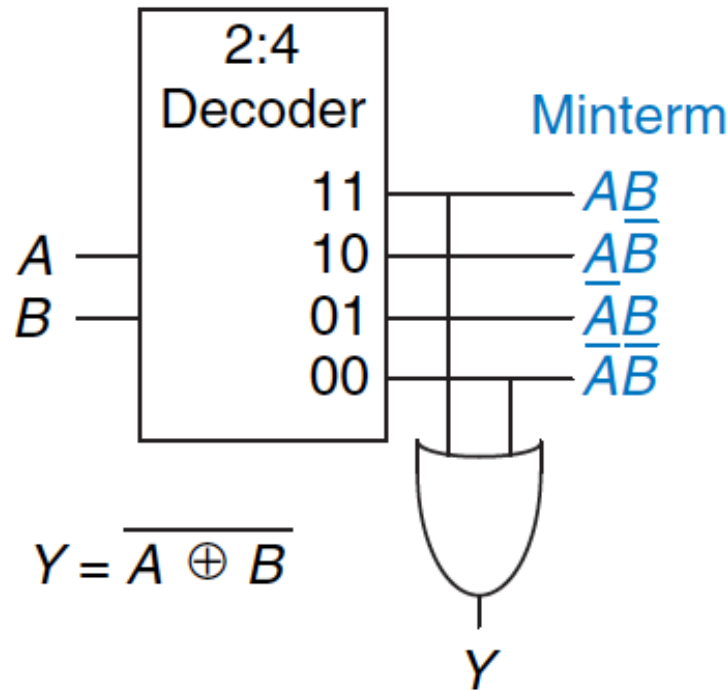
```
switch(input){
    case 0:
    case 1:
    case 2:
    case 4:
        return 0;
    default:
        return 1;}
```

In Hardware (e.g., FPGA):



# Aside: Logic Using Decoders (I)

- Decoders can be combined with OR gates to build logic functions.



**Figure 2.65** Logic function using decoder

**We Covered Until Here in Lecture**

# Digital Design & Computer Arch.

## Lecture 2: Transistors, Gates, Combinational Logic

Prof. Onur Mutlu

ETH Zürich

Spring 2026

20 February 2026

**Further Slides for Your Own Study  
(May Be Covered in Future Lectures)**

# Full Adder

# Full Adder (I)

## ■ Binary addition

- Similar to decimal addition
- From right to left
- One column at a time
- One sum and one carry bit

$$\begin{array}{r}
 a_{n-1}a_{n-2} \dots a_1a_0 \\
 b_{n-1}b_{n-2} \dots b_1b_0 \\
 \underline{C_n C_{n-1} \dots C_1} \\
 S_{n-1} \dots S_1S_0
 \end{array}$$

↓

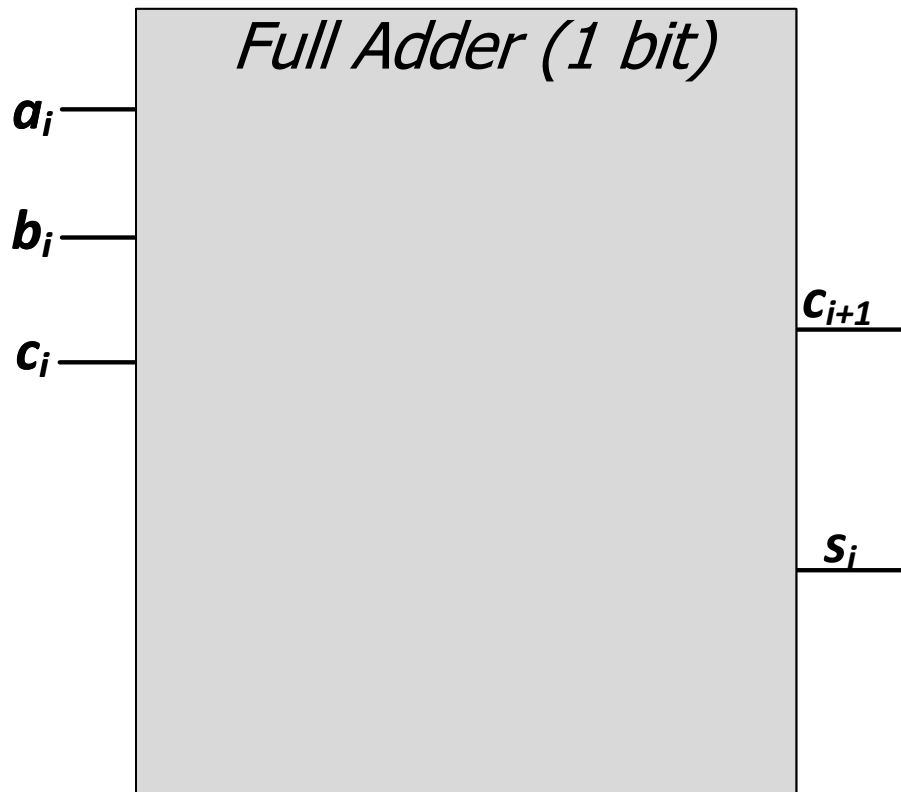
- Truth table of binary addition on **one column** of bits within two n-bit operands

| $a_i$ | $b_i$ | $carry_i$ | $carry_{i+1}$ | $S_i$ |
|-------|-------|-----------|---------------|-------|
| 0     | 0     | 0         | 0             | 0     |
| 0     | 0     | 1         | 0             | 1     |
| 0     | 1     | 0         | 0             | 1     |
| 0     | 1     | 1         | 1             | 0     |
| 1     | 0     | 0         | 0             | 1     |
| 1     | 0     | 1         | 1             | 0     |
| 1     | 1     | 0         | 1             | 0     |
| 1     | 1     | 1         | 1             | 1     |

# Full Adder (II)

## ■ Binary addition

- N 1-bit additions
- **SOP of 1-bit addition**



$$\begin{array}{r}
 a_{n-1}a_{n-2} \dots a_1a_0 \\
 b_{n-1}b_{n-2} \dots b_1b_0 \\
 \hline
 C_n C_{n-1} \dots C_1 \\
 \hline
 S_{n-1} \dots S_1S_0
 \end{array}$$

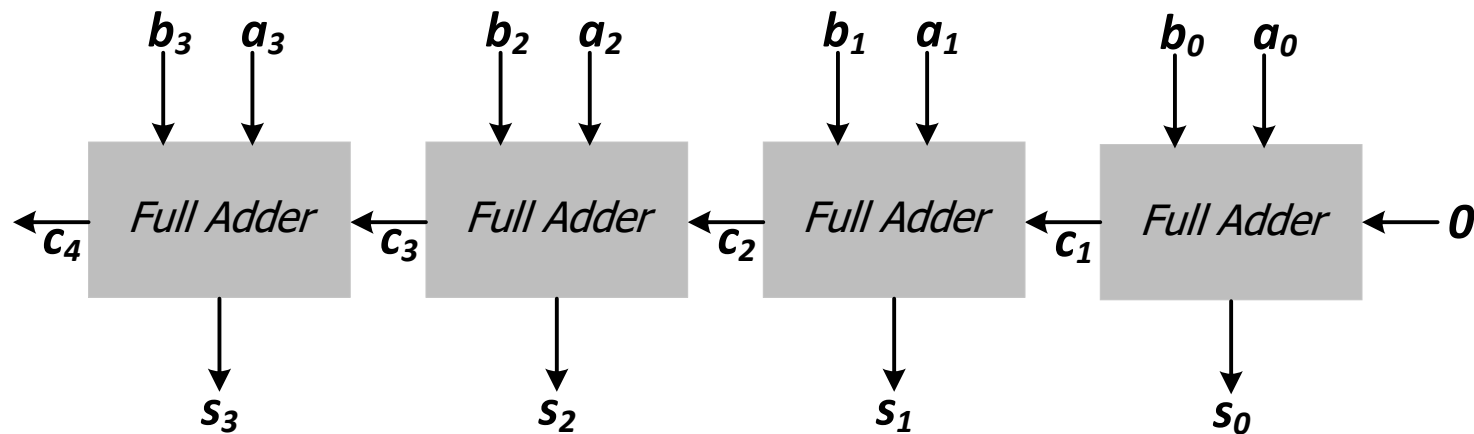
↓

| $a_i$ | $b_i$ | $carry_i$ | $carry_{i+1}$ | $S_i$ |
|-------|-------|-----------|---------------|-------|
| 0     | 0     | 0         | 0             | 0     |
| 0     | 0     | 1         | 0             | 1     |
| 0     | 1     | 0         | 0             | 1     |
| 0     | 1     | 1         | 1             | 0     |
| 1     | 0     | 0         | 0             | 1     |
| 1     | 0     | 1         | 1             | 0     |
| 1     | 1     | 0         | 1             | 0     |
| 1     | 1     | 1         | 1             | 1     |

MAJ XOR

# 4-Bit Adder from Full Adders

- Creating a **4-bit adder** out of 1-bit full adders
  - To add two 4-bit binary numbers A and B

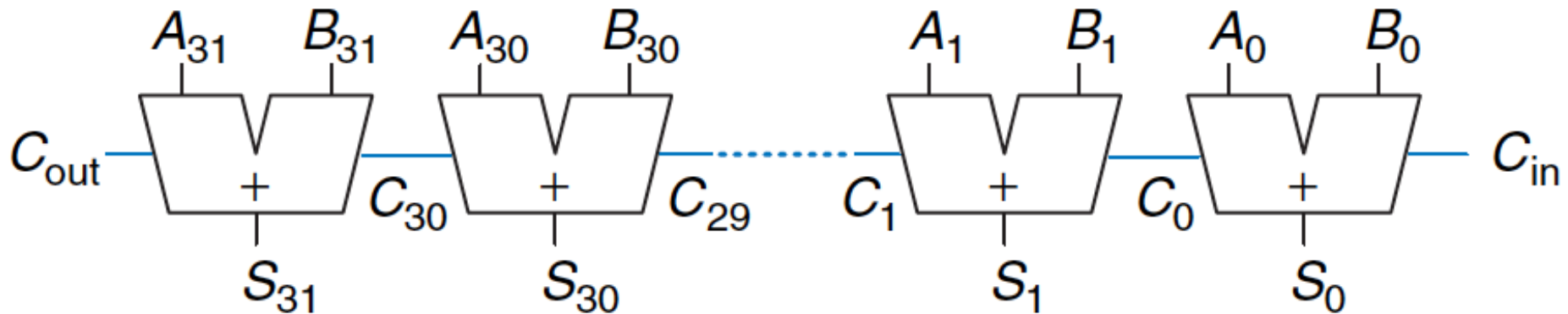


$$\begin{array}{rcccc} & a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \\ \hline c_4 & c_3 & c_2 & c_1 & \\ \hline s_3 & s_2 & s_1 & s_0 & \end{array}$$

$$\begin{array}{rcccc} & 1 & 0 & 1 & 1 \\ + & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 1 & 1 & \\ \hline 0 & 1 & 0 & 0 & \end{array}$$

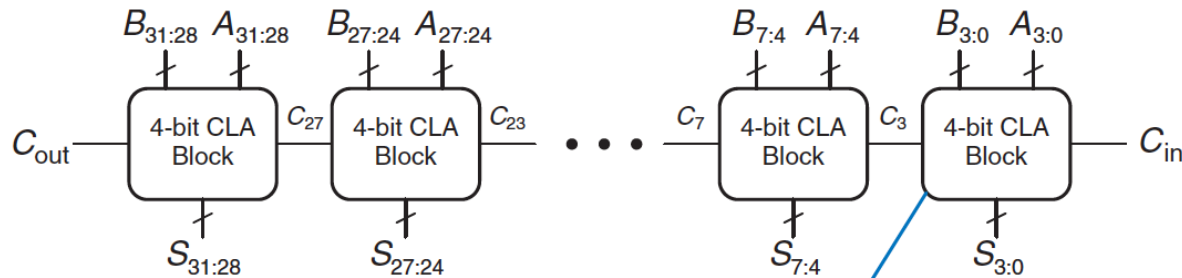
# Adder Design: Ripple Carry Adder

---

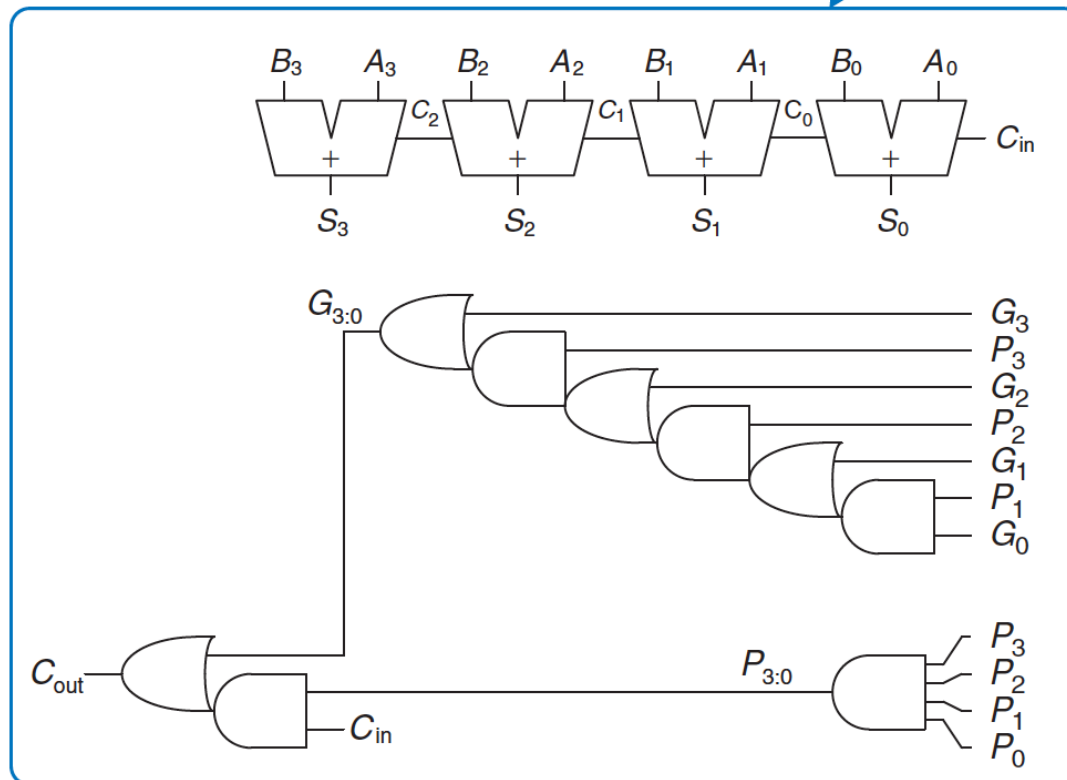


**Figure 5.5** 32-bit ripple-carry adder

# Adder Design: Carry Lookahead Adder



(a)



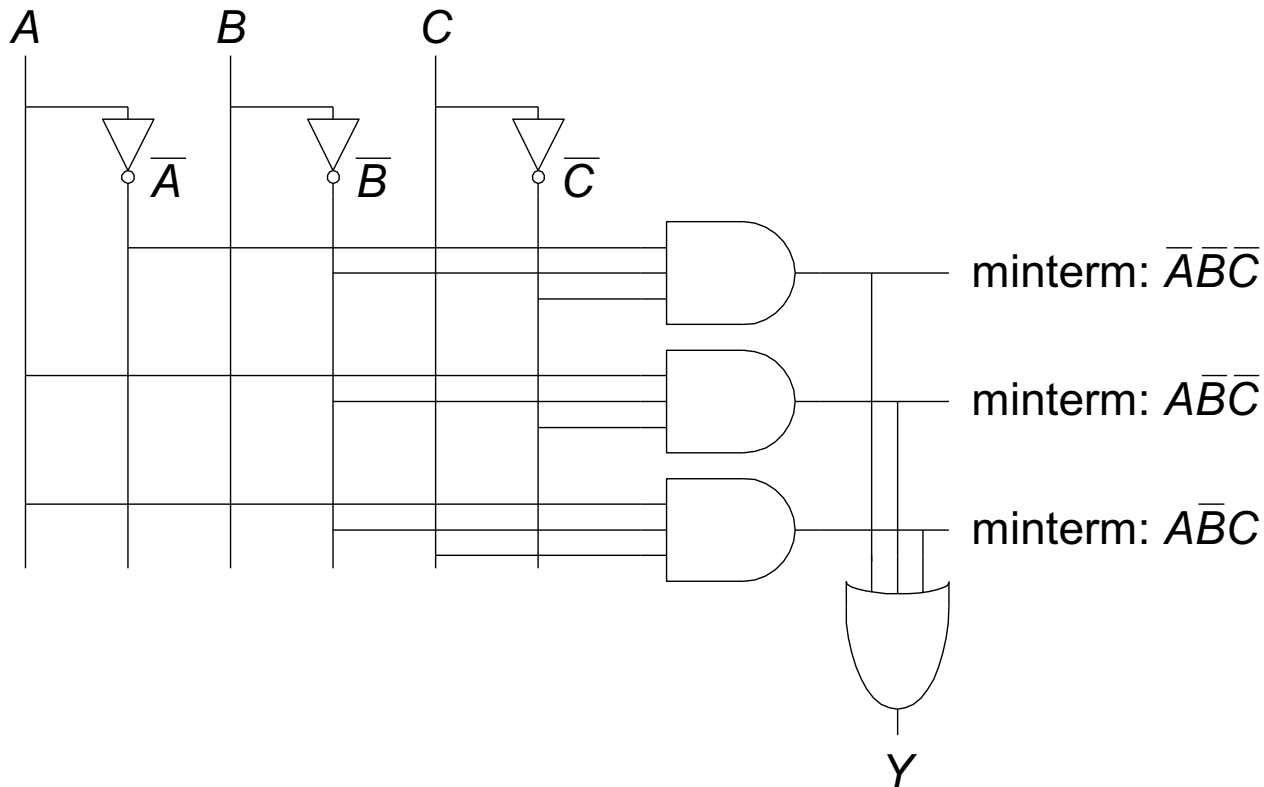
(b)

**Example of  
logic specialization:  
Specialized logic for  
fast carry calculation**

# Programmable Logic Array (PLA)

# PLA: Recall: SOP Form

- **SOP (sum-of-products) leads to two-level logic**
- Example:  $Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$



A PLA enables the two-level SOP implementation of **any** N-input M-output function

# The Programmable Logic Array (PLA)

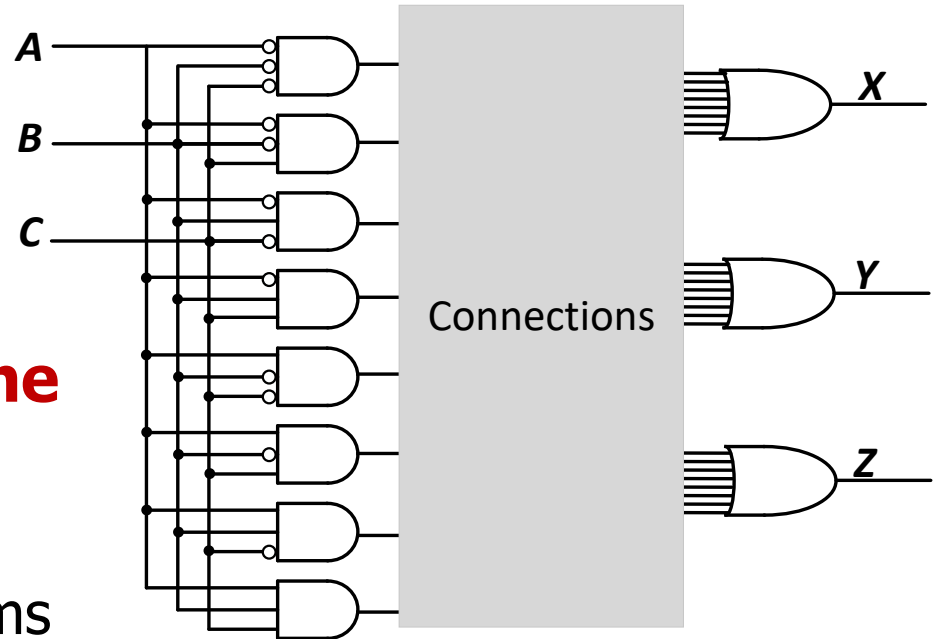
- The below logic structure is a very **common** building block for implementing any collection of logic functions one wishes to

- An **array** of AND gates followed by an **array** of OR gates
- **How do we determine the number of AND gates?**

- **Remember SOP:** the number of possible minterms

- For an  $n$ -input logic function, we need a PLA with  $2^n$   $n$ -input AND gates

- **How do we determine the number of OR gates?** The number of output columns in the truth table

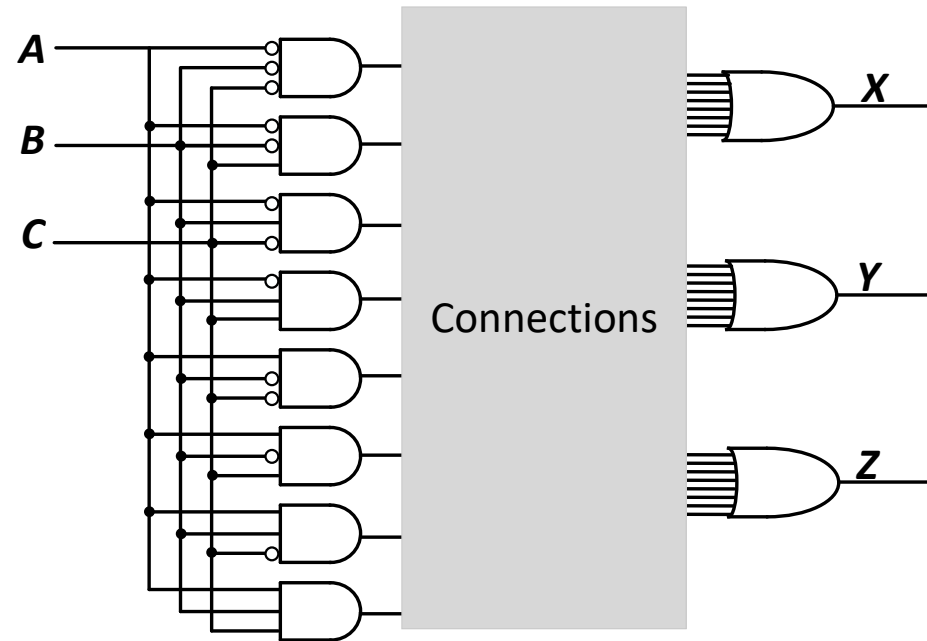


A PLA enables the two-level SOP implementation of **any**  $N$ -input  $M$ -output function

# The Programmable Logic Array (PLA)

- How do we implement a logic function?
  - Connect the output of an AND gate to the input of an OR gate if the corresponding minterm is included in the SOP
  - This is a simple programmable logic construct

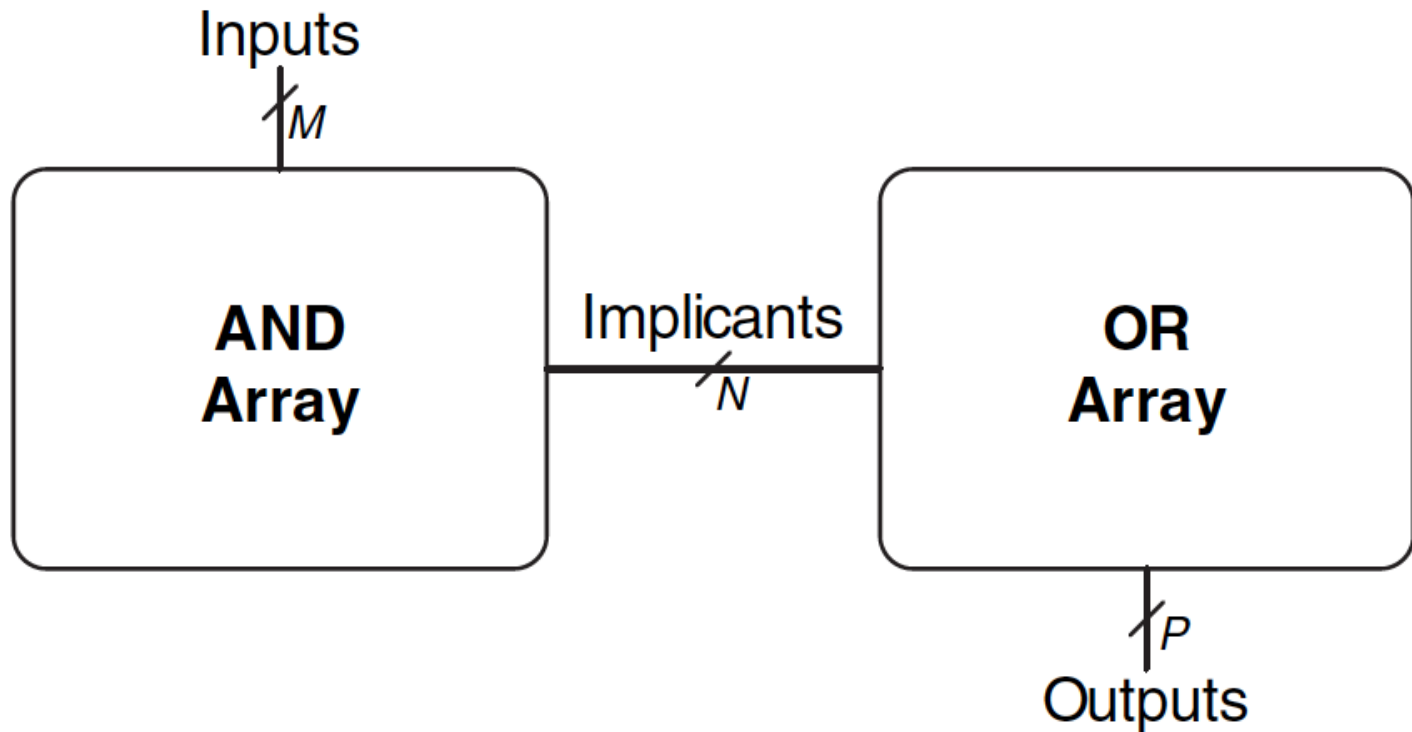
- **Programming a PLA:** we program the connections from AND gate outputs to OR gate inputs to implement a desired logic function



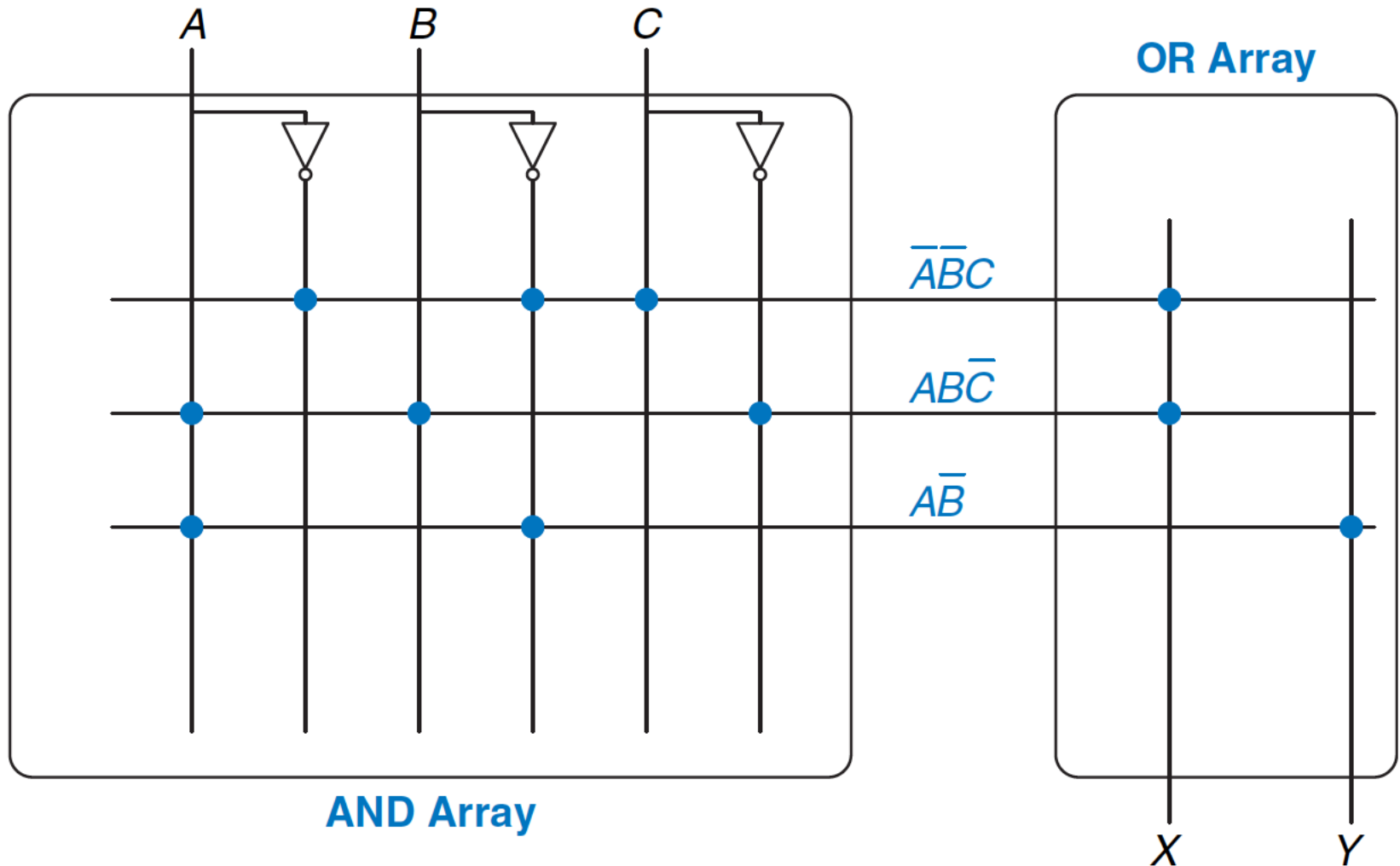
- Is there any other type of programmable logic?
  - Yes! An FPGA...
  - An FPGA uses more advanced structures, as we see in the labs

# PLA Example (I)

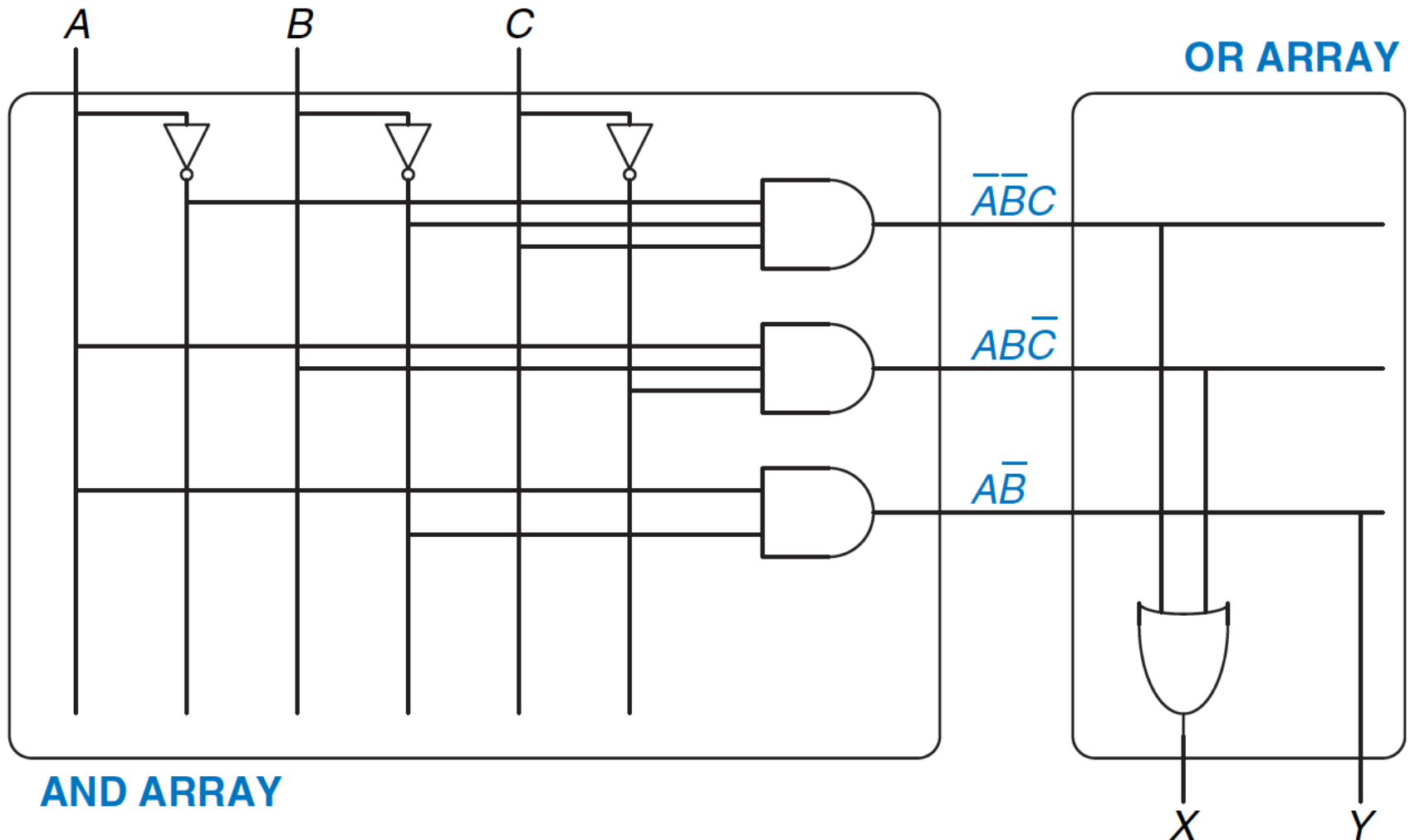
---



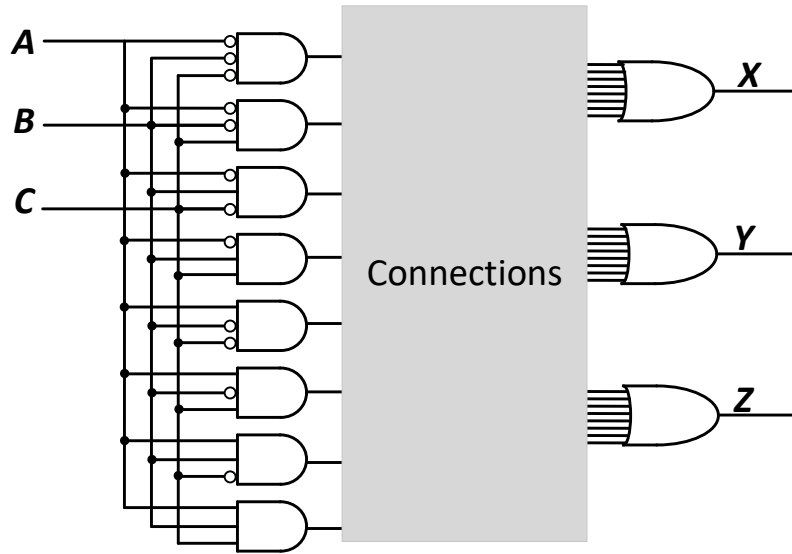
# PLA Example Function (II)



# PLA Example Function (III)

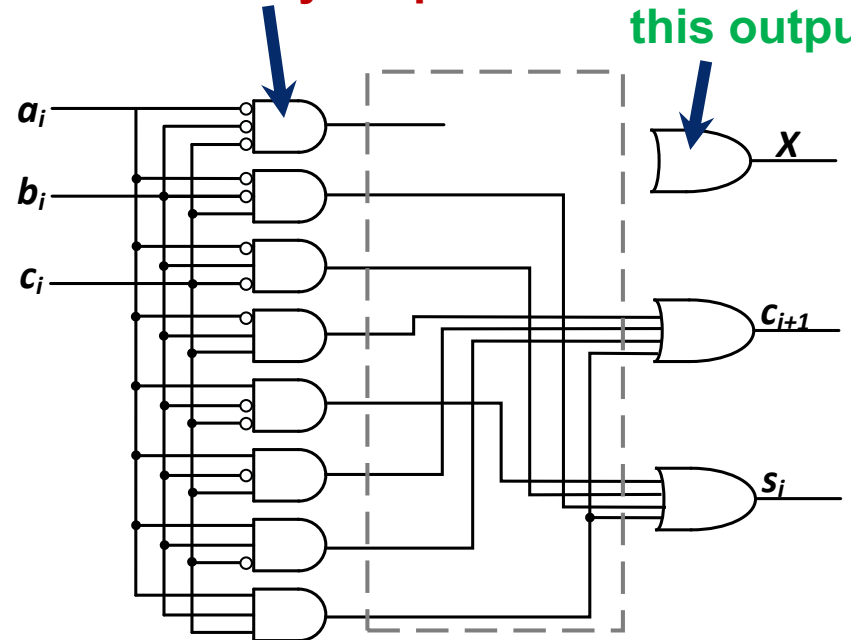


# Implementing a Full Adder Using a PLA



**This input should not be connected to any outputs**

**We do not need this output**



**Truth table of a full adder**

| $a_i$ | $b_i$ | $carry_i$ | $carry_{i+1}$ | $S_i$ |
|-------|-------|-----------|---------------|-------|
| 0     | 0     | 0         | 0             | 0     |
| 0     | 0     | 1         | 0             | 1     |
| 0     | 1     | 0         | 0             | 1     |
| 0     | 1     | 1         | 1             | 0     |
| 1     | 0     | 0         | 0             | 1     |
| 1     | 0     | 1         | 1             | 0     |
| 1     | 1     | 0         | 1             | 0     |
| 1     | 1     | 1         | 1             | 1     |

# Logical Completeness

# Logical (Functional) Completeness

---

- **Any logic function** we wish to implement could be accomplished with a PLA
  - PLA consists of **only** AND gates, OR gates, and inverters
  - We just have to program connections based on SOP of the intended logic function
- The set of gates {AND, OR, NOT} is **logically complete** because we can build a circuit to carry out the specification of **any truth table** we wish, without using any other kind of gate
- NAND is also logically complete. So is NOR.
  - **Your task:** Prove this.

# More Combinational Blocks

# More Combinational Building Blocks

---

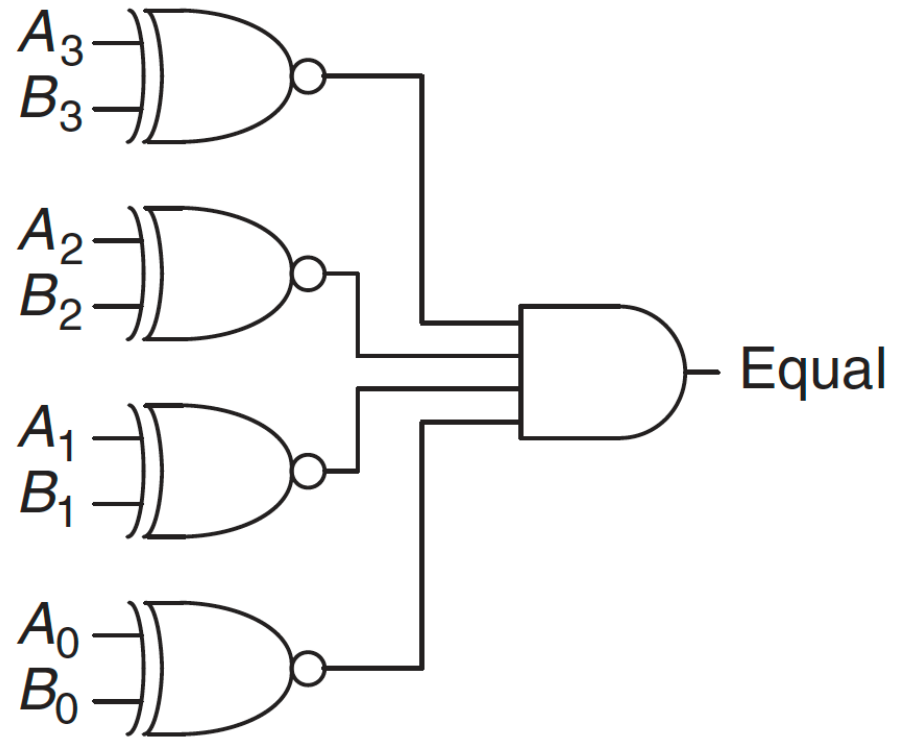
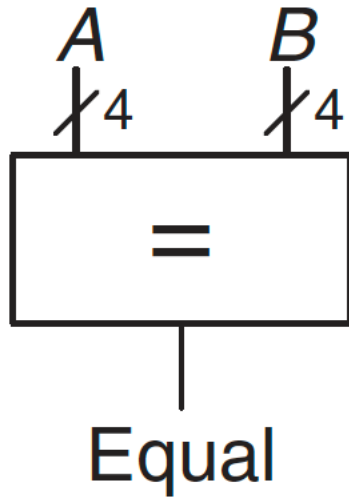
- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6
- H&H Chapter 5
  - Will be required reading soon
- You will benefit greatly by reading the “combinational” parts of Chapter 5 soon.
  - Sections 5.1 and 5.2
  - E.g., Adder, Subtractor, Comparator, Shifter/Rotator, Multiplier, Divider

# Comparator

# Equality Checker (Compare if Equal)

---

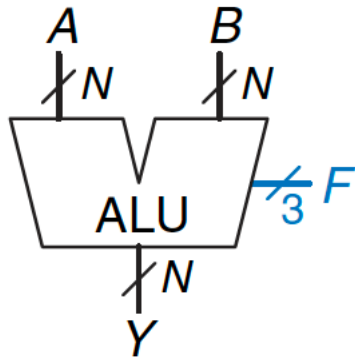
- Checks if two N-input values are exactly the same
- Example: 4-bit Comparator



# ALU (Arithmetic Logic Unit)

# ALU (Arithmetic Logic Unit)

- Combines a variety of arithmetic and logical operations into a single unit (that performs only one function at a time)
- Usually denoted with this symbol:



**Figure 5.14** ALU symbol

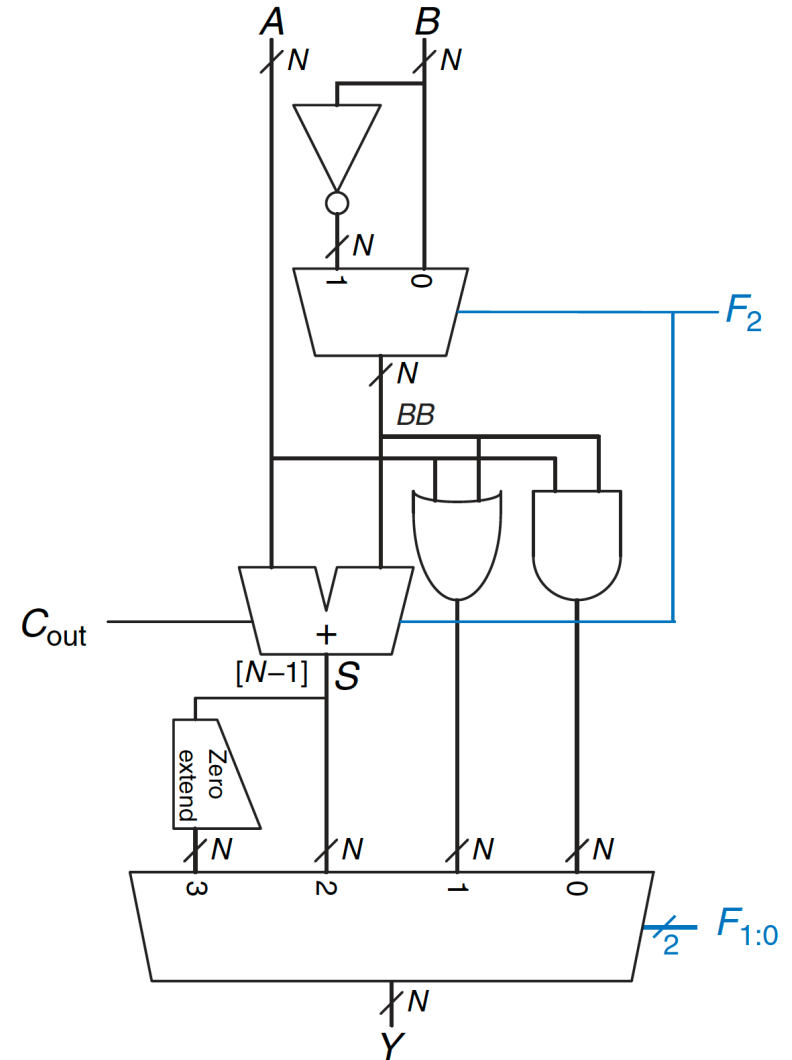
**Table 5.1** ALU operations

| $F_{2:0}$ | Function        |
|-----------|-----------------|
| 000       | A AND B         |
| 001       | A OR B          |
| 010       | A + B           |
| 011       | not used        |
| 100       | A AND $\bar{B}$ |
| 101       | A OR $\bar{B}$  |
| 110       | A - B           |
| 111       | SLT             |

# Example ALU (Arithmetic Logic Unit)

**Table 5.1 ALU operations**

| $F_{2:0}$ | Function        |
|-----------|-----------------|
| 000       | A AND B         |
| 001       | A OR B          |
| 010       | A + B           |
| 011       | not used        |
| 100       | A AND $\bar{B}$ |
| 101       | A OR $\bar{B}$  |
| 110       | A - B           |
| 111       | SLT             |



# More Combinational Building Blocks

---

- See H&H Chapter 5.2 for
  - Subtractor (using 2's Complement Representation)
  - Shifter and Rotator
  - Multiplier
  - Divider
  - ...

# More Combinational Building Blocks

---

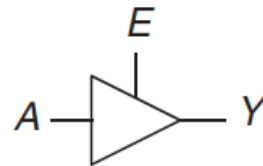
- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6
- H&H Chapter 5
  - Will be required reading soon
- You will benefit greatly by reading the “combinational” parts of Chapter 5 soon.
  - Sections 5.1 and 5.2
  - E.g., Adder, Subtractor, Comparator, Shifter/Rotator, Multiplier, Divider

# Tri-State Buffer

# Tri-State Buffer

- A tri-state buffer enables gating of different signals onto a wire

Tristate  
Buffer



| <i>E</i> | <i>A</i> | <i>Y</i> |
|----------|----------|----------|
| 0        | 0        | Z        |
| 0        | 1        | Z        |
| 1        | 0        | 0        |
| 1        | 1        | 1        |

**A tri-state buffer  
acts like a switch**

**Figure 2.40** Tristate buffer

- **Floating signal (Z):** Signal that is not driven by any circuit
  - Open circuit, floating wire

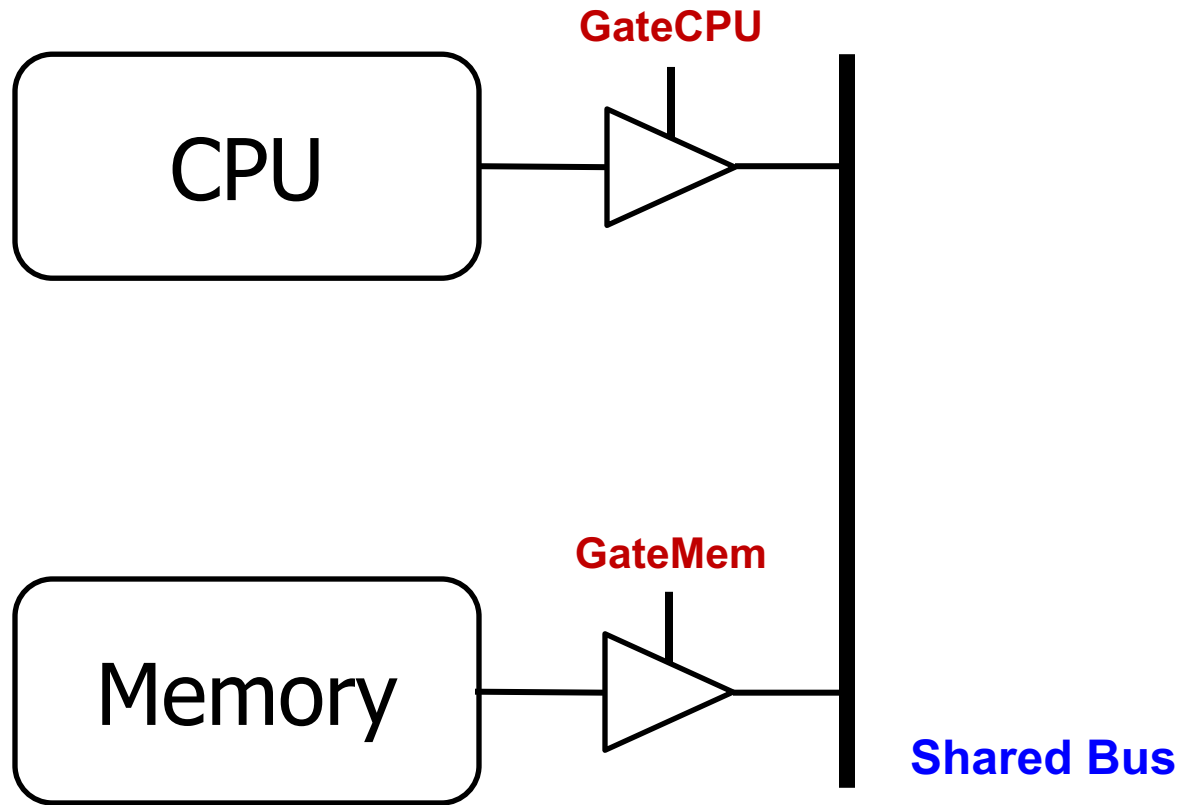
# Example: Use of Tri-State Buffers

---

- Imagine a wire connecting the CPU and memory
  - At any time only the CPU or the memory can place a value on the wire, both not both
  - You can have two tri-state buffers: one driven by CPU, the other memory; and ensure at most one is enabled at any time

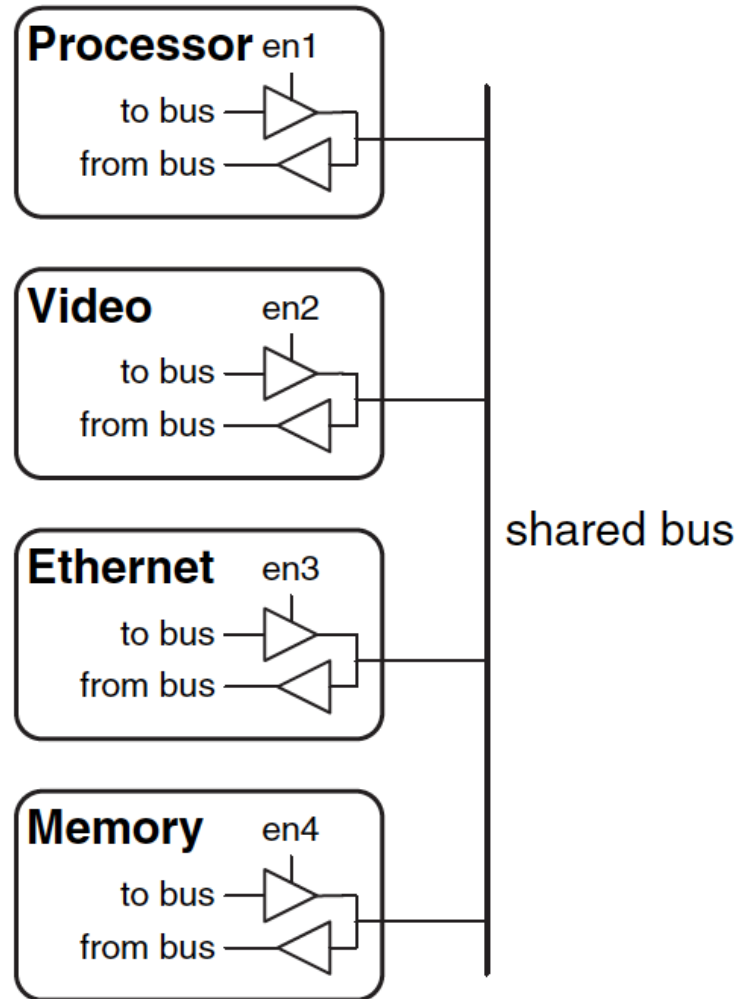
# Example Design with Tri-State Buffers

---

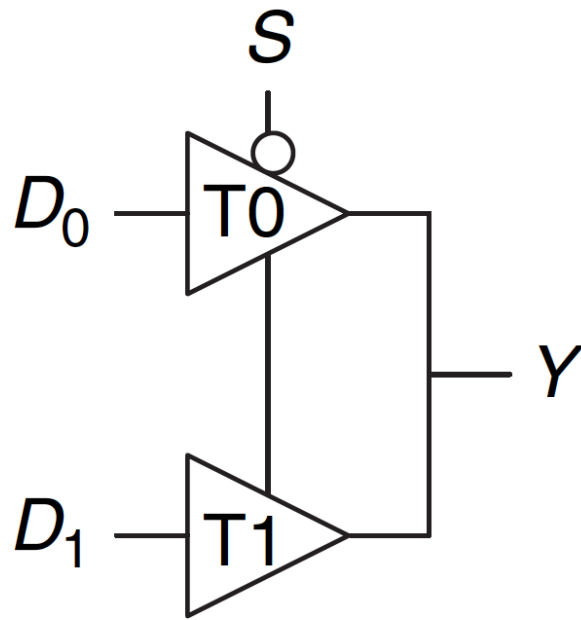


# Another Example

---

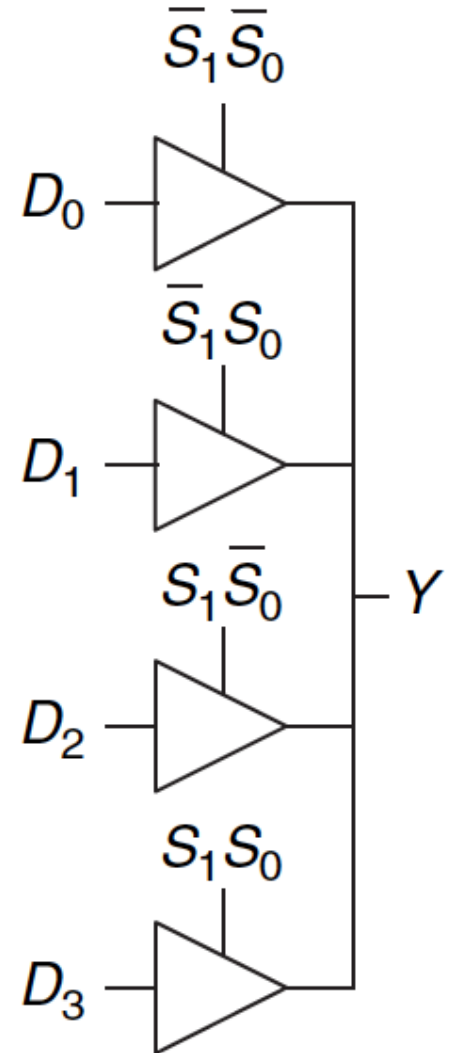


# Multiplexer Using Tri-State Buffers

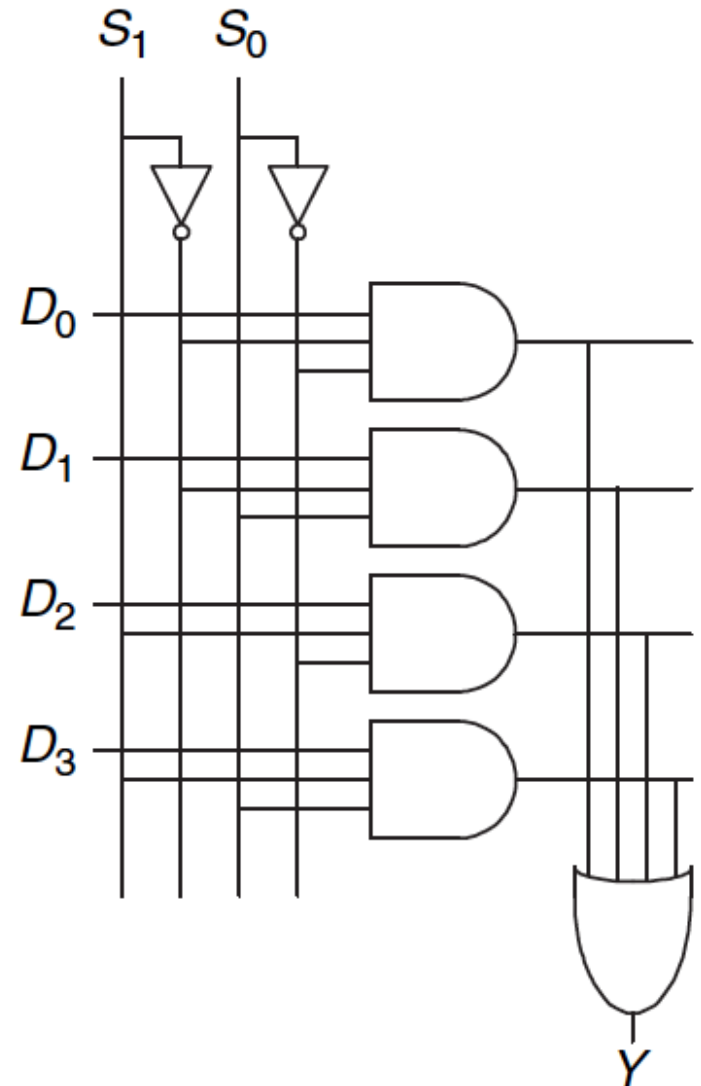
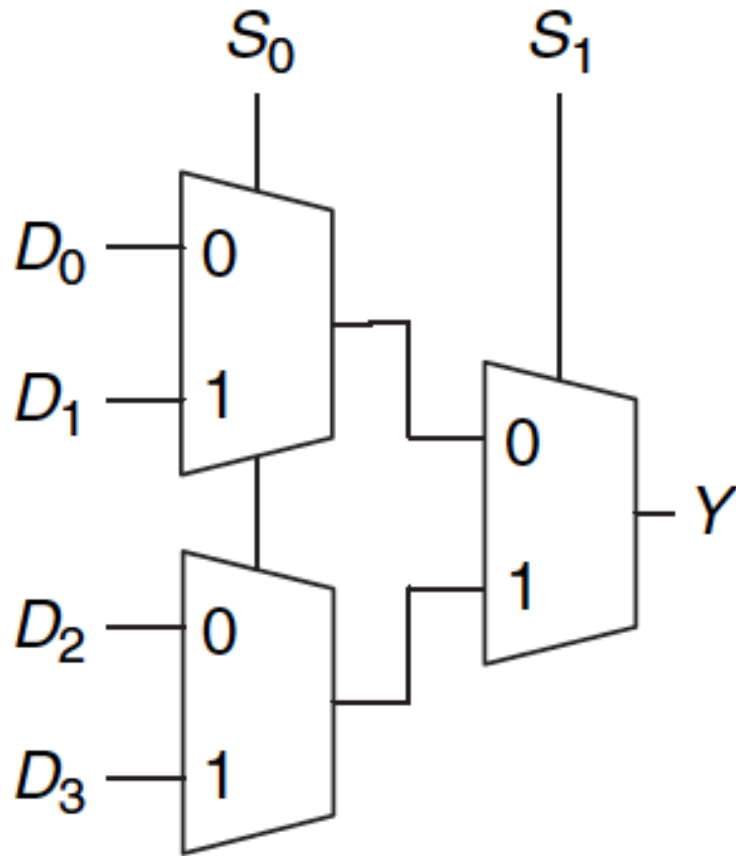


$$Y = D_0 \bar{S} + D_1 S$$

**Figure 2.56** Multiplexer using tristate buffers

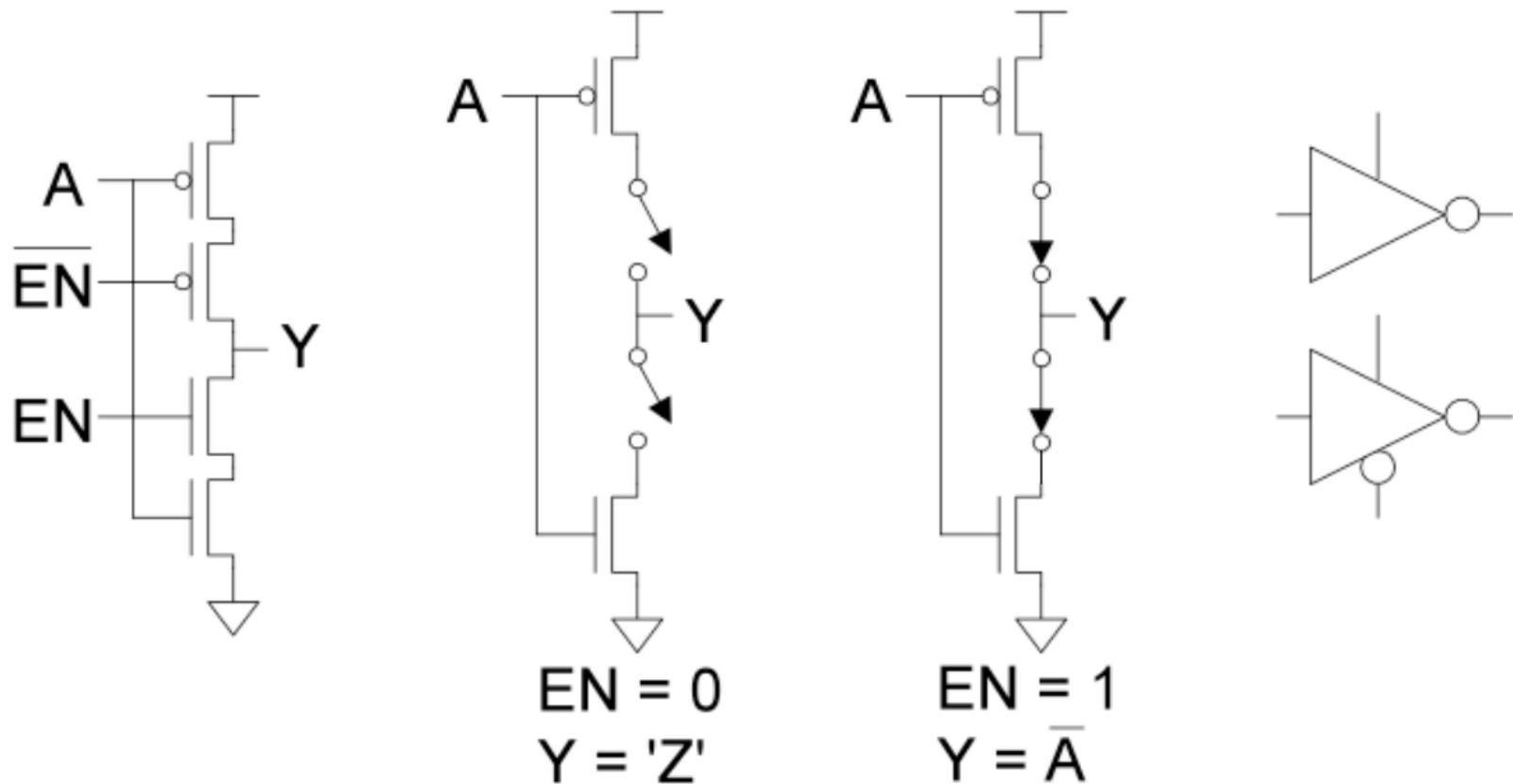


# Recall: A 4-to-1 Multiplexer



# Digging Deeper: Tri-State Buffer in CMOS

- How do you implement a Tri-State Buffer using transistors?



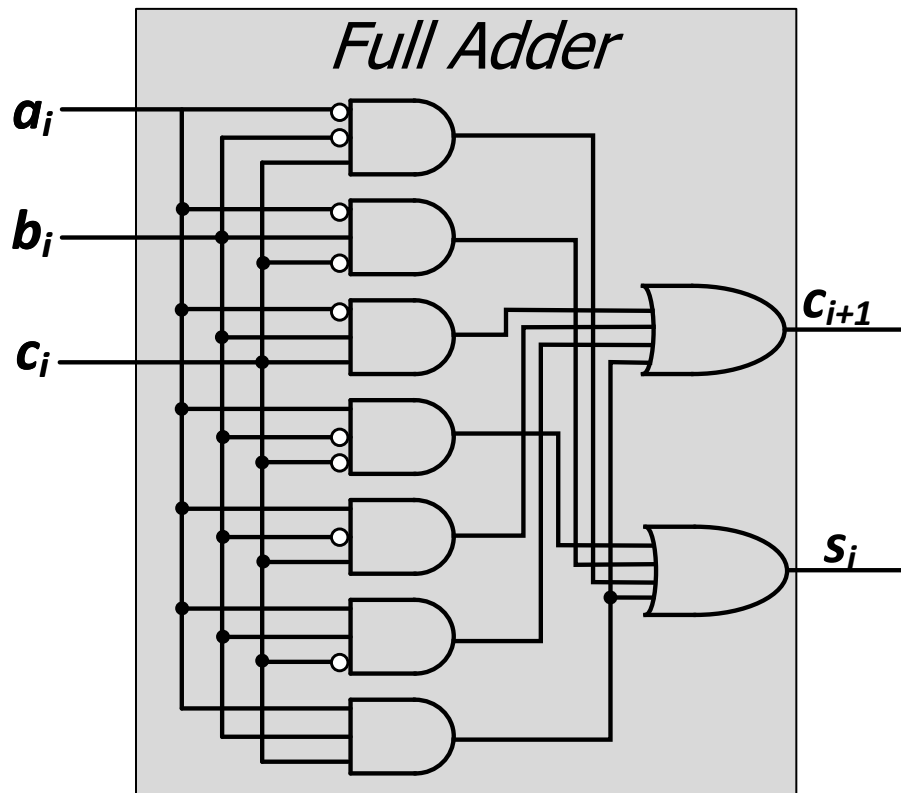
# We Covered Combinational Logic Blocks

---

- Basic logic gates (AND, OR, NOT, NAND, NOR, XOR)
  - Decoder
  - Multiplexer
  - Full Adder
  - Programmable Logic Array (PLA)
  - Comparator
  - Arithmetic Logic Unit (ALU)
  - Tri-State Buffer
- 
- Standard form representations: SOP & POS
  - Logic simplification via Boolean Algebra
  - Logical completeness

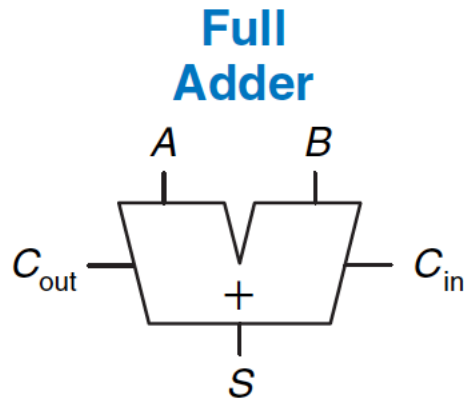
# Logic Simplification using Boolean Algebra Rules

# Recall: Full Adder in SOP Form Logic



| $a_i$ | $b_i$ | $carry_i$ | $carry_{i+1}$ | $S_i$ |
|-------|-------|-----------|---------------|-------|
| 0     | 0     | 0         | 0             | 0     |
| 0     | 0     | 1         | 0             | 1     |
| 0     | 1     | 0         | 0             | 1     |
| 0     | 1     | 1         | 1             | 0     |
| 1     | 0     | 0         | 0             | 1     |
| 1     | 0     | 1         | 1             | 0     |
| 1     | 1     | 0         | 1             | 0     |
| 1     | 1     | 1         | 1             | 1     |

# Goal: Simplified Full Adder



$$S = A \oplus B \oplus C_{\text{in}} \quad \text{3-input XOR}$$

$$C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}} \quad \text{3-input majority}$$

| $C_{\text{in}}$ | $A$ | $B$ | $C_{\text{out}}$ | $S$ |
|-----------------|-----|-----|------------------|-----|
| 0               | 0   | 0   | 0                | 0   |
| 0               | 0   | 1   | 0                | 1   |
| 0               | 1   | 0   | 0                | 1   |
| 0               | 1   | 1   | 1                | 0   |
| 1               | 0   | 0   | 0                | 1   |
| 1               | 0   | 1   | 1                | 0   |
| 1               | 1   | 0   | 1                | 0   |
| 1               | 1   | 1   | 1                | 1   |

**How do we simplify Boolean logic?**

**How do we automate simplification?**

# Quick Recap on Logic Simplification

---

- The original Boolean expression (i.e., logic circuit) may not be optimal

$$F = \sim A(A + B) + (B + AA)(A + \sim B)$$

- Can we reduce a given Boolean expression to an equivalent expression **with fewer terms**?

$$F = A + B$$

- The **goal** of logic simplification:
  - **Reduce** the number of gates/inputs
  - **Reduce** implementation cost (and potentially latency & power)

**A basis for what the automated design tools are doing today**

# Logic Simplification

## ■ Systematic techniques for simplifications

- amenable to automation

**Key Tool: The Uniting Theorem** —  $F = A\bar{B} + AB$

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

$$F = A\bar{B} + AB = A(\bar{B} + B) = A(1) = A$$

B's value changes within the rows where F=1 ("ON set")

A's value does NOT change within the ON-set rows

**If an input (B) can change without changing the output, that input value is not needed**

→ **B is eliminated, A remains**

| A | B | G |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$G = \bar{A}\bar{B} + A\bar{B} = (\bar{A} + A)\bar{B} = \bar{B}$$

B's value stays the same within the ON-set rows

A's value changes within the ON-set rows

→ **A is eliminated, B remains**

# Logic Simplification

## ■ Systematic techniques for simplifications

- amenable to automation

**Key Tool: The Uniting Theorem** —  $F = A\bar{B} + AB$

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

$$F = A\bar{B} + AB = A(\bar{B} + B) = A(1) = A$$

**Essence of Simplification:**

Find two-element subsets of the ON-set where only one variable changes its value. This single varying variable *can be eliminated!*

value is not needed

→ *B is eliminated, A remains*

| A | B | G |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$G = \bar{A}\bar{B} + A\bar{B} = (\bar{A} + A)\bar{B} = \bar{B}$$

B's value stays the same within the ON-set rows

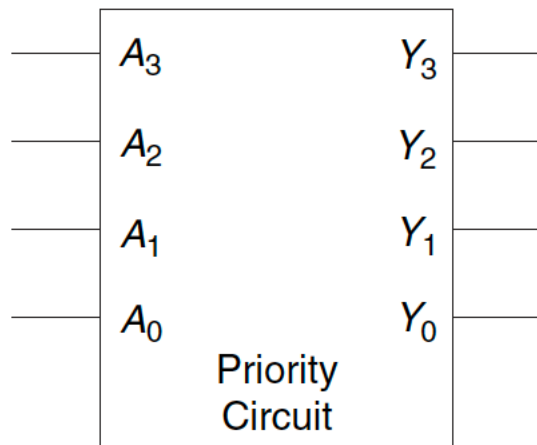
A's value changes within the ON-set rows

→ *A is eliminated, B remains*

# Logic Simplification Example: Priority Circuit

## ■ Priority Circuit

- Inputs: "Requestors" with priority levels
- Outputs: "Grant" signal for each requestor
- Example 4-bit priority circuit
- Real life example: Imagine a bus requested by 4 processors



| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1     |
| 0     | 0     | 1     | 0     | 0     | 0     | 1     | 0     |
| 0     | 0     | 1     | 1     | 0     | 0     | 1     | 0     |
| 0     | 1     | 0     | 0     | 0     | 1     | 0     | 0     |
| 0     | 1     | 0     | 1     | 0     | 1     | 0     | 0     |
| 0     | 1     | 1     | 0     | 0     | 1     | 0     | 0     |
| 0     | 1     | 1     | 1     | 0     | 1     | 0     | 0     |
| 1     | 0     | 0     | 0     | 1     | 0     | 0     | 0     |
| 1     | 0     | 0     | 1     | 1     | 0     | 0     | 0     |
| 1     | 0     | 1     | 0     | 1     | 0     | 0     | 0     |
| 1     | 0     | 1     | 1     | 1     | 0     | 0     | 0     |
| 1     | 1     | 0     | 0     | 1     | 0     | 0     | 0     |
| 1     | 1     | 0     | 1     | 1     | 0     | 0     | 0     |
| 1     | 1     | 1     | 0     | 1     | 0     | 0     | 0     |
| 1     | 1     | 1     | 1     | 1     | 0     | 0     | 0     |

# Simplified Priority Circuit

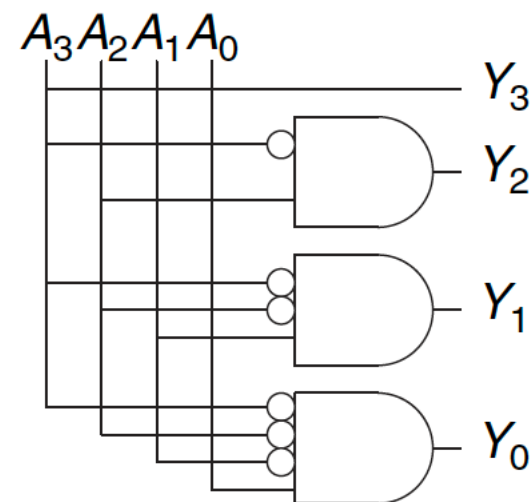
## ■ Priority Circuit

- ❑ Inputs: "Requestors" with priority levels
- ❑ Outputs: "Grant" signal for each requestor
- ❑ Example 4-bit priority circuit

| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1     |
| 0     | 0     | 1     | 0     | 0     | 0     | 1     | 0     |
| 0     | 0     | 1     | 1     | 0     | 0     | 1     | 0     |
| 0     | 1     | 0     | 0     | 0     | 1     | 0     | 0     |
| 0     | 1     | 0     | 1     | 0     | 1     | 0     | 0     |
| 0     | 1     | 1     | 0     | 0     | 1     | 0     | 0     |
| 0     | 1     | 1     | 1     | 0     | 1     | 0     | 0     |
| 1     | 0     | 0     | 0     | 1     | 0     | 0     | 0     |
| 1     | 0     | 0     | 1     | 1     | 0     | 0     | 0     |
| 1     | 0     | 1     | 0     | 1     | 0     | 0     | 0     |
| 1     | 0     | 1     | 1     | 1     | 0     | 0     | 0     |
| 1     | 1     | 0     | 0     | 1     | 0     | 0     | 0     |
| 1     | 1     | 0     | 1     | 1     | 0     | 0     | 0     |
| 1     | 1     | 1     | 0     | 1     | 0     | 0     | 0     |
| 1     | 1     | 1     | 1     | 1     | 0     | 0     | 0     |

| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1     |
| 0     | 0     | 1     | X     | 0     | 0     | 1     | 0     |
| 0     | 1     | X     | X     | 0     | 1     | 0     | 0     |
| 1     | X     | X     | X     | 1     | 0     | 0     | 0     |

**Figure 2.29** Priority circuit truth table with don't cares (X's)



**X (Don't Care)** means *I don't care what the value of this input is*

# Logic Simplification: Karnaugh Maps (K-Maps)

# Karnaugh Maps are Fun...

---

- A pictorial way of minimizing circuits by visualizing opportunities for simplification
- They are for you to **study on your own...**
  - We may cover them later if time permits
- See backup slides
- Read H&H Section 2.7
- Watch videos of Lectures 5 and 6 from 2019 DDCA course:
  - <https://youtu.be/0ks0PeaOUjE?list=PL5Q2soXY2Zi8J58xLKBNFQFHRO3GrXxA9&t=4570>
  - <https://youtu.be/ozs18ARNG6s?list=PL5Q2soXY2Zi8J58xLKBNFQFHRO3GrXxA9&t=220>

# We Are Done with Combinational Logic

---

- Building blocks of modern computers
  - Transistors
  - Logic gates
- Combinational circuits
- Boolean algebra
- Using Boolean algebra to represent combinational circuits
- Basic combinational logic blocks
- Simplifying combinational logic circuits

# Digital Design & Computer Arch.

## Lecture 2: Transistors, Gates, Combinational Logic

Prof. Onur Mutlu

ETH Zürich

Spring 2026

20 February 2026